

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN

Hệ thống quản lý học tập trực tuyến tích hợp AI hỗ trợ cá nhân hóa

AI-powered LMS for EdTech

GIANG TRUNG QUÂN

quan.gt215463@sis.hust.edu.vn

Chương trình đào tạo: IT1 - Khoa học Máy tính

Giảng viên hướng dẫn: TS. Bùi Thị Mai Anh

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN

Hệ thống quản lý học tập trực tuyến tích hợp AI hỗ trợ cá nhân hóa

AI-powered LMS for EdTech

GIANG TRUNG QUÂN

quan.gt215463@sis.hust.edu.vn

Chương trình đào tạo: IT1 - Khoa học Máy tính

Giảng viên hướng dẫn: TS. Bùi Thị Mai Anh

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành đến các thầy cô giáo, đặc biệt là **TS. Bùi Thị Mai Anh** - người đã luôn nhiệt tình hướng dẫn, chỉ bảo và hỗ trợ tôi trong suốt quá trình thực hiện đồ án tốt nghiệp.

Tiếp theo, tôi xin gửi lời cảm ơn đến tất cả thầy cô giảng viên Đại học Bách khoa Hà Nội, đặc biệt là Trường Công nghệ Thông tin và Truyền thông, nơi đã cung cấp cho tôi nền tảng kiến thức vững chắc để thực hiện đồ án này.

Cuối cùng, tôi xin gửi lời cảm ơn vô cùng đến gia đình, bố mẹ và những người thân, yêu đã luôn tin tưởng, ủng hộ và động viên tôi trong suốt quá trình học tập tại trường. Sự quan tâm và tình yêu thương của họ là nguồn động lực lớn giúp tôi vượt qua những khó khăn và thử thách trong quá trình học tập.

Tôi xin chân thành cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sự phát triển của giáo dục trực tuyến tạo điều kiện cho giáo viên cá nhân và trung tâm luyện thi quy mô nhỏ số hóa hoạt động giảng dạy. Tuy nhiên, việc vận hành bằng nhiều công cụ rời rạc làm phân tán học liệu, tăng thao tác cấp quyền và gây khó khăn khi theo dõi tiến độ của từng học sinh. Các nền tảng phổ biến như Google Classroom, Moodle và Udemy giải quyết tốt những nhóm nhu cầu riêng, nhưng không đồng thời tối ưu cho mô hình cần tự vận hành khóa học có thu phí, quản lý học tập và hỏi đáp theo học liệu riêng trên cùng một hệ thống.

Để khắc phục các hạn chế trên, đồ án xây dựng một hệ thống quản lý học tập (LMS) tập trung theo mô hình Client-Server. Hệ thống thống nhất dữ liệu khóa học, học liệu, quyền truy cập, tiến độ và kết quả đánh giá; đồng thời tích hợp các dịch vụ chuyên biệt cho xử lý media, thanh toán và hỏi đáp theo học liệu. Trong đó, kỹ thuật Sinh tăng cường truy xuất (Retrieval-Augmented Generation - RAG) được sử dụng như một chức năng hỗ trợ học sinh tự học, không phải trọng tâm duy nhất của sản phẩm.

Giải pháp của đồ án là một nền tảng học tập trực tuyến tích hợp AI, hỗ trợ đồng thời quy trình quản trị của giáo viên và quá trình học của học sinh. Hệ thống sử dụng Next.js/React ở frontend, Node.js/Express ở backend, PostgreSQL kết hợp pgvector và một dịch vụ AI bằng Python. Các chức năng chính gồm tổ chức khóa học và học liệu, xử lý video HLS, theo dõi tiến độ từng học sinh, quản lý và chấm bài kiểm tra, thông báo nghiệp vụ, cùng luồng ghi danh tự động sau khi tiếp nhận webhook thanh toán.

Đóng góp chính của đồ án là thiết kế và hiện thực một quy trình vận hành khép kín cho giáo viên hoặc trung tâm nhỏ: biên soạn và phân phối khóa học, bán và cấp quyền học, tổ chức học tập, theo dõi từng học sinh, giao và chấm bài kiểm tra. Các giải pháp RAG, HLS, kiểm soát lượt làm bài và xử lý webhook là những thành phần kỹ thuật hỗ trợ quy trình này. Sản phẩm đã được đóng gói bằng Docker và triển khai công khai qua HTTPS. Kết quả thử nghiệm cho thấy hệ thống vận hành ổn định, đáp ứng tốt các nhu cầu của giáo viên và học sinh trong mô hình tự vận hành khóa học trực tuyến có thu phí.

Sinh viên thực hiện
(*Giang Trung Quân*)

ABSTRACT

The growth of online education enables independent teachers and small test-preparation centers to digitize their teaching activities. However, using multiple disconnected tools fragments learning materials, increases manual access-control work, and makes individual learning progress difficult to monitor. Google Classroom, Moodle, and Udemy address different parts of this problem, but they are not simultaneously optimized for a small organization that wants to operate paid courses, manage learning activities, and provide course-grounded assistance in one self-managed system.

To address these constraints, this thesis develops a centralized Learning Management System (LMS) based on the client-server model. The system unifies course content, learning materials, access rights, progress, and assessment results while integrating specialized services for media processing, payment, and course-grounded assistance. Retrieval-Augmented Generation (RAG) is used as a supporting self-study feature rather than the sole focus of the product.

The proposed solution is an AI-enabled online learning platform that supports both teachers' administrative workflows and students' learning activities. It uses Next.js/React for the frontend, Node.js/Express for the backend, PostgreSQL with pgvector, and a separate Python AI service. Its main functions include structured course and material management, HLS video processing, per-student progress tracking, assessment delivery and grading, business notifications, and automated enrollment after payment webhooks.

The main contribution is the design and implementation of an end-to-end operational workflow for independent teachers and small education centers: authoring and distributing courses, selling and granting access, organizing learning activities, monitoring individual students, and delivering and grading assessments. RAG, HLS, assessment-attempt control, and payment-webhook processing are supporting technical solutions within this workflow. The product has been containerized and deployed publicly over HTTPS.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.1.1 Khảo sát nhu cầu người dùng	5
2.1.2 Khảo sát mô hình vận hành hiện tại	6
2.1.3 Khảo sát các ứng dụng tương tự.....	6
2.2 Tổng quan chức năng	8
2.2.1 Biểu đồ use case tổng quát	8
2.2.2 Biểu đồ use case phân rã Quản lý khóa học	10
2.2.3 Biểu đồ use case phân rã Học bài và xem học liệu	10
2.2.4 Biểu đồ use case phân rã Quản lý bài kiểm tra	11
2.2.5 Quy trình nghiệp vụ tạo và xuất bản bài kiểm tra	11
2.2.6 Quy trình nghiệp vụ mua khóa học và học tập	13
2.3 Đặc tả chức năng	14
2.3.1 Đặc tả use case Quản lý khóa học	16
2.3.2 Đặc tả use case Quản lý bài kiểm tra.....	17
2.3.3 Đặc tả use case Mua khóa học.....	19
2.3.4 Đặc tả use case Học bài và xem học liệu.....	20
2.3.5 Đặc tả use case Hỏi AI Tutor.....	21
2.3.6 Đặc tả use case Làm bài kiểm tra.....	22

2.4 Yêu cầu phi chức năng	24
2.4.1 Bảo mật	24
2.4.2 Hiệu năng.....	24
2.4.3 Độ tin cậy.....	25
2.4.4 Khả năng mở rộng.....	25
2.4.5 Tính dễ sử dụng	25
2.4.6 Tính dễ bảo trì	25
2.4.7 Ràng buộc công nghệ	25
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	27
3.1 Công nghệ phát triển phía máy chủ.....	27
3.1.1 Node.js, Express và TypeScript	27
3.1.2 REST API và tổ chức mã nguồn theo module	28
3.1.3 PostgreSQL và Prisma ORM.....	28
3.1.4 Zod, JWT và kiểm soát quyền truy cập.....	29
3.1.5 Transaction, số thập phân và xử lý thanh toán	29
3.1.6 Cloudflare R2 và cơ chế upload trực tiếp.....	30
3.1.7 FFmpeg, FFprobe và xử lý video HLS	30
3.1.8 Tài liệu hóa API với OpenAPI/Swagger.....	31
3.2 Công nghệ phát triển giao diện người dùng	31
3.2.1 React, Next.js và TypeScript.....	31
3.2.2 TanStack Query, Axios và Zustand	32
3.2.3 React Hook Form, Zod và kiểm tra dữ liệu phía giao diện.....	32
3.2.4 Tailwind CSS và các thư viện hỗ trợ giao diện	33
3.2.5 Video.js, hls.js và trình phát bài giảng.....	33
3.2.6 Editor Tiptap xử lý rich text	34
3.2.7 Kéo thả, thông báo và trải nghiệm quản trị	34

3.3 Nền tảng lý thuyết về RAG	35
3.3.1 Mô hình ngôn ngữ lớn	35
3.3.2 Retrieval-Augmented Generation.....	35
3.3.3 Dữ liệu ngữ cảnh và kiểm soát câu trả lời	36
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	37
4.1 Thiết kế kiến trúc.....	37
4.1.1 Lựa chọn kiến trúc phần mềm	37
4.1.2 Thiết kế tổng quan.....	39
4.1.3 Thiết kế chi tiết gói	41
4.2 Thiết kế chi tiết.....	45
4.2.1 Thiết kế giao diện	45
4.2.2 Thiết kế lớp	47
4.2.3 Thiết kế cơ sở dữ liệu	53
4.3 Xây dựng ứng dụng.....	68
4.3.1 Thư viện và công cụ sử dụng.....	68
4.3.2 Kết quả đạt được	70
4.3.3 Minh họa các chức năng chính	71
4.4 Kiểm thử.....	75
4.4.1 Kiểm thử chức năng đăng nhập	75
4.4.2 Kiểm thử chức năng mua khóa học	77
4.4.3 Kiểm thử chức năng học bài và hỏi AI	78
4.4.4 Kiểm thử theo dõi học viên và bài kiểm tra.....	79
4.4.5 Đánh giá định lượng AI Tutor RAG	80
4.4.6 Kết quả kiểm tra tự động.....	82
4.5 Triển khai	82

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	85
5.1 Trợ lý AI hỏi đáp theo học liệu nội bộ.....	85
5.1.1 Bài toán và vấn đề	85
5.1.2 Giải pháp	86
5.1.3 Kết quả đạt được và đánh giá.....	89
5.2 Kiểm soát quá trình làm bài kiểm tra	90
5.2.1 Bài toán và vấn đề	90
5.2.2 Giải pháp	90
5.2.3 Kết quả đạt được và đánh giá.....	94
5.3 Xử lý video bài giảng bằng HLS và lưu trữ đối tượng.....	95
5.3.1 Bài toán và vấn đề	95
5.3.2 Giải pháp	95
5.3.3 Kết quả đạt được và đánh giá.....	97
5.4 Tự động hóa ghi danh sau thanh toán qua webhook.....	98
5.4.1 Bài toán và vấn đề	98
5.4.2 Giải pháp	98
5.4.3 Kết quả đạt được và đánh giá.....	100
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	102
6.1 Kết luận	102
6.2 Hướng phát triển.....	103
TÀI LIỆU THAM KHẢO.....	107

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	9
Hình 2.2	Biểu đồ use case phân rã Quản lý khóa học	10
Hình 2.3	Biểu đồ use case phân rã Học bài và xem học liệu	10
Hình 2.4	Biểu đồ use case phân rã Quản lý bài kiểm tra	11
Hình 2.5	Biểu đồ hoạt động quy trình tạo và xuất bản bài kiểm tra	12
Hình 2.6	Biểu đồ hoạt động quy trình mua khóa học và học tập	14
Hình 4.1	Kiến trúc phân lớp của hệ thống	38
Hình 4.2	Biểu đồ gói tổng quan của hệ thống	40
Hình 4.3	Biểu đồ gói phân tầng phía máy chủ	42
Hình 4.4	Thiết kế chi tiết gói chức năng tạo khóa học	43
Hình 4.5	Thiết kế chi tiết gói chức năng upload video bài giảng	44
Hình 4.6	Wireframe màn hình danh sách khóa học	46
Hình 4.7	Wireframe màn hình học tập	47
Hình 4.8	Biểu đồ lớp Course	48
Hình 4.9	Biểu đồ lớp Assessment	49
Hình 4.10	Biểu đồ trình tự học bài	51
Hình 4.11	Biểu đồ trình tự hỏi đáp với AI Tutor	52
Hình 4.12	Biểu đồ trình tự tạo và xuất bản bài kiểm tra	54
Hình 4.13	Biểu đồ trình tự nộp bài kiểm tra	55
Hình 4.14	Biểu đồ thực thể liên kết của cơ sở dữ liệu	57
Hình 4.15	Màn hình thanh toán khóa học bằng mã QR	72
Hình 4.16	Không gian học tập tích hợp video, học liệu và AI Tutor	73
Hình 4.17	Màn hình xây dựng cấu trúc khóa học	74
Hình 4.18	Màn hình xây dựng bài kiểm tra từ đề PDF	74
Hình 4.19	Màn hình quản lý đơn hàng, thanh toán và quyền học	75
Hình 4.20	Màn hình quản lý giao dịch và đối soát webhook thanh toán . .	76

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh một số giải pháp EdTech với hệ thống đề xuất	8
Bảng 2.2	Danh sách tổng quan các use case của hệ thống	15
Bảng 2.3	Đặc tả use case Quản lý khóa học	16
Bảng 2.4	Đặc tả use case Quản lý bài kiểm tra	18
Bảng 2.5	Đặc tả use case Mua khóa học	19
Bảng 2.6	Đặc tả use case Học bài và xem học liệu	20
Bảng 2.7	Đặc tả use case Hỏi AI Tutor	22
Bảng 2.8	Đặc tả use case Làm bài kiểm tra	23
Bảng 4.1	Danh sách các bảng dữ liệu chính	58
Bảng 4.2	Chi tiết bảng users	59
Bảng 4.3	Chi tiết bảng media	60
Bảng 4.4	Chi tiết bảng courses	61
Bảng 4.5	Chi tiết bảng chapters	62
Bảng 4.6	Chi tiết bảng lessons	62
Bảng 4.7	Chi tiết bảng lesson_materials	63
Bảng 4.8	Chi tiết bảng enrollments	63
Bảng 4.9	Chi tiết bảng orders	64
Bảng 4.10	Chi tiết bảng payments	64
Bảng 4.11	Chi tiết bảng assessments	65
Bảng 4.12	Chi tiết bảng assessment_placements	66
Bảng 4.13	Chi tiết bảng assessment_items	66
Bảng 4.14	Chi tiết bảng submissions	67
Bảng 4.15	Chi tiết bảng knowledge_chunks	68
Bảng 4.16	Danh sách thư viện và công cụ sử dụng	68
Bảng 4.17	Thông kê quy mô mã nguồn của hệ thống	71
Bảng 4.18	Test case cho chức năng đăng nhập	76
Bảng 4.19	Test case cho chức năng mua khóa học	77
Bảng 4.20	Test case cho chức năng học bài và hỏi AI	78
Bảng 4.21	Test case theo dõi học viên và bài kiểm tra	79
Bảng 4.22	Kết quả đánh giá định lượng AI Tutor RAG	81
Bảng 4.23	Kết quả truy xuất theo nhóm câu hỏi	81
Bảng 4.24	Kết quả kiểm tra kỹ thuật trên mã nguồn	82
Bảng 4.25	Kết quả triển khai và đo thử trên môi trường public	83
Bảng 5.1	Các nhóm kết quả đối chiếu giao dịch thanh toán	100

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CDN	Mạng phân phối nội dung (Content Delivery Network)
CNTT	Công nghệ thông tin
CORS	Chia sẻ tài nguyên khác nguồn (Cross-Origin Resource Sharing)
CSDL	Cơ sở dữ liệu
EdTech	Công nghệ giáo dục (Education Technology)
HLS	Truyền phát trực tuyến qua HTTP (HTTP Live Streaming)
HMAC	Mã xác thực thông điệp dựa trên hàm băm (Hash-based Message Authentication Code)
HTTP	Giao thức truyền tải siêu văn bản (Hypertext Transfer Protocol)
HTTPS	Giao thức truyền tải siêu văn bản bảo mật (Hypertext Transfer Protocol Secure)
JWT	Chuẩn mã thông báo JSON (JSON Web Token)
LLM	Mô hình ngôn ngữ lớn (Large Language Model)
LMS	Hệ thống quản lý học tập (Learning Management System)
MIME	Định dạng mở rộng thư điện tử đa mục đích (Multipurpose Internet Mail Extensions)
ORM	Ánh xạ đối tượng - quan hệ (Object-Relational Mapping)
RAG	Sinh tăng cường truy xuất (Retrieval-Augmented Generation)
REST	Chuyển giao trạng thái biểu diễn (Representational State Transfer)

Thuật ngữ	Ý nghĩa
S3	Dịch vụ lưu trữ đối tượng đơn giản (Simple Storage Service)
SDK	Bộ công cụ phát triển phần mềm (Software Development Kit)
THPT	Trung học phổ thông
VOD	Video theo yêu cầu (Video on Demand)
VPS	Máy chủ riêng ảo (Virtual Private Server)
ĐATN	Đồ án tốt nghiệp

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Sự phát triển của học tập trực tuyến tạo ra nhu cầu mới đối với các hệ thống phần mềm hỗ trợ dạy học, quản lý học liệu và theo dõi quá trình học tập. Trong bối cảnh đó, nhiều giáo viên và nhóm luyện thi nhỏ đã bắt đầu chuyển một phần hoạt động giảng dạy lên môi trường số. Tuy nhiên, khi quy mô học sinh tăng, cách vận hành dựa trên livestream, nhóm mạng xã hội và ứng dụng nhắn tin bộc lộ nhiều hạn chế về tổ chức học liệu, quản lý học sinh và hỗ trợ người học. Chương này trình bày bối cảnh hình thành đề tài, mục tiêu cần đạt được, phạm vi triển khai và bố cục của toàn bộ báo cáo.

1.1 Đặt vấn đề

Học trực tuyến đáp ứng nhu cầu xem lại bài giảng và luyện tập ngoài giờ chính khóa, đồng thời giúp học sinh tiếp cận giáo viên mà không bị giới hạn bởi khoảng cách địa lý. Đối với các lớp luyện thi THPT Quốc gia hoặc tuyển sinh vào lớp 10, nhiều giáo viên tự do và trung tâm quy mô nhỏ đã tận dụng mạng xã hội, phần mềm họp trực tuyến và ứng dụng nhắn tin để tổ chức lớp học.

Cách tổ chức này phù hợp trong giai đoạn ban đầu vì giáo viên có thể triển khai nhanh mà không cần đầu tư hệ thống riêng. Một buổi học có thể được tổ chức qua livestream hoặc phòng họp trực tuyến, tài liệu được gửi qua nhóm chat, còn việc đăng ký khóa học được xử lý bằng chuyển khoản và duyệt thủ công. Khi số lượng học sinh tăng từ vài chục lên vài trăm hoặc vài nghìn, mô hình phân tán bắt đầu bộc lộ nhiều khó khăn và làm tăng khối lượng công việc. Giáo viên hoặc trợ lý phải kiểm tra biên lai, duyệt quyền truy cập, gửi lại tài liệu, trả lời các câu hỏi lặp lại và tổng hợp tiến độ từ nhiều nguồn khác nhau.

Vấn đề trên ảnh hưởng trực tiếp đến cả giáo viên và học sinh. Đối với giáo viên, học liệu cũ như video livestream, đề luyện tập và tài liệu lý thuyết không được tổ chức thành một cấu trúc ổn định nên khó tái sử dụng cho các khóa học sau. Đối với học sinh, việc học qua nhiều nền tảng rời rạc làm giảm khả năng theo dõi lộ trình cá nhân, đặc biệt với những học sinh tham gia sau hoặc cần xem lại bài giảng cũ. Khi không có một hệ thống tập trung, học sinh khó biết mình đã học đến đâu, còn thiếu bài tập nào và cần hỏi lại nội dung nào.

Từ thực tế đó, bài toán đặt ra là cần một hệ thống phần mềm có khả năng tập trung hóa việc tổ chức khóa học, học liệu, quyền truy cập và quá trình học tập. Hệ thống này cần phù hợp với giáo viên cá nhân hoặc nhóm giáo viên quy mô nhỏ và vừa, nơi nguồn lực vận hành còn hạn chế nhưng nhu cầu quản lý học sinh và tái sử

dung học liệu ngày càng lớn. Nếu bài toán được giải quyết, giáo viên có thể giảm bớt các thao tác thủ công, còn học sinh có thể tiếp cận nội dung học tập theo một lộ trình rõ ràng hơn.

1.2 Mục tiêu và phạm vi đề tài

Mục tiêu của đề tài là xây dựng một hệ thống quản lý học tập trực tuyến phục vụ đồng thời nhu cầu tổ chức dạy học của giáo viên và nhu cầu học tập có cấu trúc của học sinh trong môi trường EdTech. Ở phía vận hành, hệ thống hướng tới giáo viên tự do, nhóm giáo viên hoặc trung tâm luyện thi quy mô nhỏ và vừa cần một nền tảng để đóng gói bài giảng thành khóa học, quản lý học sinh và giảm khối lượng thao tác thủ công. Ở phía người học, hệ thống hướng tới học sinh cần một không gian học tập tập trung, trong đó bài giảng, tài liệu, bài đánh giá và tiến độ cá nhân được tổ chức theo một lộ trình rõ ràng.

Về chức năng, đề tài tập trung vào các nghiệp vụ cốt lõi của một hệ thống học tập trực tuyến. Hệ thống cần cho phép quản trị viên hoặc giáo viên tạo khóa học theo cấu trúc khóa học, chương và bài học; quản lý học liệu như video, tài liệu và nội dung bài viết; quản lý học sinh và quyền truy cập khóa học; theo dõi tiến độ học tập; tổ chức bài đánh giá hoặc bài luyện tập; đồng thời cung cấp chức năng trợ lý AI hỗ trợ hỏi đáp theo nội dung học liệu nội bộ. Các chức năng này được lựa chọn nhằm giải quyết trực tiếp tình trạng học liệu phân tán và khó theo dõi quá trình học tập đã nêu ở Mục 1.1.

Về phạm vi, đề tài không hướng tới việc xây dựng một nền tảng mạng xã hội học tập hoặc một sàn thương mại khóa học quy mô lớn. Hệ thống được xây dựng như một nền tảng quản lý học tập có thể triển khai cho một đơn vị giáo dục cụ thể, trong đó dữ liệu học viên, khóa học và học liệu thuộc quyền quản lý của đơn vị vận hành. Đối với chức năng AI, đề tài không đặt mục tiêu huấn luyện mô hình ngôn ngữ từ đầu, mà tập trung tích hợp mô hình hoặc dịch vụ có sẵn vào quy trình hỏi đáp dựa trên học liệu của hệ thống.

Trên cơ sở đó, đề tài hướng tới khắc phục hạn chế cốt lõi của mô hình dạy học trực tuyến phân tán: học liệu không được đóng gói có hệ thống, quyền truy cập khóa học còn phụ thuộc nhiều vào thao tác thủ công và quá trình học tập của học sinh khó được theo dõi liên tục. Điểm mới của phần mềm nằm ở việc kết hợp các chức năng quản lý học tập truyền thống với trợ lý AI dựa trên học liệu nội bộ, qua đó tạo ra một môi trường học tập tập trung cho học sinh và cung cấp hệ thống quản lý học tập hiệu quả cho giáo viên, giảm bớt các thao tác thủ công và tăng khả năng tái sử dụng học liệu.

1.3 Định hướng giải pháp

Từ mục tiêu đã xác định ở Mục 1.2, đề tài lựa chọn định hướng xây dựng hệ thống dưới dạng ứng dụng web theo mô hình Client-Server. Phần giao diện người dùng được phát triển bằng Next.js và React, phần xử lý nghiệp vụ phía máy chủ được xây dựng bằng Node.js, Express và TypeScript, còn dữ liệu nghiệp vụ được lưu trữ bằng PostgreSQL thông qua Prisma ORM. Định hướng này phù hợp với một hệ thống LMS vì giao diện học tập, API nghiệp vụ và dữ liệu có thể được tách biệt rõ ràng, giúp hệ thống dễ bảo trì và mở rộng theo từng phần.

Đối với chức năng AI, đề tài lựa chọn kỹ thuật RAG để xây dựng trợ lý hỏi đáp theo học liệu nội bộ. Theo định hướng này, học liệu của giáo viên được trích xuất, chia nhỏ và biểu diễn dưới dạng vector embedding; khi học sinh đặt câu hỏi, hệ thống truy xuất các đoạn học liệu liên quan để cung cấp ngữ cảnh cho mô hình ngôn ngữ tạo câu trả lời. Cách tiếp cận này giúp chức năng hỏi đáp bám sát nội dung bài học hơn so với việc sử dụng mô hình ngôn ngữ độc lập, đồng thời không yêu cầu huấn luyện mô hình từ đầu.

Giải pháp của đề án là xây dựng một nền tảng quản lý học tập trực tuyến tích hợp các nghiệp vụ cốt lõi gồm tổ chức khóa học, quản lý học liệu, quản lý người học, theo dõi tiến độ, quản lý quyền truy cập, tổ chức bài đánh giá và hỗ trợ hỏi đáp bằng trợ lý AI. Hệ thống được thiết kế theo hướng phân tầng để tách biệt phần giao diện, phần điều phối nghiệp vụ, phần truy cập dữ liệu và phần tích hợp dịch vụ ngoài. Cách tổ chức này giúp các nhóm chức năng như khóa học, bài học, đánh giá, thanh toán và trợ lý AI có thể phát triển tương đối độc lập nhưng vẫn hoạt động trong cùng một nền tảng.

Đóng góp chính của đề tài nằm ở việc đưa các nghiệp vụ thường bị phân tán trong mô hình dạy học trực tuyến quy mô nhỏ vào một hệ thống tập trung, đồng thời bổ sung cơ chế hỏi đáp theo học liệu nội bộ để hỗ trợ học sinh trong quá trình tự học. Kết quả hướng tới là một hệ thống LMS có khả năng phục vụ đồng thời giáo viên, học sinh và người quản trị, trong đó nội dung học tập, tiến độ, quyền truy cập và hỗ trợ học tập được quản lý thống nhất hơn so với cách vận hành thủ công trên nhiều công cụ rời rạc.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày quá trình khảo sát và phân tích yêu cầu hệ thống. Chương này phân tích hiện trạng dạy học trực tuyến phân tán, xác định các nhóm người dùng chính, khảo sát một số nền tảng có liên quan và rút ra các yêu cầu chức năng,

phi chức năng cần đáp ứng. Trên cơ sở đó, chương này trình bày tổng quan chức năng, biểu đồ use case và đặc tả một số use case quan trọng của hệ thống.

Chương 3 trình bày các nền tảng lý thuyết và công nghệ được sử dụng trong quá trình thực hiện đề án. Chương này giới thiệu các công nghệ chính như Next.js, React, Node.js, Express, TypeScript, PostgreSQL, Prisma ORM và kỹ thuật RAG, đồng thời giải thích lý do lựa chọn các công nghệ này trong mối liên hệ với yêu cầu đã xác định ở Chương 2.

Chương 4 trình bày phân phân tích, thiết kế, triển khai và đánh giá hệ thống. Nội dung chương bao gồm thiết kế kiến trúc tổng quan, thiết kế chi tiết các module chính, thiết kế giao diện, thiết kế cơ sở dữ liệu, quá trình xây dựng ứng dụng, kết quả đạt được, kiểm thử và mô tả triển khai. Chương này cũng trình bày các hình ảnh minh họa chức năng chính để làm rõ cách hệ thống được hiện thực hóa.

Chương 5 trình bày các giải pháp và đóng góp nổi bật của đề án, gồm pipeline RAG cho AI Tutor, kiểm soát quá trình làm bài kiểm tra, xử lý và phân phối video HLS, cùng cơ chế xử lý webhook thanh toán an toàn. Mỗi nội dung được trình bày theo cấu trúc khó khăn, giải pháp thực hiện và kết quả đạt được.

Chương 6 trình bày kết luận và hướng phát triển của đề án. Chương này tổng kết các kết quả đã đạt được so với mục tiêu ban đầu, chỉ ra những hạn chế còn tồn tại trong phạm vi thực hiện và đề xuất một số hướng mở rộng trong tương lai, đặc biệt là khả năng triển khai thực tế, mở rộng quy mô người dùng và cải thiện các chức năng hỗ trợ học tập bằng AI.

Chương này đã trình bày bối cảnh hình thành đề tài, bài toán cần giải quyết, mục tiêu, phạm vi và định hướng giải pháp ở mức tổng quan. Từ các vấn đề đã nêu, chương tiếp theo sẽ khảo sát hiện trạng và phân tích yêu cầu để xác định cụ thể các chức năng cần có của hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã xác định bài toán chính của đề tài là tình trạng học liệu phân tán, quy trình vận hành thủ công và thiếu công cụ theo dõi quá trình học tập trong mô hình dạy học trực tuyến quy mô nhỏ và vừa. Trên cơ sở đó, chương này tiến hành khảo sát hiện trạng để làm rõ nhu cầu của các nhóm người dùng, phân tích mô hình vận hành đang được sử dụng và so sánh với một số hệ thống tương tự. Kết quả khảo sát là cơ sở để xác định các chức năng và yêu cầu cần có của hệ thống ở các mục tiếp theo.

2.1 Khảo sát hiện trạng

Mục tiêu của phần này là làm rõ bối cảnh thực tế mà hệ thống cần phục vụ. Nội dung khảo sát được xem xét từ ba nguồn chính: nhu cầu của người dùng, mô hình vận hành hiện tại và các ứng dụng tương tự trên thị trường. Việc xác định yêu cầu không xuất phát từ ý kiến chủ quan của người phát triển, mà từ các vấn đề đang tồn tại trong quá trình dạy và học trực tuyến.

2.1.1 Khảo sát nhu cầu người dùng

Trong phạm vi đề tài, hệ thống hướng tới ba nhóm người dùng chính gồm học sinh, giáo viên và quản trị viên. Mỗi nhóm tham gia vào hệ thống với mục tiêu khác nhau, do đó nhu cầu của từng nhóm cũng cần được phân tích riêng để tránh thiết kế chức năng theo hướng quá chung chung.

Đối với giáo viên, nhu cầu quan trọng nhất là tổ chức lại học liệu giảng dạy thành một cấu trúc có thể tái sử dụng. Trong mô hình dạy học trực tuyến hiện nay, video bài giảng, tài liệu PDF, đề luyện tập và thông báo thường được lưu ở nhiều nơi khác nhau. Giáo viên cần một công cụ cho phép đóng gói các tài nguyên này thành khóa học có cấu trúc, quản lý chương và bài học, tạo bài kiểm tra, theo dõi kết quả học tập của học sinh và giảm bớt các thao tác vận hành lặp lại.

Đối với học sinh, nhu cầu chính là có một không gian học tập tập trung và có lộ trình rõ ràng. Học sinh cần dễ dàng tìm lại video bài giảng, truy cập tài liệu đi kèm, biết mình đã học đến bài nào và thực hiện bài kiểm tra để tự đánh giá mức độ nắm vững kiến thức. Bên cạnh đó, trong quá trình tự học, học sinh cũng cần một kênh hỗ trợ nhanh cho các câu hỏi liên quan trực tiếp đến nội dung bài học, đặc biệt khi giáo viên không thể phản hồi ngay.

Đối với quản trị viên, nhu cầu tập trung vào vận hành và kiểm soát hệ thống. Vai trò này cần quản lý tài khoản người dùng, tạo tài khoản giáo viên, theo dõi đơn hàng, kiểm tra trạng thái thanh toán và xử lý các giao dịch phát sinh lỗi, có thể thao

tác xử lý thủ công. Khi số lượng học sinh tăng, các thao tác này nếu được xử lý thủ công sẽ tốn nhiều thời gian và dễ gây sai sót, do đó hệ thống cần hỗ trợ quản trị viên theo dõi dữ liệu vận hành một cách tập trung.

2.1.2 Khảo sát mô hình vận hành hiện tại

Mô hình vận hành hiện tại của nhiều giáo viên tự do hoặc trung tâm luyện thi quy mô nhỏ thường dựa trên sự kết hợp của nhiều công cụ rời rạc. Việc dạy học được tổ chức qua livestream trong nhóm mạng xã hội hoặc qua các phần mềm họp trực tuyến. Sau buổi học, video có thể được ghi lại hoặc được lưu trực tiếp trong nhóm Facebook của lớp học. Khi một học sinh mới tham gia nhóm, em phải tự tìm lại các bài đăng cũ, cuộn qua nhiều thông báo và bản ghi livestream để xác định đâu là bài học đầu tiên, đâu là bài tiếp theo và tài liệu nào đi kèm với từng buổi học. Trải nghiệm này cho thấy video bài giảng tuy vẫn tồn tại trên nền tảng cũ, nhưng không được tổ chức thành một cấu trúc khóa học rõ ràng để học sinh có thể theo dõi lộ trình học tập.

Học liệu đi kèm bài giảng cũng thường được phân phối riêng qua các nhóm nhắn tin hoặc dịch vụ lưu trữ đám mây. Cách làm này giúp giáo viên triển khai nhanh, nhưng về lâu dài làm tăng tình trạng phân tán thông tin. Một học sinh muốn học lại một chủ đề cụ thể phải tìm video ở một nơi, tài liệu ở một nơi khác và thông báo lịch học ở một kênh khác. Sự phân tán này làm giảm tính liên tục của quá trình học tập và khiến học sinh khó tự quản lý tiến độ cá nhân.

Quy trình mua khóa học và cấp quyền truy cập trong mô hình hiện tại cũng phụ thuộc nhiều vào thao tác thủ công. Học sinh thường chuyển khoản, gửi ảnh chụp giao dịch, sau đó giáo viên hoặc trợ lý đối soát và duyệt học sinh vào nhóm học. Khi số lượng học sinh còn nhỏ, quy trình này có thể chấp nhận được. Tuy nhiên, khi quy mô lớp học tăng lên, việc kiểm tra biên lai, xác nhận thanh toán và cấp quyền truy cập thủ công dễ gây chậm trễ hoặc nhầm lẫn.

Hoạt động kiểm tra và đánh giá cũng gặp vấn đề tương tự. Bài tập hoặc đề luyện tập có thể được gửi dưới dạng tệp, học sinh làm bài rồi nộp lại qua tin nhắn, biểu mẫu hoặc thư mục lưu trữ. Giáo viên và trợ lý phải tổng hợp kết quả từ nhiều nguồn, dẫn đến khó theo dõi lịch sử làm bài, tiến độ hoàn thành và mức độ tiến bộ của từng học sinh. Vì vậy, mô hình vận hành hiện tại thiếu một cơ chế tập trung để kết nối học liệu, quyền truy cập, tiến độ học tập và kết quả đánh giá.

2.1.3 Khảo sát các ứng dụng tương tự

Để định vị giải pháp của đề tài, phần này khảo sát ba hệ thống đại diện cho ba hướng tiếp cận phổ biến trong lĩnh vực EdTech: quản lý lớp học trực tuyến, hệ quản trị học tập tổng quát và nền tảng thương mại hóa khóa học. Ba hệ thống được lựa

chọn gồm Google Classroom, Moodle và Udemy.

Google Classroom là công cụ quản lý lớp học trực tuyến hỗ trợ giáo viên tổ chức lớp, giao bài, phản hồi và theo dõi hoạt động học tập trong hệ sinh thái Google [1]. Google cũng đã tích hợp Gemini vào Classroom với các chức năng hỗ trợ tạo học liệu và hỗ trợ học tập, tùy thuộc phiên bản, giấy phép và điều kiện sử dụng [2]. Tuy nhiên, Classroom không tập trung vào quy trình bán khóa học và ghi danh tự động sau thanh toán của một đơn vị đào tạo nhỏ.

Moodle là hệ quản trị học tập mã nguồn mở có phạm vi chức năng rộng và khả năng tự triển khai. Từ phiên bản 4.5, Moodle cung cấp AI subsystem để kết nối nhiều nhà cung cấp mô hình và hỗ trợ các tác vụ như sinh, tóm tắt hoặc giải thích nội dung [3]. Đối lại, đơn vị sử dụng phải tự lựa chọn hạ tầng, cấu hình nhà cung cấp AI và vận hành hệ thống; đây là khối lượng công việc đáng kể đối với giáo viên cá nhân hoặc trung tâm nhỏ.

Udemy là nền tảng phân phối khóa học theo mô hình marketplace, có sẵn hạ tầng bán khóa học và tệp người học lớn. Udemy AI Assistant có thể giải thích, tóm tắt và trả lời dựa trên nội dung giảng viên đối với các khóa học, ngôn ngữ và chương trình được hỗ trợ [4]. Tuy nhiên, giáo viên vận hành trong quy tắc của marketplace, không tự triển khai hệ thống và không kiểm soát toàn bộ mô hình dữ liệu hoặc quy trình nghiệp vụ riêng.

Để hạn chế đánh giá cảm tính, Bảng 2.1 sử dụng các tiêu chí chức năng có thể kiểm chứng từ tài liệu chính thức. Các giá trị “Có điều kiện” chỉ ra rằng năng lực phụ thuộc giấy phép, cấu hình nhà cung cấp hoặc phạm vi khóa học được hỗ trợ; chúng không đồng nghĩa với việc nền tảng không có chức năng đó.

Bảng 2.1: So sánh một số giải pháp EdTech với hệ thống đề xuất

Tiêu chí so sánh	Google Classroom	Moodle	Udemy	Hệ thống đề xuất
Cấu trúc khóa học và học liệu	Có	Có	Có	Có
Giao bài/kiểm tra và theo dõi kết quả	Có	Có	Có giới hạn theo loại nội dung	Có
Thanh toán gắn với ghi danh	Không phải chức năng lõi	Qua mở rộng/tích hợp	Có	Có
Tự triển khai và kiểm soát mô hình dữ liệu	Không	Có	Không	Có
AI hỗ trợ theo nội dung khóa học	Có điều kiện	Có điều kiện	Có điều kiện	Có, theo học liệu đã lập chỉ mục
Quy trình riêng cho trung tâm nhỏ	Cần ghép thêm công cụ	Cần cấu hình	Theo quy trình marketplace	Là phạm vi thiết kế

Kết quả khảo sát cho thấy mỗi hệ thống hiện có giải quyết tốt một phần của bài toán. Google Classroom phù hợp với quản lý lớp học cơ bản, Moodle mạnh ở phạm vi chức năng của một LMS tổng quát, còn Udemy hỗ trợ tốt việc phân phối khóa học theo mô hình marketplace. Tuy nhiên, các giải pháp này chưa đồng thời đáp ứng nhu cầu triển khai một LMS tinh gọn cho giáo viên hoặc trung tâm nhỏ, trong đó khóa học, học liệu, quyền truy cập, tiến độ học tập, bài kiểm tra và AI Tutor theo học liệu nội bộ được vận hành trong cùng một hệ thống.

2.2 Tổng quan chức năng

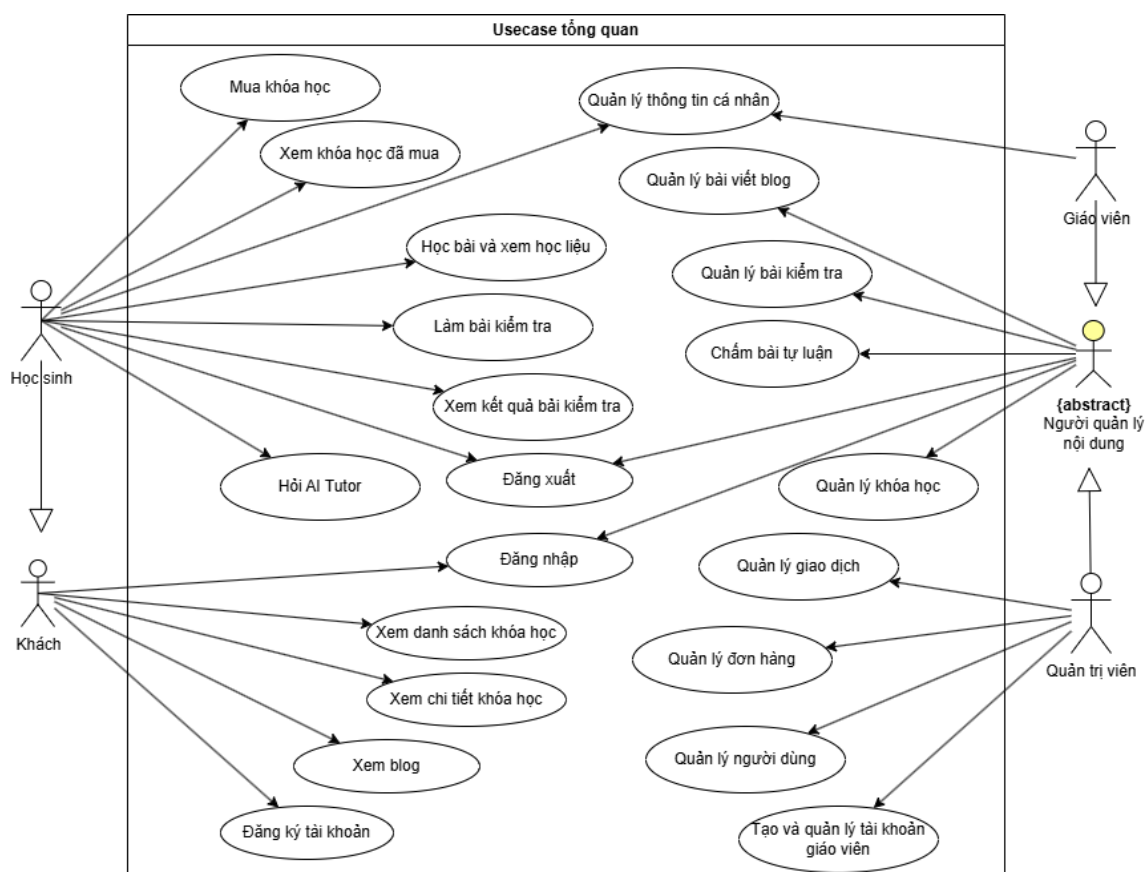
Từ kết quả khảo sát ở Mục 2.1, hệ thống được định hướng xây dựng quanh các nghiệp vụ chính của một nền tảng học tập trực tuyến: công khai thông tin khóa học, quản lý người dùng, quản lý nội dung học tập, mua khóa học, học bài, làm bài kiểm tra và hỗ trợ hỏi đáp bằng AI Tutor.

2.2.1 Biểu đồ use case tổng quát

Hệ thống bao gồm bốn tác nhân nghiệp vụ chính tham gia vào các use case, mỗi tác nhân có vai trò và quyền hạn khác nhau. Tác nhân Khách truy cập là người dùng chưa đăng nhập vào hệ thống, có thể xem danh sách khóa học, xem chi tiết khóa học, xem blog, đăng ký tài khoản học sinh và đăng nhập. Tác nhân Học sinh là người dùng đã đăng ký tài khoản học tập, kế thừa các quyền xem nội dung công khai của Khách truy cập và có thêm các chức năng như quản lý thông tin cá nhân,

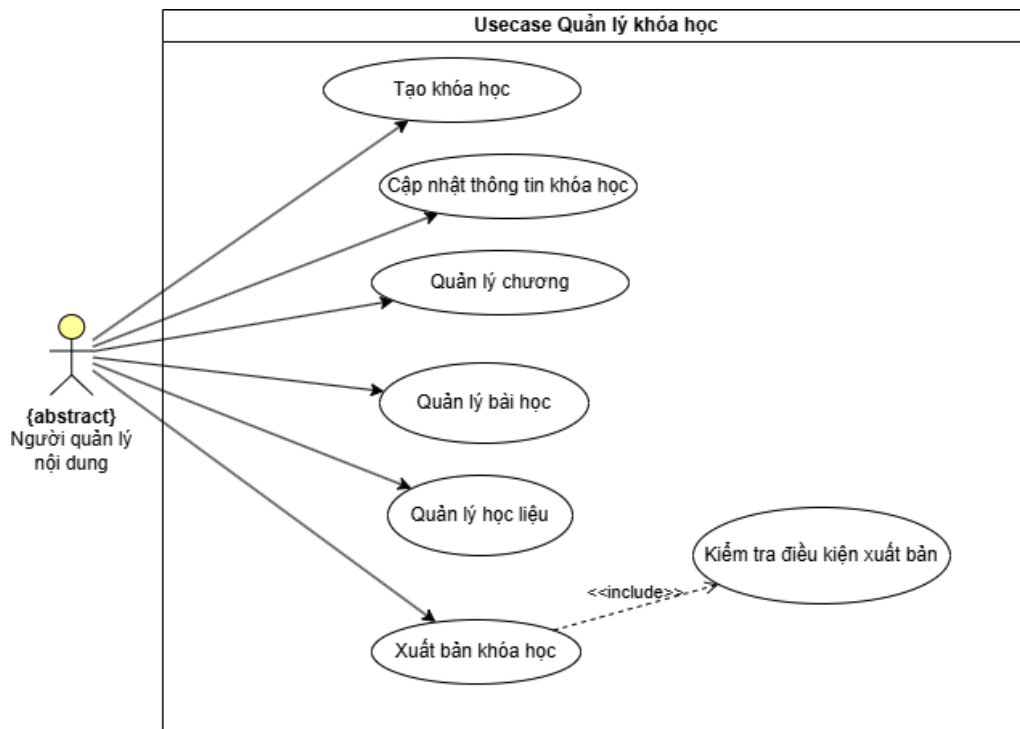
mua khóa học, học bài và xem học liệu, làm bài kiểm tra, xem kết quả bài kiểm tra và hỏi AI Tutor. Tác nhân Giáo viên phụ trách nội dung chuyên môn, có thể quản lý khóa học, quản lý bài kiểm tra, chấm điểm bài làm, xem kết quả học tập của học sinh và quản lý bài viết blog. Tác nhân Quản trị viên có quyền vận hành hệ thống tổng thể, bao gồm quản lý người dùng, tạo và quản lý tài khoản giáo viên, quản lý khóa học, quản lý bài kiểm tra, quản lý đơn hàng, quản lý giao dịch và quản lý bài viết blog. Trong biểu đồ, tác nhân Người quản lý nội dung được sử dụng như một tác nhân trừu tượng để gom nhóm các chức năng quản lý nội dung dùng chung giữa Giáo viên và Quản trị viên; tác nhân này không đại diện cho một loại tài khoản đăng nhập độc lập trong hệ thống.

Hình 2.1 trình bày biểu đồ use case tổng quát của hệ thống, thể hiện mối quan hệ giữa các tác nhân và các nhóm chức năng chính.



Hình 2.1: Biểu đồ use case tổng quát của hệ thống

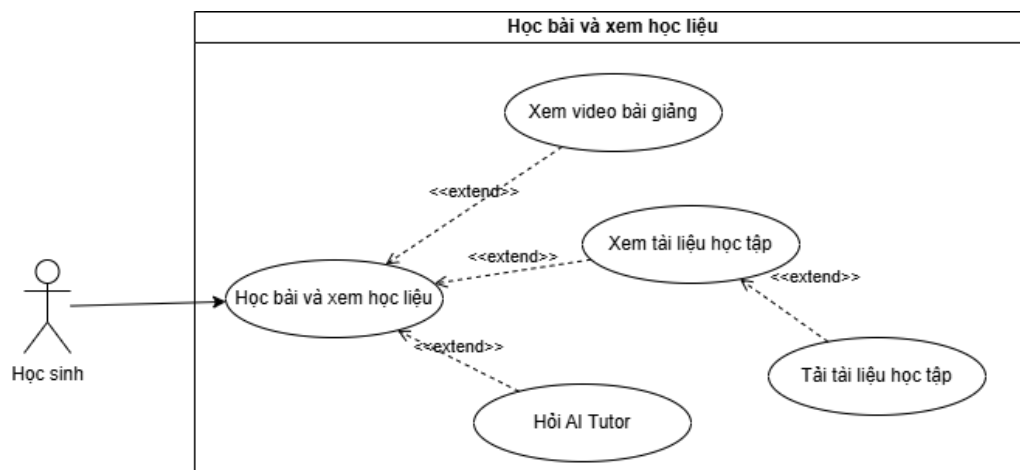
2.2.2 Biểu đồ use case phân rã Quản lý khóa học



Hình 2.2: Biểu đồ use case phân rã Quản lý khóa học

Hình 2.2 tập trung phân rã nhánh quản lý nội dung của tác nhân Người quản lý nội dung, gồm tạo khóa học, cập nhật thông tin, quản lý chương, bài học, học liệu và xuất bản. Ngoài nhánh này, màn hình quản lý khóa học còn có nhánh quản lý học viên: xem danh sách ghi danh, tiến độ tổng quan và chi tiết từng bài học, bài kiểm tra của mỗi học sinh. Việc tách hai tab giúp giáo viên vừa biên soạn nội dung, vừa theo dõi các trường hợp chưa học hoặc chưa nộp bài để có biện pháp nhắc nhở.

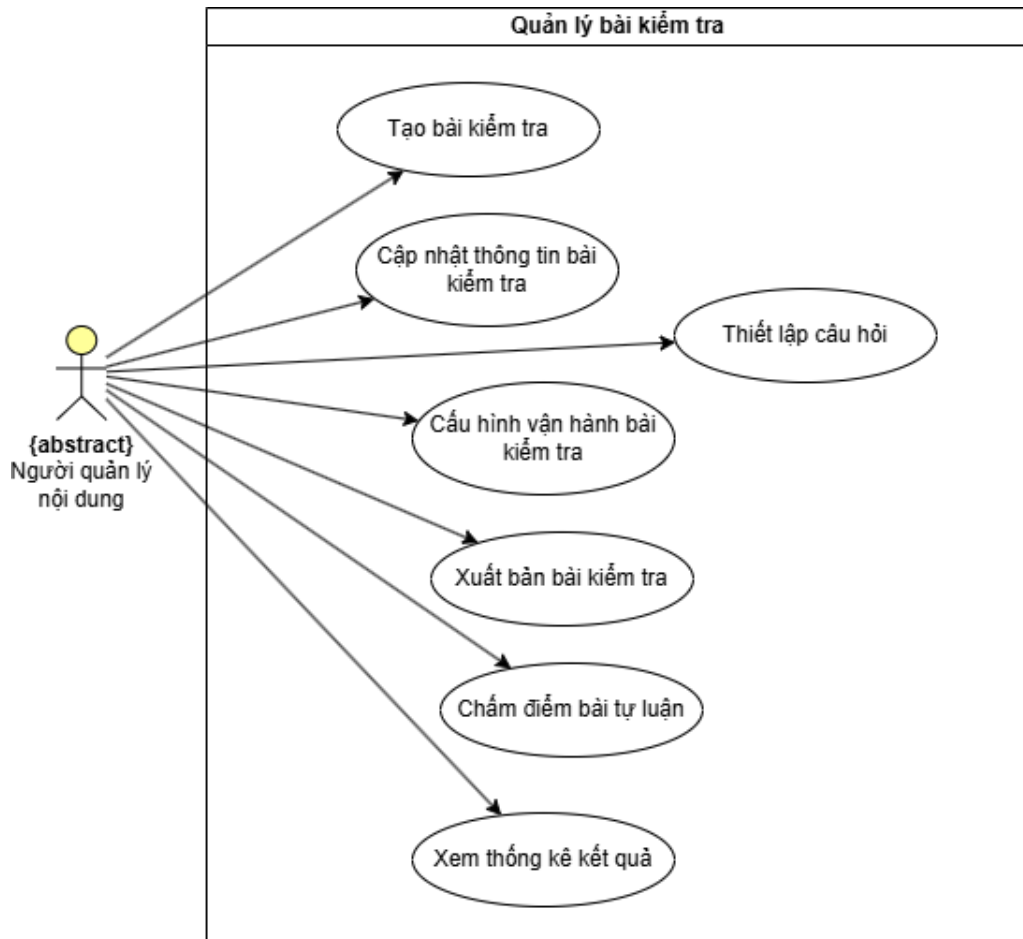
2.2.3 Biểu đồ use case phân rã Học bài và xem học liệu



Hình 2.3: Biểu đồ use case phân rã Học bài và xem học liệu

Hình 2.3 phân rã use case Học bài và xem học liệu của tác nhân Học sinh. Trong phạm vi bài học, học sinh có thể xem video bài giảng, xem tài liệu học tập, tải tài liệu học tập và hỏi AI Tutor. Các xử lý nội bộ như kiểm tra quyền truy cập và ghi nhận tiến độ không được biểu diễn thành use case riêng.

2.2.4 Biểu đồ use case phân rã Quản lý bài kiểm tra

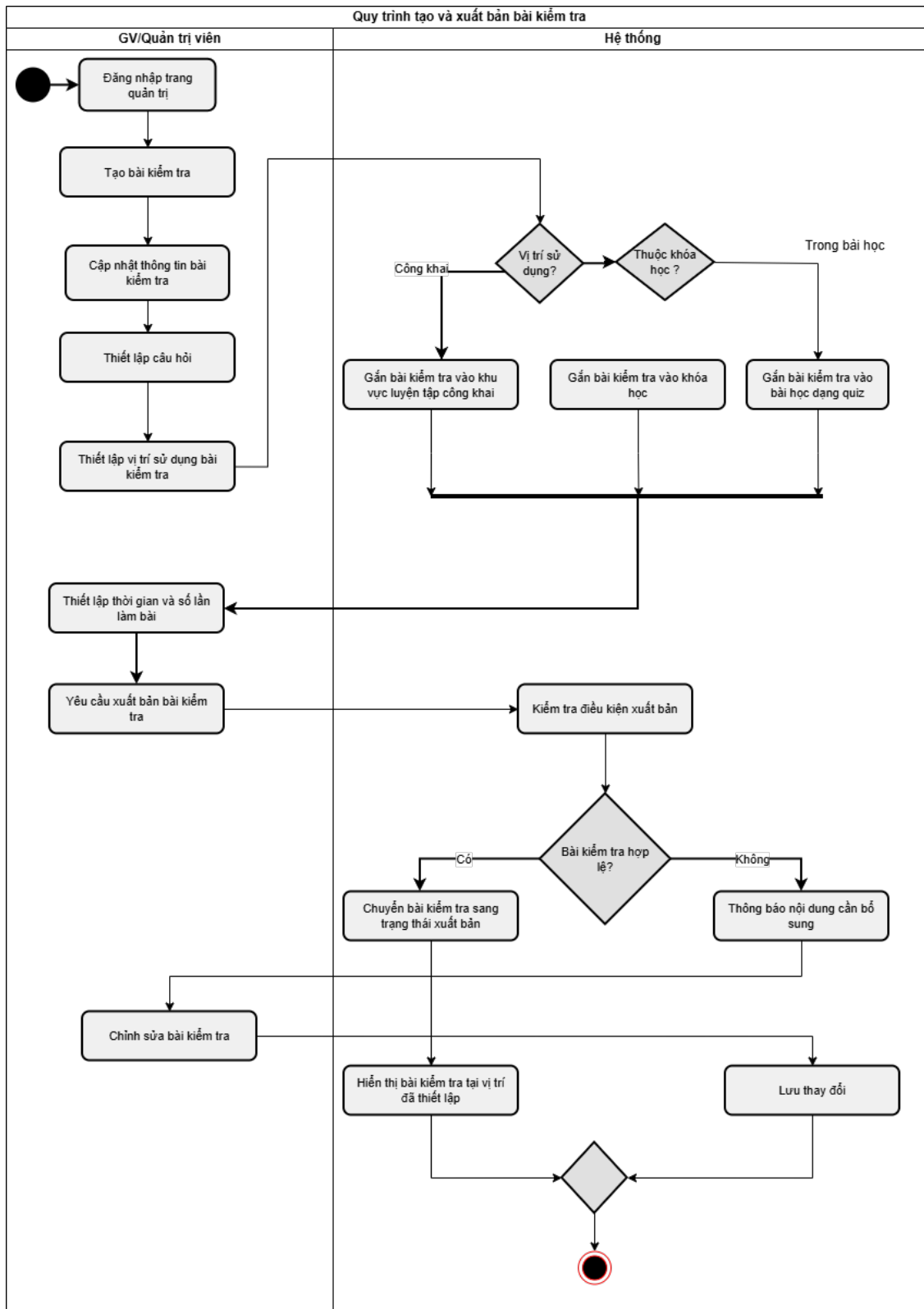


Hình 2.4: Biểu đồ use case phân rã Quản lý bài kiểm tra

Hình 2.4 phân rã use case Quản lý bài kiểm tra của tác nhân Người quản lý nội dung. Nhóm chức năng này gồm tạo bài kiểm tra, cập nhật thông tin bài kiểm tra, thiết lập câu hỏi, cấu hình vận hành bài kiểm tra, xuất bản bài kiểm tra, chấm điểm bài tự luận và xem thống kê kết quả. Cấu hình vận hành bài kiểm tra bao gồm các thiết lập như thời gian làm bài, thời gian mở - đóng bài thi, số lần làm bài và vị trí sử dụng bài kiểm tra trong khóa học.

2.2.5 Quy trình nghiệp vụ tạo và xuất bản bài kiểm tra

Quy trình tạo và xuất bản bài kiểm tra kết hợp nhiều chức năng trong nhóm Quản lý bài kiểm tra, được trình bày trong Hình 2.5.



Hình 2.5: Biểu đồ hoạt động quy trình tạo và xuất bản bài kiểm tra

Quy trình nghiệp vụ tạo và xuất bản bài kiểm tra bao gồm hai tác nhân chính là Giáo viên hoặc Quản trị viên và Hệ thống. Quy trình bắt đầu khi Giáo viên hoặc Quản trị viên đăng nhập trang quản trị, tạo bài kiểm tra, cập nhật thông tin mô tả và thiết lập câu hỏi. Ở bước thiết lập vị trí sử dụng, hệ thống hỗ trợ ba ngữ cảnh triển

khai bài kiểm tra. Bài kiểm tra công khai được dùng cho hoạt động luyện tập và không phụ thuộc vào quyền truy cập khóa học. Bài kiểm tra trong khóa học được dùng cho kiểm tra định kỳ hoặc giao bài tập bổ sung theo lộ trình học. Bài kiểm tra gắn vào bài học dạng quiz được dùng như hoạt động tự luyện sau một nội dung bài giảng cụ thể.

Sau khi xác định vị trí sử dụng, người quản lý nội dung thiết lập các thông tin vận hành như thời gian làm bài, thời gian mở - đóng bài thi và số lần làm bài. Khi người quản lý nội dung yêu cầu xuất bản, hệ thống kiểm tra điều kiện của bài kiểm tra, bao gồm thông tin cơ bản, câu hỏi và vị trí sử dụng. Nếu bài kiểm tra hợp lệ, hệ thống chuyển bài kiểm tra sang trạng thái xuất bản và hiển thị tại vị trí đã thiết lập. Nếu bài kiểm tra chưa hợp lệ, hệ thống thông báo nội dung cần bổ sung để người quản lý nội dung chỉnh sửa và lưu thay đổi.

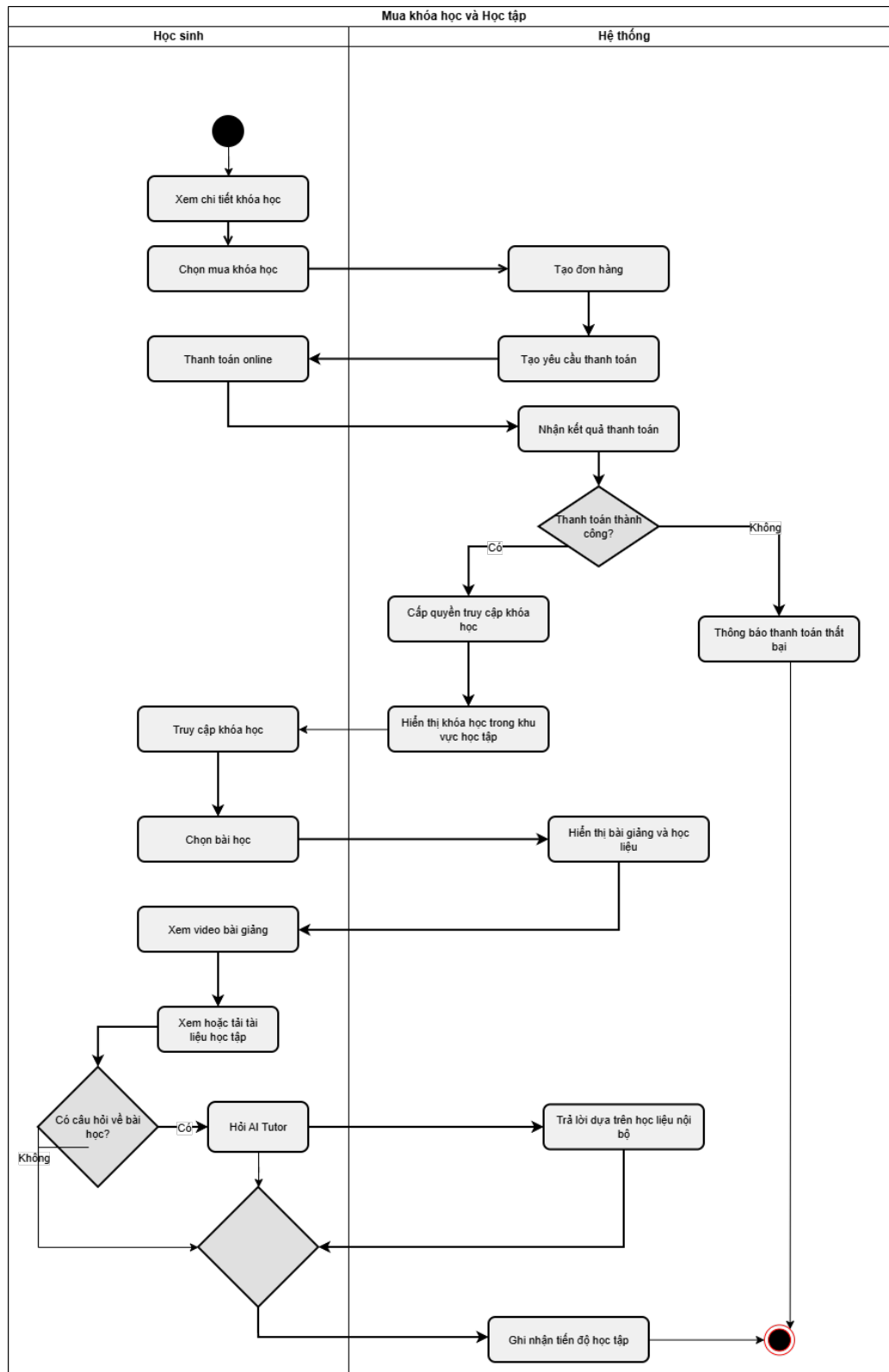
2.2.6 Quy trình nghiệp vụ mua khóa học và học tập

Quy trình mua khóa học và học tập kết hợp các use case mua khóa học, thanh toán, cấp quyền truy cập, học bài và hỏi AI Tutor, được trình bày trong Hình 2.6.

Quy trình nghiệp vụ mua khóa học và học tập bao gồm hai tác nhân chính là Học sinh và Hệ thống. Quy trình bắt đầu khi Học sinh xem chi tiết khóa học và chọn mua khóa học. Hệ thống tạo đơn hàng, sinh yêu cầu thanh toán và hiển thị thông tin thanh toán online, chẳng hạn mã QR chuyển khoản gắn với đơn hàng. Học sinh thực hiện thanh toán theo thông tin này, sau đó hệ thống nhận kết quả giao dịch để xác định trạng thái đơn hàng.

Nếu thanh toán thất bại, hệ thống thông báo lỗi để Học sinh có thể thực hiện lại giao dịch. Nếu thanh toán thành công, hệ thống tự động cấp quyền truy cập khóa học và hiển thị khóa học trong khu vực học tập của Học sinh. Quy trình này thay thế thao tác gửi biên lai và duyệt quyền truy cập thủ công trong mô hình vận hành cũ.

Sau khi được cấp quyền truy cập, Học sinh vào khóa học, chọn bài học, xem video bài giảng và xem hoặc tải tài liệu học tập. Trong quá trình học, nếu phát sinh câu hỏi liên quan đến nội dung bài học, Học sinh có thể hỏi AI Tutor ngay trong ngữ cảnh bài học hiện tại. AI Tutor trả lời dựa trên học liệu nội bộ mà giáo viên cung cấp, giúp học sinh tiếp tục học mà không phải chờ phản hồi trực tiếp từ giáo viên. Sau quá trình học, hệ thống ghi nhận tiến độ học tập của Học sinh.



Hình 2.6: Biểu đồ hoạt động quy trình mua khóa học và học tập

2.3 Đặc tả chức năng

Phần này tập trung đặc tả 6 use case cốt lõi của hệ thống, gồm Quản lý khóa học, Quản lý bài kiểm tra, Mua khóa học, Học bài và xem học liệu, Hỏi AI Tutor và Làm bài kiểm tra. Các use case này có nhiều quy tắc nghiệp vụ và thể hiện rõ

tương tác giữa các tác nhân trong hệ thống.

Bảng 2.2 mô tả danh sách đầy đủ các use case của hệ thống, được nhóm theo các nhóm chức năng chính. Hệ thống có tổng cộng 21 use case, bao phủ quản lý khóa học, bài kiểm tra, mua và học khóa học, tài khoản, blog, thông báo và vận hành hệ thống. Để người đọc tiện theo dõi, 6 use case cốt lõi được ưu tiên đặt ở đầu bảng và đặc tả chi tiết.

Bảng 2.2: Danh sách tổng quan các use case của hệ thống

Mã use case	Tên use case	Tác nhân	Nhóm use case
UC01	Quản lý khóa học	Giáo viên, Quản trị viên	Quản lý khóa học
UC02	Quản lý bài kiểm tra	Giáo viên, Quản trị viên	Quản lý bài kiểm tra
UC03	Mua khóa học	Học sinh	Mua khóa học
UC04	Học bài và xem học liệu	Học sinh	Học bài và xem học liệu
UC05	Hỏi AI Tutor	Học sinh	Học bài và xem học liệu
UC06	Làm bài kiểm tra	Học sinh	Làm bài kiểm tra
UC07	Đăng ký tài khoản	Khách truy cập	Tài khoản
UC08	Đăng nhập	Người dùng	Tài khoản
UC09	Xem danh sách khóa học	Học sinh, Khách truy cập	Khóa học công khai
UC10	Xem chi tiết khóa học	Học sinh, Khách truy cập	Khóa học công khai
UC11	Xem blog	Người dùng, Khách truy cập	Blog
UC12	Quản lý thông tin cá nhân	Người dùng	Tài khoản
UC13	Xem kết quả bài kiểm tra	Học sinh	Làm bài kiểm tra
UC14	Chấm điểm bài làm	Giáo viên	Quản lý bài kiểm tra
UC15	Xem kết quả học tập của học sinh	Giáo viên	Theo dõi học tập
UC16	Quản lý bài viết blog	Giáo viên, Quản trị viên	Blog
UC17	Quản lý người dùng	Quản trị viên	Quản trị hệ thống
UC18	Tạo và quản lý tài khoản giáo viên	Quản trị viên	Quản trị hệ thống
Tiếp trang sau			

Bảng 2.2 - Tiếp trang trước

Mã use case	Tên use case	Tác nhân	Nhóm use case
UC19	Quản lý đơn hàng	Quản trị viên	Quản lý đơn hàng
UC20	Quản lý giao dịch	Quản trị viên	Quản lý thanh toán
UC21	Xem và xử lý thông báo	Người dùng	Thông báo

2.3.1 Đặc tả use case Quản lý khóa học

Use case Quản lý khóa học hỗ trợ Người quản lý nội dung ở hai phạm vi: biên soạn nội dung và theo dõi học viên. Ở tab Quản lý nội dung, người dùng tạo khóa học, xây dựng chương, bài học, học liệu và xuất bản. Tab Quản lý học viên là tab mặc định, hiển thị danh sách người đã ghi danh, phần trăm tiến độ và cho phép mở chi tiết từng học sinh để xem bài đã hoàn thành, bài chưa hoàn thành, trạng thái làm bài kiểm tra và điểm cao nhất.

Đặc tả chi tiết use case Quản lý khóa học được trình bày trong Bảng 2.3.

Bảng 2.3: Đặc tả use case Quản lý khóa học

Thông tin	Mô tả
Mã use case	UC01
Tên use case	Quản lý khóa học
Các tác nhân	Người quản lý nội dung (Giáo viên hoặc Quản trị viên)
Mục tiêu	Tạo lập, biên soạn và xuất bản nội dung; đồng thời theo dõi danh sách học viên và tiến độ học tập trong khóa học.
Tiền điều kiện	Người quản lý nội dung đã đăng nhập vào hệ thống. Nếu người dùng là Giáo viên, phạm vi thao tác bị giới hạn trong các khóa học được phân công; nếu là Quản trị viên, phạm vi thao tác là toàn hệ thống.
Luồng sự kiện chính	1. Người quản lý nội dung chọn chức năng quản lý khóa học.
	2. Hệ thống hiển thị danh sách khóa học trong phạm vi quản lý.
	3. Người quản lý nội dung chọn tạo mới khóa học.
	4. Hệ thống hiển thị biểu mẫu nhập thông tin tổng quan của khóa học.
	5. Người quản lý nội dung nhập tiêu đề, mô tả, tệp ảnh đại diện, giá bán và chọn lưu.
	6. Hệ thống kiểm tra tính hợp lệ của thông tin, lưu khóa học ở trạng thái bản nháp.
<i>Tiếp trang sau</i>	

<i>Bảng 2.3 - Tiếp trang trước</i>	
Thông tin	Mô tả
	7. Người quản lý nội dung thêm các chương học mới và sắp xếp thứ tự của chúng.
	8. Người quản lý nội dung tạo các bài học trong từng chương, lựa chọn hình thức bài học và tải lên video hoặc học liệu tương ứng.
	9. Người quản lý nội dung chọn yêu cầu xuất bản khóa học.
	10. Hệ thống kiểm tra điều kiện xuất bản và thay đổi trạng thái khóa học sang công khai.
Luồng thay thế	5.a. Thông tin khóa học nhập vào không hợp lệ, ví dụ thiếu tiêu đề hoặc giá bán không hợp lệ: Hệ thống báo lỗi chi tiết trên biểu mẫu; Người quản lý nội dung chỉnh sửa thông tin và thực hiện lưu lại.
	10.a. Hệ thống phát hiện các bài học dạng video của khóa học chưa có tệp video được tải lên hoàn tất hoặc chưa ở trạng thái sẵn sàng; Hệ thống thông báo lỗi điều kiện xuất bản và giữ khóa học ở trạng thái bản nháp; Người quản lý nội dung quay lại bước 8 để cập nhật học liệu.
	2.a. Người quản lý nội dung mở tab Quản lý học viên: Hệ thống hiển thị danh sách ghi danh, tiến độ chung và trạng thái hoạt động; khi chọn một học sinh, hệ thống trình bày curriculum theo thứ tự chương-bài, mức hoàn thành từng bài và kết quả các bài kiểm tra gắn với khóa học.
Hậu điều kiện	Khóa học và học liệu đi kèm được lưu trữ, cập nhật hoặc xuất bản thành công trên hệ thống.

2.3.2 Đặc tả use case Quản lý bài kiểm tra

Use case Quản lý bài kiểm tra cho phép Người quản lý nội dung xây dựng các bài đánh giá năng lực học sinh. Luồng sự kiện bắt đầu khi Người quản lý nội dung tạo bài kiểm tra nháp, nhập thông tin mô tả, thiết lập câu hỏi và cấu hình các thông số vận hành. Sau đó, bài kiểm tra được gắn vào một vị trí sử dụng cụ thể, có thể là khu vực luyện tập công khai, một khóa học hoặc một bài học dạng quiz.

Đặc tả chi tiết use case Quản lý bài kiểm tra được trình bày trong Bảng 2.4.

Bảng 2.4: Đặc tả use case Quản lý bài kiểm tra

Thông tin	Mô tả
Mã use case	UC02
Tên use case	Quản lý bài kiểm tra
Các tác nhân	Người quản lý nội dung (Giáo viên hoặc Quản trị viên)
Mục tiêu	Khởi tạo, thiết lập nội dung câu hỏi, cấu hình các thông số và xuất bản bài kiểm tra tại các vị trí học tập.
Tiền điều kiện	Người quản lý nội dung đã đăng nhập vào hệ thống. Nếu người dùng là Giáo viên, phạm vi thao tác bị giới hạn trong các bài kiểm tra được phân quyền; nếu là Quản trị viên, phạm vi thao tác là toàn hệ thống.
Luồng sự kiện chính	1. Người quản lý nội dung chọn chức năng quản lý bài kiểm tra. 2. Hệ thống hiển thị danh sách các bài kiểm tra hiện hành. 3. Người quản lý nội dung yêu cầu tạo bài kiểm tra mới. 4. Hệ thống hiển thị giao diện nhập thông tin tổng quan của bài kiểm tra. 5. Người quản lý nội dung thiết lập tiêu đề, chủ đề, khối lớp, thời gian làm bài, số lượt làm bài tối đa và xác nhận lưu. 6. Hệ thống khởi tạo bài kiểm tra dưới dạng bản nháp. 7. Người quản lý nội dung xây dựng các phần và thêm các câu hỏi vào bài kiểm tra. 8. Người quản lý nội dung thiết lập vị trí hiển thị cho bài kiểm tra, bao gồm bài luyện tập công khai, bài kiểm tra thuộc khóa học hoặc quiz gắn với bài học. 9. Người quản lý nội dung gửi yêu cầu xuất bản bài kiểm tra. 10. Hệ thống xác thực nội dung, vị trí sử dụng và chuyển trạng thái bài kiểm tra sang xuất bản.
<i>Tiếp trang sau</i>	

Bảng 2.4 - Tiếp trang trước

Thông tin	Mô tả
Luồng thay thế	7.a. Người quản lý nội dung nhập danh sách câu hỏi từ tệp theo định dạng không hợp lệ: Hệ thống báo lỗi chi tiết; Người quản lý nội dung chỉnh sửa dữ liệu và thực hiện nhập lại. 10.a. Hệ thống phát hiện bài kiểm tra chưa có câu hỏi hoặc chưa thiết lập vị trí sử dụng hợp lệ: Hệ thống hiển thị cảnh báo lỗi và giữ bài kiểm tra ở trạng thái nháp; Người quản lý nội dung quay lại bước 7 hoặc 8 để hiệu chỉnh thông tin. 2.a. Người quản lý nội dung mở danh sách bài làm: Hệ thống nhóm kết quả theo bài kiểm tra, phân biệt học sinh đã nộp, chưa nộp và đang chờ chấm; người quản lý có thể mở bài tự luận, cho điểm và hoàn tất kết quả.
Hậu điều kiện	Bài kiểm tra được tạo lập, cấu hình thông số và hiển thị thành công tại vị trí được chỉ định.

2.3.3 Đặc tả use case Mua khóa học

Use case Mua khóa học hỗ trợ Học sinh đăng ký quyền tham gia vào các khóa học trả phí. Luồng sự kiện bắt đầu khi Học sinh chọn mua khóa học từ trang chi tiết khóa học. Hệ thống tạo đơn hàng, hiển thị thông tin thanh toán online bằng mã QR chuyển khoản và tự động cấp quyền truy cập khóa học sau khi nhận được kết quả thanh toán thành công.

Đặc tả chi tiết use case Mua khóa học được trình bày trong Bảng 2.5.

Bảng 2.5: Đặc tả use case Mua khóa học

Thông tin	Mô tả
Mã use case	UC03
Tên use case	Mua khóa học
Các tác nhân	Học sinh
Mục tiêu	Thực hiện giao dịch mua khóa học trực tuyến và tự động cấp quyền truy cập khóa học cho học sinh.
Tiền điều kiện	Học sinh đã đăng nhập thành công và khóa học được chọn là khóa học trả phí mà tài khoản học sinh chưa sở hữu quyền truy cập.
<i>Tiếp trang sau</i>	

Bảng 2.5 - Tiếp trang trước

Thông tin	Mô tả
Luồng sự kiện chính	1. Học sinh truy cập trang chi tiết khóa học trả phí. 2. Học sinh chọn chức năng mua khóa học. 3. Hệ thống tạo một đơn hàng mới ở trạng thái chờ thanh toán. 4. Hệ thống hiển thị thông tin hướng dẫn thanh toán online (gồm mã QR động chuyển khoản liên kết với mã đơn hàng). 5. Học sinh thực hiện chuyển khoản thanh toán theo thông tin được hiển thị. 6. Hệ thống nhận thông báo kết quả thanh toán thành công từ phía ngân hàng hoặc cổng thanh toán đối tác. 7. Hệ thống cập nhật trạng thái đơn hàng thành hoàn thành. 8. Hệ thống tự động khởi tạo quyền truy cập khóa học cho học sinh và gửi thông báo giao dịch thành công.
Luồng thay thế	6.a. Giao dịch bị hủy hoặc thanh toán không thành công: Hệ thống cập nhật trạng thái đơn hàng thành thất bại/hủy; Học sinh nhận thông báo giao dịch thất bại và có thể tiến hành thanh toán lại. 6.b. Hệ thống đã ghi nhận giao dịch nhưng chưa cấp được quyền truy cập khóa học: Hệ thống lưu trạng thái giao dịch để Quản trị viên đối soát; Quản trị viên xác minh đơn hàng và cấp quyền truy cập thủ công thông qua trang quản trị.
Hậu điều kiện	Trạng thái đơn hàng được cập nhật và học sinh được cấp quyền truy cập khóa học thành công.

2.3.4 Đặc tả use case Học bài và xem học liệu

Use case Học bài và xem học liệu mô tả trải nghiệm học tập chính của Học sinh trên hệ thống. Luồng sự kiện bắt đầu khi Học sinh truy cập khu vực học tập, chọn khóa học đã được cấp quyền và mở một bài học cụ thể. Hệ thống hiển thị nội dung bài học gồm video, tài liệu học tập và ghi nhận tiến độ để Học sinh có thể tiếp tục học ở những lần truy cập sau.

Đặc tả chi tiết use case Học bài và xem học liệu được trình bày trong Bảng 2.6.

Bảng 2.6: Đặc tả use case Học bài và xem học liệu

Thông tin	Mô tả
Mã use case	UC04
Tên use case	Học bài và xem học liệu
Các tác nhân	Học sinh
<i>Tiếp trang sau</i>	

<i>Bảng 2.6 - Tiếp trang trước</i>	
Thông tin	Mô tả
Mục tiêu	Truy cập nội dung bài học, xem video bài giảng, xem và tải tài liệu lý thuyết, và tự động ghi nhận tiến độ học tập.
Tiền điều kiện	Học sinh đã đăng nhập thành công và đã được cấp quyền truy cập khóa học tương ứng.
Luồng sự kiện chính	1. Học sinh truy cập vào khu vực học tập cá nhân. 2. Học sinh lựa chọn khóa học muốn học trong danh mục khóa học của tôi. 3. Hệ thống hiển thị danh sách cấu trúc chương trình học gồm các chương và bài học. 4. Học sinh chọn một bài học cụ thể. 5. Hệ thống kiểm tra quyền truy cập khóa học của học sinh, tải dữ liệu bài học và hiển thị giao diện học tập (trình phát video bài giảng, tài liệu đọc trực tuyến). 6. Học sinh xem video bài giảng hoặc xem các tài liệu đính kèm. 7. Học sinh chọn chức năng tải tài liệu đính kèm về thiết bị cá nhân (nếu có). 8. Hệ thống ghi nhận vị trí xem video và tiến độ học tập của học sinh để hỗ trợ tiếp tục học ở lần truy cập sau.
Luồng thay thế	5.a. Hệ thống phát hiện tài khoản học sinh chưa có quyền truy cập khóa học: Hệ thống chặn tải dữ liệu bài học, hiển thị thông báo lỗi và chuyển hướng học sinh về trang thông tin khóa học. 8.a. Hệ thống không ghi nhận được tiến độ học tập do lỗi kết nối hoặc lỗi máy chủ: Hệ thống hiển thị thông báo lỗi; Học sinh có thể tải lại bài học và tiếp tục học tại vị trí gần nhất đã được lưu trước đó.
Hậu điều kiện	Nội dung học tập được hiển thị chính xác và tiến độ học tập của học sinh được ghi nhận thành công trên máy chủ.

2.3.5 Đặc tả use case Hỏi AI Tutor

Use case Hỏi AI Tutor cung cấp cho Học sinh khả năng đặt câu hỏi và nhận giải đáp tự động ngay trong giao diện học tập. Luồng sự kiện bắt đầu khi Học sinh mở khung chat trong bài học, nhập câu hỏi và gửi tới hệ thống. Hệ thống chuyển câu hỏi cùng ngữ cảnh khóa học hoặc bài học hiện tại tới dịch vụ AI Tutor, sau đó hiển thị câu trả lời và các tham chiếu học liệu liên quan nếu có.

Đặc tả chi tiết use case Hỏi AI Tutor được trình bày trong Bảng 2.7.

Bảng 2.7: Đặc tả use case Hỏi AI Tutor

Thông tin	Mô tả
Mã use case	UC05
Tên use case	Hỏi AI Tutor
Các tác nhân	Học sinh
Mục tiêu	Đặt câu hỏi thắc mắc và nhận câu trả lời giải đáp từ trợ lý AI dựa trên học liệu nội bộ của bài học.
Tiền điều kiện	Học sinh đang ở trong giao diện học tập của bài học có cấu hình hỗ trợ AI Tutor.
Luồng sự kiện chính	1. Học sinh kích hoạt khung chat AI Tutor tại giao diện bài học. 2. Hệ thống tải phiên hội thoại mới hoặc tiếp tục phiên cũ, hiển thị các gợi ý câu hỏi liên quan đến nội dung bài học. 3. Học sinh nhập câu hỏi và gửi đi. 4. Hệ thống gửi câu hỏi cùng thông tin ngữ cảnh của khóa học hoặc bài học hiện tại tới dịch vụ AI Tutor. 5. Dịch vụ AI Tutor xử lý câu hỏi và trả về câu trả lời kèm các tham chiếu học liệu nếu có. 6. Hệ thống hiển thị câu trả lời và các tham chiếu liên quan trên khung chat của học sinh.
Luồng thay thế	3.a. Học sinh gửi câu hỏi nằm ngoài phạm vi bài học: Hệ thống vẫn gửi câu hỏi tới AI Tutor, nhưng câu trả lời cần định hướng học sinh quay lại nội dung học tập liên quan. 5.a. Dịch vụ AI Tutor gặp sự cố kết nối hoặc không phản hồi: Hệ thống hiển thị thông báo lỗi và gợi ý học sinh gửi lại câu hỏi sau.
Hậu điều kiện	Câu hỏi được ghi nhận trong phiên hội thoại và học sinh nhận được câu trả lời từ AI Tutor nếu dịch vụ xử lý thành công.

2.3.6 Đặc tả use case Làm bài kiểm tra

Use case Làm bài kiểm tra mô tả tiến trình làm bài của Học sinh để tự đánh giá năng lực hoặc hoàn thành các yêu cầu trong khóa học. Luồng sự kiện bắt đầu khi Học sinh truy cập một bài kiểm tra hợp lệ, bắt đầu lượt làm bài, trả lời câu hỏi và nộp bài. Hệ thống ghi nhận bài làm, tự chấm các câu có đáp án xác định và chuyển sang trạng thái chờ chấm nếu bài làm có câu tự luận.

Đặc tả chi tiết use case Làm bài kiểm tra được trình bày trong Bảng 2.8.

Bảng 2.8: Đặc tả use case Làm bài kiểm tra

Thông tin	Mô tả
Mã use case	UC06
Tên use case	Làm bài kiểm tra
Các tác nhân	Học sinh; Giáo viên trong luồng chấm bài tự luận
Mục tiêu	Thực hiện làm bài kiểm tra, nộp bài làm để hệ thống ghi nhận kết quả và tính điểm.
Tiền điều kiện	Học sinh đã đăng nhập thành công và có quyền truy cập vào bài kiểm tra tương ứng (tùy thuộc vào vị trí hiển thị là luyện tập công khai, bài kiểm tra khóa học, hoặc quiz bài học).
Luồng sự kiện chính	1. Học sinh chọn bài kiểm tra từ lộ trình học hoặc trang bài tập công khai. 2. Hệ thống hiển thị giao diện giới thiệu quy chế bài kiểm tra và số lượt làm bài còn lại. 3. Học sinh chọn bắt đầu lượt làm bài mới. 4. Hệ thống kiểm tra điều kiện lượt làm bài, mở giao diện làm bài và bắt đầu tính thời gian làm bài. 5. Học sinh thực hiện trả lời các câu hỏi và chọn lưu câu trả lời trên hệ thống. 6. Học sinh chọn nộp bài trước khi thời gian đếm ngược kết thúc. 7. Hệ thống ghi nhận bài làm, lưu đáp án của học sinh vào cơ sở dữ liệu. 8. Hệ thống tự chấm các câu trắc nghiệm, đúng sai và câu trả lời bằng số. 9. Hệ thống lưu điểm tự động và cập nhật bài làm thành đã chấm xong nếu đề không có câu tự luận; trường hợp còn lại được chuyển sang trạng thái chờ chấm. 10. Học sinh xem kết quả bài kiểm tra; đáp án đúng và phần giải thích chỉ được hiển thị sau khi bài đã chấm xong và chỉ khi giáo viên đã nhập dữ liệu giải thích.
<i>Tiếp trang sau</i>	

Bảng 2.8 - Tiếp trang trước

Thông tin	Mô tả
Luồng thay thế	4.a. Học sinh đã sử dụng hết số lần làm bài cho phép: Hệ thống thông báo lỗi giới hạn lượt làm bài và chặn không cho mở lượt làm bài mới. 6.a. Đến hạn sớm hơn giữa thời điểm bắt đầu cộng thời gian làm bài và thời điểm đóng bài: backend tự động thu bài dựa trên các câu trả lời đã lưu, kể cả khi trình duyệt đã đóng; các câu có đáp án xác định được chấm tự động, còn câu tự luận chuyển sang chờ chấm. 9.a. Bài làm chứa câu hỏi tự luận: Hệ thống cập nhật trạng thái bài làm là chờ chấm điểm; Giáo viên tiến hành xem xét bài làm tự luận và cho điểm thủ công trên trang quản trị; Hệ thống sau đó cập nhật điểm số cuối cùng và hiển thị kết quả cho học sinh.
Hậu điều kiện	Phiên làm bài kiểm tra được ghi nhận, chấm điểm và lưu trữ lịch sử học tập thành công.

2.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng đã được trình bày ở Mục 2.2 và Mục 2.3, hệ thống cần đáp ứng một số yêu cầu phi chức năng để có thể vận hành ổn định trong bối cảnh dạy và học trực tuyến. Các yêu cầu quan trọng bao gồm bảo mật, hiệu năng, độ tin cậy, khả năng mở rộng, tính dễ sử dụng, tính dễ bảo trì và một số ràng buộc công nghệ.

2.4.1 Bảo mật

Hệ thống cần xác thực người dùng trước khi truy cập các chức năng yêu cầu đăng nhập, mật khẩu không được lưu dưới dạng văn bản gốc và các API quan trọng phải kiểm tra vai trò người dùng. Học sinh chỉ được truy cập nội dung khóa học khi đã có quyền học hợp lệ; Giáo viên chỉ được quản lý nội dung trong phạm vi được phân công; Quản trị viên có quyền vận hành toàn hệ thống. Các dữ liệu nhạy cảm liên quan đến thanh toán, phiên học tập và AI Tutor phải được xử lý theo đúng phạm vi người dùng hiện tại.

2.4.2 Hiệu năng

Hệ thống cần phản hồi ổn định với các thao tác thường xuyên như xem danh sách khóa học, mở bài học, lưu tiến độ, tạo đơn hàng, nộp bài kiểm tra và gửi câu hỏi tới AI Tutor. Các danh sách lớn như khóa học, người dùng, đơn hàng, bài kiểm tra và giao dịch thanh toán cần hỗ trợ phân trang, lọc và tìm kiếm. API chỉ nên trả về dữ liệu cần thiết cho từng màn hình; riêng video bài giảng không được lưu trực

tiếp trên server backend, mà cần được quản lý như tài nguyên media riêng và chỉ cho phép sử dụng khi tệp đã sẵn sàng.

2.4.3 Độ tin cậy

Hệ thống cần đảm bảo dữ liệu không bị cập nhật dở dang trong các nghiệp vụ quan trọng như tạo đơn hàng, ghi nhận thanh toán, cấp quyền truy cập khóa học, lưu bài làm và chấm điểm. Các thao tác liên quan đến nhiều bảng dữ liệu phải được xử lý nhất quán bằng transaction. Đối với thanh toán online, hệ thống cần có cơ chế chống xử lý trùng yêu cầu, lưu lại giao dịch để đối soát và chỉ hoàn tất đơn hàng khi thông tin giao dịch hợp lệ.

2.4.4 Khả năng mở rộng

Hệ thống cần có khả năng mở rộng thêm vai trò người dùng, loại bài học, loại bài kiểm tra, phương thức thanh toán và khả năng hỗ trợ của AI Tutor. Cấu trúc dữ liệu và API cần tránh phụ thuộc vào một kịch bản duy nhất, ví dụ bài kiểm tra có thể được dùng làm bài luyện tập công khai, bài kiểm tra trong khóa học hoặc quiz sau bài học. Phần AI Tutor cần được tách thành thành phần tích hợp riêng để có thể thay đổi mô hình hoặc nguồn học liệu trong tương lai.

2.4.5 Tính dễ sử dụng

Giao diện cần tổ chức rõ luồng sử dụng của từng nhóm người dùng. Học sinh cần dễ dàng tìm khóa học, mua khóa học, mở bài học, xem học liệu, hỏi AI Tutor và làm bài kiểm tra. Giáo viên và Quản trị viên cần thao tác thuận tiện khi tạo khóa học, xây dựng chương và bài học, thiết lập bài kiểm tra và theo dõi kết quả học tập. Các lỗi nghiệp vụ phải được thông báo rõ nguyên nhân, ví dụ khóa học chưa thể xuất bản do thiếu học liệu hợp lệ hoặc bài kiểm tra chưa đủ điều kiện công khai.

2.4.6 Tính dễ bảo trì

Mã nguồn cần được tổ chức theo các module nghiệp vụ rõ ràng như khóa học, học tập, bài kiểm tra, đơn hàng, thanh toán, người dùng, blog và AI Tutor. Các phần xử lý dữ liệu, xử lý nghiệp vụ và giao tiếp API cần được tách biệt để việc sửa lỗi hoặc bổ sung chức năng không ảnh hưởng lan rộng. Các ràng buộc quan trọng như quyền truy cập khóa học, điều kiện xuất bản, trạng thái đơn hàng và trạng thái bài làm cần được đặt ở phía máy chủ để đảm bảo thống nhất giữa các giao diện.

2.4.7 Ràng buộc công nghệ

Hệ thống cần sử dụng CSDL quan hệ để quản lý các dữ liệu có ràng buộc chặt chẽ như người dùng, khóa học, chương, bài học, bài kiểm tra, đơn hàng, giao dịch thanh toán, quyền truy cập khóa học và tiến độ học tập. Frontend và backend giao tiếp thông qua REST API. Các học liệu dạng media cần được lưu trữ và phục vụ

bằng cơ chế phù hợp với trình duyệt; thanh toán cần hỗ trợ mã QR hoặc chuyển khoản online; AI Tutor cần nhận câu hỏi theo ngữ cảnh khóa học hoặc bài học và trả về câu trả lời kèm tham chiếu học liệu khi có dữ liệu phù hợp.

Các yêu cầu phi chức năng trên là cơ sở để lựa chọn công nghệ, thiết kế kiến trúc và tổ chức triển khai hệ thống ở các chương tiếp theo. Trong đó, các yêu cầu về phân quyền, dữ liệu, thanh toán, học liệu và AI Tutor có ảnh hưởng trực tiếp tới thiết kế tổng thể của hệ thống.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã trình bày bối cảnh bài toán, các chức năng chính và các yêu cầu phi chức năng của hệ thống. Từ các yêu cầu đó, chương này trình bày các công nghệ, nền tảng lý thuyết và thư viện được sử dụng trong quá trình xây dựng hệ thống. Nội dung chương tập trung làm rõ mỗi công nghệ được dùng để giải quyết yêu cầu nào, có những lựa chọn thay thế nào và vì sao đề án lựa chọn hướng triển khai hiện tại.

3.1 Công nghệ phát triển phía máy chủ

Phía máy chủ chịu trách nhiệm xử lý các nghiệp vụ cốt lõi của hệ thống như quản lý người dùng, khóa học, bài học, học liệu, bài kiểm tra, đơn hàng, thanh toán và quyền truy cập khóa học. Các yêu cầu ở Chương 2 cho thấy backend không chỉ cần cung cấp API cho frontend, mà còn phải bảo đảm tính nhất quán dữ liệu, kiểm soát phân quyền, xử lý tệp video bài giảng dữ liệu lớn và giữ cho các luồng nghiệp vụ có thể mở rộng về sau. Vì vậy, các công nghệ phía máy chủ được lựa chọn theo hướng rõ ràng, dễ kiểm soát và phù hợp với hệ sinh thái TypeScript.

3.1.1 Node.js, Express và TypeScript

Backend của hệ thống sử dụng Node.js làm môi trường thực thi, Express làm framework xây dựng ứng dụng web và TypeScript làm ngôn ngữ phát triển chính. Node.js phù hợp với các ứng dụng web có nhiều thao tác vào ra như truy vấn CSDL, xử lý yêu cầu HTTP, gọi dịch vụ lưu trữ, nhận thông báo thanh toán và phục vụ dữ liệu học tập [5]. Express cung cấp cơ chế định tuyến và middleware gọn nhẹ, giúp tổ chức các API theo từng nhóm chức năng như khóa học, bài kiểm tra, media, đơn hàng, thanh toán và người dùng [6]. TypeScript bổ sung kiểu tĩnh cho JavaScript, giúp giảm lỗi khi dữ liệu đi qua nhiều lớp như controller, service, repository và mapper [7].

Các lựa chọn thay thế cho backend có thể là Spring Boot, Django, NestJS hoặc Fastify. Spring Boot có hệ sinh thái mạnh và phù hợp với hệ thống doanh nghiệp lớn, nhưng cần nhiều cấu hình hơn so với phạm vi đề án. Django hỗ trợ phát triển nhanh, tuy nhiên không thống nhất ngôn ngữ với frontend. NestJS có kiến trúc đầy đủ hơn Express, nhưng cũng tạo thêm lớp trừu tượng và quy ước framework. Fastify có hiệu năng tốt, nhưng Express phổ biến hơn, tài liệu phong phú hơn và đủ đáp ứng quy mô hiện tại. Vì vậy, Express kết hợp TypeScript được lựa chọn để giữ backend gọn, dễ đọc, đồng thời vẫn có khả năng tổ chức mã nguồn theo nghiệp vụ.

3.1.2 REST API và tổ chức mã nguồn theo module

Hệ thống sử dụng REST API làm phương thức giao tiếp chính giữa frontend và backend. Các nghiệp vụ đã mô tả ở Chương 2 như xem khóa học, mua khóa học, mở bài học, làm bài kiểm tra, quản lý bài viết, quản lý đơn hàng hoặc gửi câu hỏi theo học liệu đều có thể được biểu diễn bằng các tài nguyên và endpoint rõ ràng. Cách tiếp cận này giúp frontend gọi dữ liệu theo từng màn hình, còn backend có thể đặt các kiểm tra về xác thực, phân quyền và trạng thái nghiệp vụ ở một nơi tập trung.

Ở mức tổ chức mã nguồn, backend được chia theo các nhóm nghiệp vụ như auth, users, courses, assessments, orders, payments, media, blogs, enrollments và tutor. Trong mỗi nhóm, các phần như route, controller, service, repository, validator và mapper được tách riêng theo vai trò. Cách tổ chức này giúp người đọc dễ lần theo luồng xử lý: request đi vào route/controller, dữ liệu được kiểm tra, service xử lý nghiệp vụ, repository làm việc với CSDL, sau đó mapper chuẩn hóa dữ liệu trả về. Một lựa chọn khác là gom toàn bộ logic trong controller để triển khai nhanh hơn, nhưng cách đó dễ làm mã nguồn khó bảo trì khi số lượng chức năng tăng. GraphQL cũng có thể thay thế REST API, nhưng đồ án có nhiều luồng nghiệp vụ cần kiểm soát chặt ở backend, nên REST API phù hợp hơn về tính trực quan và khả năng kiểm thử.

3.1.3 PostgreSQL và Prisma ORM

Hệ thống sử dụng PostgreSQL làm hệ quản trị CSDL quan hệ để lưu trữ các dữ liệu có ràng buộc chặt chẽ như người dùng, vai trò, khóa học, chương, bài học, bài kiểm tra, câu hỏi, bài làm, đơn hàng, giao dịch thanh toán, quyền truy cập khóa học và tiến độ học tập. PostgreSQL hỗ trợ mô hình dữ liệu quan hệ, khóa ngoại, transaction và các cơ chế truy vấn phù hợp với yêu cầu về tính nhất quán dữ liệu đã nêu ở Mục 2.4.3 [8]. Đây là lựa chọn phù hợp hơn so với CSDL NoSQL trong bối cảnh hệ thống có nhiều quan hệ giữa các thực thể nghiệp vụ và cần bảo toàn dữ liệu khi xử lý thanh toán hoặc nộp bài kiểm tra.

Để thao tác với CSDL từ backend TypeScript, hệ thống sử dụng Prisma ORM. Prisma cho phép mô tả mô hình dữ liệu bằng schema, sinh client truy vấn có kiểu và hỗ trợ quản lý thay đổi lược đồ thông qua migration [9]. So với viết SQL thuần, Prisma giảm lượng mã lặp lại trong các thao tác CRUD và giúp dữ liệu trả về có kiểu rõ ràng. So với Sequelize hoặc TypeORM, Prisma phù hợp hơn với định hướng TypeScript của đồ án vì schema và client được sinh ra nhất quán, giúp service và repository ít phụ thuộc vào chuỗi truy vấn thủ công. Trong hệ thống hiện tại, Prisma được đặt chủ yếu ở repository để phần service tập trung vào nghiệp vụ thay vì chỉ

tiết truy vấn.

3.1.4 Zod, JWT và kiểm soát quyền truy cập

Các API của hệ thống nhận dữ liệu từ nhiều loại người dùng khác nhau, do đó backend cần kiểm tra dữ liệu đầu vào trước khi xử lý nghiệp vụ. Hệ thống sử dụng Zod để định nghĩa schema và kiểm tra dữ liệu ở runtime [10]. Cách làm này được áp dụng cho các yêu cầu như đăng nhập, đăng ký, tạo khóa học, thiết lập bài kiểm tra, cập nhật bài học, tạo đơn hàng và hoàn tất upload học liệu. Các lựa chọn thay thế có thể là Joi hoặc Yup; tuy nhiên Zod được lựa chọn vì hỗ trợ tốt TypeScript và giúp giữ schema kiểm tra dữ liệu gần với kiểu dữ liệu mà backend sử dụng.

Về xác thực, hệ thống sử dụng JWT để biểu diễn thông tin phiên đăng nhập theo chuẩn RFC 7519 [11]. JWT phù hợp với ứng dụng web tách riêng frontend và backend vì frontend có thể gửi token trong các yêu cầu cần xác thực, còn backend kiểm tra token trước khi thực hiện nghiệp vụ. Về kiểm soát quyền truy cập, hệ thống sử dụng middleware kiểm tra vai trò như Học sinh, Giáo viên và Quản trị viên ở các nhóm route cần bảo vệ. Với các nghiệp vụ cần kiểm tra sâu hơn, service sử dụng thêm policy theo tài nguyên, ví dụ giáo viên chỉ được quản lý khóa học hoặc bài kiểm tra thuộc phạm vi được phép, còn quản trị viên có quyền vận hành tổng thể. Cách làm này chưa phải một mô hình phân quyền động đầy đủ, nhưng phù hợp với quy mô hiện tại và khớp với mô hình tác nhân đã trình bày trong biểu đồ use case tổng quát.

3.1.5 Transaction, số thập phân và xử lý thanh toán

Các nghiệp vụ liên quan đến đơn hàng và thanh toán cần bảo đảm rằng dữ liệu không bị cập nhật dở dang. Ví dụ, khi một giao dịch thanh toán được xác nhận, hệ thống phải ghi nhận giao dịch, cập nhật trạng thái đơn hàng và cấp quyền truy cập khóa học cho học sinh một cách nhất quán. Vì vậy, backend sử dụng transaction của CSDL cho các thao tác liên quan đến nhiều bảng dữ liệu. Nếu một bước trong nghiệp vụ thất bại, các thay đổi trước đó có thể được hủy để tránh trạng thái sai, chẳng hạn đơn hàng đã thanh toán nhưng học sinh chưa được cấp quyền học.

Đối với dữ liệu tiền tệ và điểm số, hệ thống cần tránh sai số số thực khi tính toán. Thư viện decimal.js xử lý giá trị thập phân chính xác hơn kiểu extttnumber thông thường của JavaScript, phù hợp với các luồng tính điểm, ghi nhận điểm tự luận, tổng hợp kết quả và thanh toán. Một lựa chọn khác là quy đổi số tiền về đơn vị nhỏ nhất rồi dùng số nguyên; cách này phù hợp với tiền tệ nhưng không đủ linh hoạt cho điểm số có phần thập phân. Vì vậy, decimal.js được dùng cho các phép tính cần độ chính xác ổn định ở backend.

3.1.6 Cloudflare R2 và cơ chế upload trực tiếp

Hệ thống có nhiều media cần lưu trữ như ảnh đại diện khóa học, ảnh minh họa bài viết, tài liệu PDF, tệp đính kèm và video bài giảng. Nếu các tệp này được tải trực tiếp qua backend rồi lưu trên ổ đĩa của máy chủ, backend sẽ phải gánh băng thông lớn, khó mở rộng và khó bảo toàn dữ liệu khi triển khai trên môi trường khác. Vì vậy, hệ thống sử dụng hướng lưu trữ đối tượng cho media, trong đó backend chỉ quản lý thông tin tệp, trạng thái xử lý và quyền truy cập. Cloudflare R2 được sử dụng như dịch vụ lưu trữ đối tượng tương thích với S3 API [12]. Nhờ tương thích S3, backend có thể dùng AWS SDK để tạo presigned URL, kiểm tra object, tải object và xóa object bằng các thao tác của lưu trữ đối tượng [13].

Luồng upload trong hệ thống được thiết kế để frontend tải tệp trực tiếp lên R2 thông qua presigned URL. Khi người dùng chọn ảnh, tài liệu hoặc video, backend tạo bản ghi media ở trạng thái chờ upload, sinh object key theo loại tài nguyên và trả về URL upload tạm thời cho frontend. Sau khi frontend upload xong, backend kiểm tra object trên R2 để xác nhận tệp tồn tại, cập nhật kích thước, kiểu nội dung và trạng thái media. Với ảnh và tài liệu, media có thể chuyển sang trạng thái sẵn sàng. Với video, media chuyển sang trạng thái đã upload và tiếp tục đi qua bước xử lý HLS. Cách làm này giảm tải cho backend, đồng thời vẫn giữ được khả năng kiểm soát quyền sở hữu tệp và trạng thái xử lý media trong CSDL.

3.1.7 FFmpeg, FFprobe và xử lý video HLS

Video bài giảng là loại học liệu có dung lượng lớn và ảnh hưởng trực tiếp đến trải nghiệm học tập. Nếu hệ thống phát trực tiếp tệp video gốc, học sinh có thể phải tải một tệp lớn, khó tua, khó thích nghi với điều kiện mạng và khó kiểm soát trạng thái xử lý. Vì vậy, sau khi video nguồn được upload lên R2, backend sử dụng FFprobe để đọc thông tin video như thời lượng và dùng FFmpeg để chuyển video sang định dạng HLS [14], [15]. HLS chia video thành một tệp playlist và nhiều đoạn nhỏ, cho phép trình duyệt phát dần qua HTTP thay vì tải toàn bộ tệp từ đầu [16].

Trong hệ thống hiện tại, video nguồn được lưu ở vùng source trên Cloudflare R2, sau đó tiến hành xử lý tải video về thư mục tạm, lấy thời lượng bằng FFprobe, chạy FFmpeg để tạo playlist `index.m3u8` và các đoạn `.ts`, rồi upload kết quả lên R2 theo thư mục riêng của media. Khi quá trình hoàn tất, backend cập nhật media sang trạng thái sẵn sàng cùng đường dẫn playlist. Nếu xử lý thất bại, media được đánh dấu lỗi để giao diện có thể thông báo thay vì hiển thị một video hỏng. Một lựa chọn thay thế là dùng dịch vụ xử lý video chuyên dụng, nhưng điều đó làm tăng chi phí và phụ thuộc nền tảng bên ngoài. Với phạm vi đề án, FFmpeg và

FFprobe cho phép chủ động xử lý video ở mức cần thiết, đủ để hỗ trợ bài giảng dạng HLS.

3.1.8 Tài liệu hóa API với OpenAPI/Swagger

Backend sử dụng tài liệu API ở dạng OpenAPI/Swagger để mô tả endpoint, request, response và các trạng thái lỗi. Tài liệu API giúp frontend và backend thống nhất hợp đồng dữ liệu, đặc biệt với các luồng có nhiều trạng thái như bài kiểm tra, upload media, thanh toán và học tập.

Một lựa chọn thay thế là chỉ viết tài liệu API thủ công trong văn bản hoặc để frontend đọc trực tiếp từ mã backend. Cách đó có thể nhanh ở giai đoạn đầu nhưng dễ sai lệch khi API thay đổi. Việc duy trì tài liệu API theo cấu trúc máy đọc được giúp quá trình kiểm tra hợp đồng FE/BE rõ ràng hơn và giảm nhầm lẫn khi hệ thống có nhiều nhóm chức năng.

3.2 Công nghệ phát triển giao diện người dùng

Frontend của hệ thống phục vụ đồng thời hai nhóm trải nghiệm chính. Nhóm thứ nhất là trải nghiệm học tập của học sinh, bao gồm tìm khóa học, mua khóa học, xem bài giảng, mở tài liệu, làm bài kiểm tra và hỏi đáp theo học liệu. Nhóm thứ hai là trải nghiệm quản trị dành cho giáo viên và quản trị viên, bao gồm tạo khóa học, sắp xếp chương bài, upload học liệu, thiết lập bài kiểm tra, chấm điểm, quản lý đơn hàng và viết blog. Vì vậy, frontend cần tổ chức giao diện theo component, quản lý dữ liệu lấy từ API, xử lý nhiều biểu mẫu phức tạp và hiển thị tốt các loại học liệu khác nhau.

3.2.1 React, Next.js và TypeScript

Hệ thống sử dụng React để xây dựng giao diện theo mô hình component. Cách tiếp cận này phù hợp với các màn hình có nhiều phần tái sử dụng như thẻ khóa học, danh sách bài học, form tạo bài kiểm tra, trình phát video, bảng quản trị và trình soạn thảo bài viết [17]. Tuy nhiên, nếu chỉ dùng React theo mô hình ứng dụng một trang, hệ thống phải tự bổ sung nhiều phần như routing, cấu trúc build, tối ưu tải trang và xử lý SEO cho các trang công khai như danh sách khóa học, chi tiết khóa học hoặc bài viết blog. Vì vậy, trên nền React, hệ thống sử dụng Next.js để tổ chức route, chia nhóm trang công khai, trang học tập và trang quản trị, đồng thời hỗ trợ quá trình build và triển khai frontend [18]. TypeScript được dùng ở frontend để mô tả kiểu dữ liệu của form, response từ API và props của component, giúp giảm lỗi khi các màn hình trao đổi dữ liệu với nhau.

Các lựa chọn thay thế gồm Vue, Angular hoặc React kết hợp Vite. Vue có cú pháp dễ tiếp cận, Angular có framework đầy đủ nhưng nặng hơn nhu cầu của đồ

án, còn React kết hợp Vite phù hợp với ứng dụng một trang gọn hơn nhưng không giải quyết sẵn các vấn đề về định tuyến theo file, tổ chức route nhiều khu vực và khả năng tối ưu các trang cần xuất hiện tốt trên công cụ tìm kiếm. Hệ thống lựa chọn Next.js vì có cấu trúc route rõ ràng, dễ tách khu vực quản trị và khu vực học tập, đồng thời vẫn giữ được lợi thế component của React. Điều này phù hợp với yêu cầu về tính dễ bảo trì đã nêu ở Mục 2.4.7.

3.2.2 TanStack Query, Axios và Zustand

Frontend có hai loại trạng thái chính cần quản lý. Loại thứ nhất là server state, tức dữ liệu lấy từ backend như danh sách khóa học, chi tiết bài học, trạng thái media, tiến độ học tập, thông tin bài kiểm tra, đơn hàng và dữ liệu quản trị. Những dữ liệu này có vòng đời phụ thuộc vào API: đang tải, tải lỗi, tải lại, lưu cache tạm thời, hết hạn hoặc cần được cập nhật sau một thao tác ghi. Hệ thống sử dụng TanStack Query để quản lý nhóm trạng thái này, bao gồm loading, fetching, error, cache, refetch và invalidate query khi dữ liệu thay đổi [19]. Nhờ đó, các màn hình như danh sách khóa học, trang học bài hoặc trang quản trị không cần tự viết lại nhiều logic lặp về gọi API, lưu dữ liệu tạm và làm mới dữ liệu sau khi thêm, sửa hoặc xóa.

Đối với việc gửi yêu cầu HTTP, hệ thống sử dụng Axios để tạo lớp gọi API thống nhất, hỗ trợ cấu hình địa chỉ cơ sở, gửi token, xử lý lỗi và tái sử dụng logic request/response [20]. Nếu chỉ dùng Fetch API trực tiếp, mã nguồn ở các màn hình sẽ ngắn trong trường hợp đơn giản nhưng dễ lặp lại khi cần xử lý lỗi, header và trạng thái xác thực. Loại trạng thái thứ hai là trạng thái cục bộ phía trình duyệt, chẳng hạn phiên đăng nhập đã được khôi phục từ bộ nhớ, giỏ hàng hoặc lựa chọn giao diện. Những trạng thái này được quản lý bằng Zustand, một thư viện nhỏ gọn dựa trên hook [21]. TanStack Query đảm nhiệm dữ liệu từ máy chủ, còn Zustand quản lý trạng thái cục bộ; so với Redux, Zustand cần ít mã lặp lại hơn và phù hợp với quy mô hiện tại của hệ thống.

3.2.3 React Hook Form, Zod và kiểm tra dữ liệu phía giao diện

Các màn hình của hệ thống có nhiều form cần kiểm tra dữ liệu trước khi gửi lên backend, ví dụ đăng nhập, đăng ký, tạo tài khoản giáo viên, tạo khóa học, tạo chương, tạo bài học, thiết lập bài kiểm tra, tạo bài viết blog và ghi danh học sinh. Hệ thống sử dụng React Hook Form để quản lý trạng thái form và giảm số lần render không cần thiết khi người dùng nhập liệu [22]. Cách làm này giúp các form dài trong khu vực quản trị hoạt động mượt hơn và mã nguồn dễ tách thành các component nhỏ.

Zod được sử dụng cùng React Hook Form để kiểm tra dữ liệu phía frontend

[10]. Việc kiểm tra ở frontend không thay thế kiểm tra ở backend, nhưng giúp người dùng nhận phản hồi sớm khi nhập thiếu thông tin hoặc sai định dạng, đồng thời giảm số yêu cầu không hợp lệ gửi tới máy chủ. Các lựa chọn thay thế gồm Formik, Yup hoặc tự quản lý biểu mẫu bằng React state. Formik thường tạo nhiều mã lặp lại hơn với biểu mẫu lớn; tự quản lý bằng state phù hợp với biểu mẫu nhỏ nhưng khó bảo trì khi số trường và điều kiện tăng. Vì vậy, React Hook Form kết hợp Zod phù hợp với các màn hình nhập liệu của hệ thống.

3.2.4 Tailwind CSS và các thư viện hỗ trợ giao diện

Hệ thống sử dụng Tailwind CSS để xây dựng giao diện bằng các lớp tiện ích [23]. Cách tiếp cận này giúp việc tạo layout, khoảng cách, màu sắc, border, trạng thái hover và responsive được thực hiện trực tiếp trong component. Với một hệ thống có nhiều màn hình như trang học tập, trang chi tiết khóa học, bảng quản trị, form tạo bài kiểm tra và trình soạn thảo blog, Tailwind CSS giúp giao diện được phát triển nhanh mà vẫn giữ tính nhất quán.

Bên cạnh Tailwind CSS, frontend sử dụng một số thư viện nhỏ để làm giao diện gọn và rõ hơn. `clsx` và `tailwind-merge` hỗ trợ ghép class có điều kiện và xử lý xung đột class Tailwind. `lucide-react` cung cấp hệ icon nhất quán cho các nút thao tác, menu và trạng thái giao diện. `sonner` được dùng cho thông báo phản hồi như tạo thành công, lỗi upload hoặc lỗi lưu dữ liệu. `nprogress` giúp hiển thị trạng thái chuyển trang. Các thư viện này không quyết định kiến trúc hệ thống, nhưng giúp cải thiện tính dễ dùng đã nêu ở Mục 2.4.4.

3.2.5 Video.js, hls.js và trình phát bài giảng

Ở phía giao diện học tập, video bài giảng cần được phát ổn định, hỗ trợ tua, ghi nhận tiến độ và xử lý trường hợp video vẫn đang được backend xử lý. Hệ thống sử dụng Video.js trong component React để xây dựng trình phát video trên trình duyệt [24]. Đối với nguồn HLS, hls.js được sử dụng khi trình duyệt cần thư viện hỗ trợ để đọc playlist và các đoạn video. Cách kết hợp này phù hợp với pipeline media ở backend: video được upload lên R2, chuyển sang HLS bằng FFmpeg, sau đó frontend phát playlist HLS thay vì phát trực tiếp tệp gốc.

Trình phát video không chỉ có nhiệm vụ hiển thị video. Trong hệ thống, nó còn ghi nhận thời điểm xem, trạng thái đang phát, tạm dừng và hoàn thành để phục vụ tiến độ học tập của học sinh. Khi video chưa xử lý xong hoặc gặp lỗi, giao diện cần hiển thị trạng thái rõ ràng thay vì để màn hình trống. Các lựa chọn thay thế là dùng thẻ `video` thuần của HTML hoặc nhúng video từ nền tảng bên ngoài. Thẻ `video` thuần đơn giản nhưng thiếu nhiều tiện ích khi làm việc với HLS và trạng thái người học. Nhúng video bên ngoài triển khai nhanh nhưng làm giảm khả năng

kiểm soát quyền truy cập và dữ liệu tiến độ. Vì vậy, trình phát video riêng phù hợp hơn với yêu cầu của một LMS có học liệu nội bộ.

3.2.6 Editor Tiptap xử lý rich text

Hệ thống có chức năng viết blog và quản lý nội dung học tập, trong đó giáo viên hoặc quản trị viên cần nhập văn bản có định dạng như tiêu đề, danh sách, liên kết, hình ảnh, căn lề và các đoạn nhấn mạnh. Frontend sử dụng Tiptap để xây dựng trình soạn thảo rich text trên nền React [25]. Tiptap phù hợp vì có kiến trúc extension, cho phép thêm các khả năng như heading, link, image, underline và text align mà không phải tự xây dựng toàn bộ editor từ đầu.

Một lựa chọn thay thế là sử dụng textarea thuần hoặc một editor dựng sẵn ít khả năng tùy biến hơn. Textarea phù hợp với nội dung văn bản đơn giản, nhưng không đáp ứng tốt nhu cầu viết bài chia sẻ kiến thức có hình ảnh, liên kết và bố cục rõ ràng. Các editor dựng sẵn có thể triển khai nhanh, tuy nhiên khó kiểm soát cấu trúc dữ liệu và các nút thao tác theo đúng giao diện quản trị của hệ thống. Tiptap được lựa chọn vì vừa đủ linh hoạt cho nhu cầu viết blog, vừa cho phép mở rộng thêm các định dạng nội dung khi hệ thống phát triển.

3.2.7 Kéo thả, thông báo và trải nghiệm quản trị

Một số màn hình quản trị của hệ thống có tính chất builder, trong đó giáo viên hoặc quản trị viên không chỉ nhập dữ liệu mà còn phải tổ chức thứ tự nội dung. Ví dụ, khi xây dựng khóa học, người dùng cần sắp xếp chương, bài học và học liệu theo đúng lộ trình học; khi tạo bài kiểm tra, người dùng cần tổ chức các câu hỏi theo thứ tự mong muốn. Nếu chỉ nhập số thứ tự thủ công, thao tác này dễ gây nhầm lẫn và không phản ánh trực tiếp cấu trúc mà người dùng đang hình dung. Vì vậy, hệ thống sử dụng @hello-pangea/dnd để hỗ trợ kéo thả trên các màn hình builder. Cách tiếp cận này giúp thao tác sắp xếp tự nhiên hơn, giảm số bước nhập liệu và làm cho trải nghiệm tạo khóa học, tạo bài kiểm tra gần hơn với cách giáo viên chuẩn bị nội dung thực tế.

Ngoài thao tác kéo thả, frontend còn sử dụng một số thư viện nhỏ để hoàn thiện trải nghiệm sử dụng. sonner được dùng để hiển thị phản hồi sau các thao tác như lưu khóa học, upload media hoặc cập nhật bài kiểm tra; nprogress hiển thị tiến trình khi chuyển trang; js-cookie hỗ trợ làm việc với cookie phía trình duyệt; isomorphic-dompurify hỗ trợ làm sạch nội dung HTML trước khi hiển thị. Những thư viện này không phải trọng tâm lý thuyết của đề án, nhưng giúp giao diện quản trị dễ dùng hơn, giảm số tình huống người dùng không biết hệ thống đã xử lý đến đâu và hạn chế việc phải tự viết lại các xử lý phổ biến ở frontend.

3.3 Nền tảng lý thuyết về RAG

Ngoài các công nghệ phát triển web, đồ án còn sử dụng nền tảng lý thuyết liên quan đến hỏi đáp theo học liệu. Chức năng này khác với các chức năng quản lý thông thường ở chỗ câu trả lời cần được tạo bằng ngôn ngữ tự nhiên, nhưng vẫn phải bám vào nội dung học liệu do giáo viên cung cấp. Vì vậy, phần này trình bày ngắn gọn về mô hình ngôn ngữ lớn và kỹ thuật RAG, làm cơ sở cho phần thiết kế và triển khai ở chương sau.

3.3.1 Mô hình ngôn ngữ lớn

Các mô hình ngôn ngữ lớn có khả năng sinh văn bản tự nhiên và trả lời câu hỏi dựa trên ngữ cảnh đầu vào. Một nền tảng quan trọng của nhiều mô hình ngôn ngữ hiện đại là kiến trúc Transformer, được đề xuất trong công trình “Attention Is All You Need” [26]. Trong phạm vi đồ án, mô hình ngôn ngữ lớn được sử dụng như một dịch vụ hoặc mô hình tích hợp có sẵn, không phải đối tượng được huấn luyện lại từ đầu. Cách tiếp cận này phù hợp với đồ án ứng dụng vì mục tiêu chính là xây dựng hệ thống LMS hoàn chỉnh, không phải nghiên cứu huấn luyện mô hình mới.

Nếu chỉ gọi trực tiếp mô hình ngôn ngữ bằng câu hỏi của học sinh, câu trả lời có thể thiếu bám sát nội dung bài học hoặc chứa thông tin ngoài phạm vi học liệu. Một lựa chọn khác là chatbot theo luật cố định, nhưng cách này khó bao phủ các cách hỏi đa dạng của học sinh. Vì vậy, hệ thống cần một hướng tiếp cận kết hợp giữa khả năng sinh câu trả lời của mô hình ngôn ngữ và khả năng truy xuất nội dung liên quan từ học liệu nội bộ.

3.3.2 Retrieval-Augmented Generation

RAG là hướng tiếp cận kết hợp truy xuất thông tin và sinh ngôn ngữ. Thay vì để mô hình trả lời chỉ dựa trên tri thức đã học trong quá trình huấn luyện, hệ thống trước hết truy xuất các đoạn tài liệu liên quan, sau đó đưa các đoạn này vào ngữ cảnh để mô hình sinh câu trả lời [27]. Cách tiếp cận này phù hợp với yêu cầu hỏi đáp theo học liệu đã nêu ở Chương 2, vì câu trả lời cần dựa trên nội dung bài học thay vì chỉ dựa trên tri thức chung của mô hình.

Các lựa chọn thay thế cho RAG gồm fine-tuning mô hình hoặc xây dựng hệ hỏi đáp dựa trên luật. Fine-tuning yêu cầu dữ liệu huấn luyện đủ lớn, chi phí tính toán cao và quy trình đánh giá phức tạp, chưa phù hợp với phạm vi đồ án ứng dụng. Hệ hỏi đáp dựa trên luật dễ kiểm soát nhưng thiếu linh hoạt khi học sinh đặt câu hỏi theo nhiều cách khác nhau. RAG được lựa chọn vì có thể tận dụng mô hình có sẵn, bổ sung ngữ cảnh từ học liệu riêng và phù hợp với định hướng không huấn luyện mô hình từ đầu.

3.3.3 Dữ liệu ngữ cảnh và kiểm soát câu trả lời

Trong hệ thống LMS, học liệu của mỗi khóa học có phạm vi riêng. Một câu hỏi của học sinh khi đang học một bài cụ thể nên được ưu tiên trả lời dựa trên nội dung của bài đó hoặc các tài liệu liên quan trong khóa học. Vì vậy, dữ liệu ngữ cảnh của RAG cần được tổ chức theo các đơn vị học liệu như khóa học, chương, bài học, tài liệu. Khi nhận câu hỏi, hệ thống có thể tìm các đoạn liên quan, đưa vào prompt và yêu cầu mô hình trả lời trong phạm vi ngữ cảnh đã cung cấp.

Cách làm này không loại bỏ hoàn toàn rủi ro câu trả lời sai, nhưng giúp giảm tình huống mô hình trả lời xa nội dung bài học. Đồng thời, nó cũng phù hợp với định vị của đồ án: chức năng hỏi đáp đóng vai trò hỗ trợ học sinh tự học và giảm các câu hỏi lặp lại cho giáo viên, không thay thế vai trò chuyên môn của giáo viên. Những chi tiết như cách tách đoạn tài liệu, lưu vector, truy xuất ngữ cảnh và thiết kế prompt sẽ được trình bày ở phần thiết kế và triển khai hệ thống.

Chương này đã trình bày các công nghệ và nền tảng lý thuyết được sử dụng để hiện thực hóa các yêu cầu đã phân tích ở Chương 2. Phía máy chủ sử dụng Node.js, Express, TypeScript, PostgreSQL, Prisma, Zod, JWT, middleware kiểm tra vai trò và các cơ chế xử lý giao dịch để xây dựng API nghiệp vụ và bảo đảm tính nhất quán dữ liệu. Các media của hệ thống được xử lý bằng Cloudflare R2, presigned URL, FFmpeg, FFprobe và HLS để hỗ trợ upload, lưu trữ và phát video bài giảng. Phía giao diện sử dụng React, Next.js, TanStack Query, Axios, Zustand, React Hook Form, Tailwind CSS, Video.js và Tiptap để xây dựng các luồng học tập và quản trị trên trình duyệt. Đối với chức năng hỏi đáp theo học liệu, chương này đã trình bày cơ sở của mô hình ngôn ngữ lớn và kỹ thuật RAG. Trên cơ sở các công nghệ đã lựa chọn, Chương 4 sẽ trình bày chi tiết quá trình phân tích, thiết kế, triển khai và đánh giá hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã trình bày các nền tảng lý thuyết và công nghệ được lựa chọn để đáp ứng yêu cầu của hệ thống. Trên cơ sở đó, chương này trình bày cách các công nghệ được áp dụng vào thiết kế và hiện thực hệ thống quản lý học tập trực tuyến. Nội dung chương bao gồm lựa chọn kiến trúc, thiết kế tổng quan và chi tiết, thiết kế giao diện và cơ sở dữ liệu, công cụ phát triển, kết quả xây dựng, kiểm thử và môi trường triển khai thử nghiệm.

4.1 Thiết kế kiến trúc

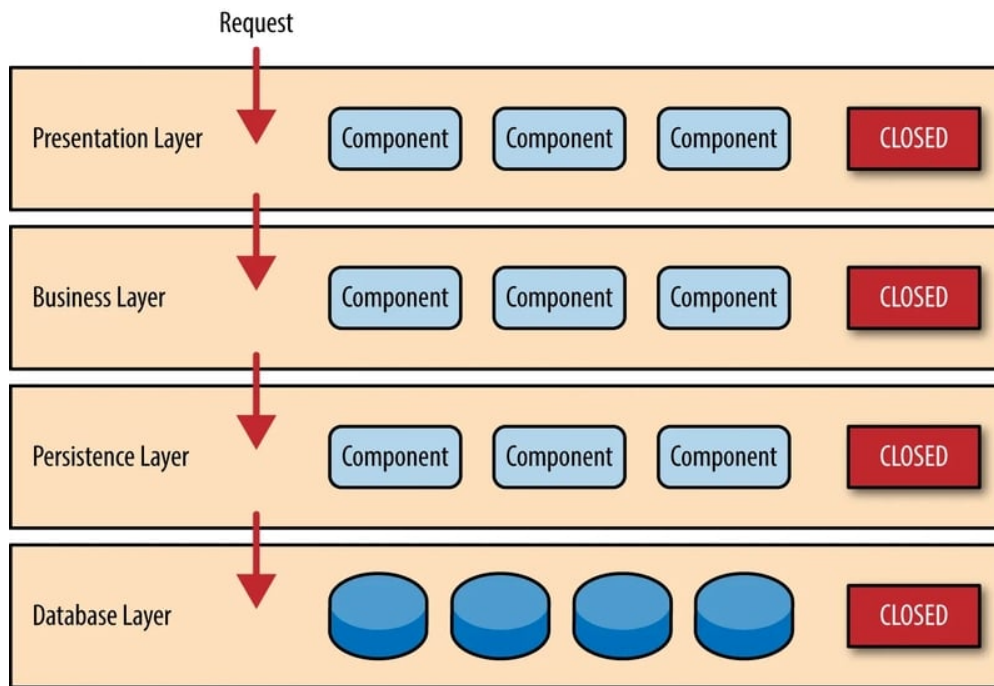
4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống trong đề án được thiết kế theo kiến trúc phân lớp (Layered Architecture). Đây là kiểu kiến trúc tổ chức phần mềm thành các lớp theo chiều ngang, trong đó mỗi lớp đảm nhiệm một nhóm trách nhiệm riêng. Theo mô tả của Mark Richards, kiến trúc phân lớp thường được triển khai với bốn lớp phổ biến là Presentation, Business, Persistence và Database [28]. Cách tổ chức này cũng phù hợp với các hệ thống ứng dụng doanh nghiệp, nơi cần tách giao diện người dùng, xử lý nghiệp vụ và truy cập dữ liệu thành các phần tương đối độc lập [29].

Kiến trúc phân lớp được lựa chọn vì phù hợp với yêu cầu dễ bảo trì, dễ mở rộng và dễ kiểm soát thay đổi đã nêu ở Mục 2.4. Hệ thống LMS trong đề án có nhiều nhóm chức năng như quản lý khóa học, quản lý bài kiểm tra, mua khóa học, xử lý media, theo dõi học tập và hỗ trợ hỏi đáp theo nội dung bài học. Nếu các chức năng này được viết lẫn giữa giao diện, xử lý nghiệp vụ và truy vấn dữ liệu, mã nguồn sẽ khó đọc và khó sửa khi hệ thống mở rộng. Việc chia lớp giúp mỗi phần của hệ thống có trách nhiệm rõ ràng hơn: giao diện tập trung vào tương tác với người dùng, service tập trung vào nghiệp vụ, còn repository và CSDL tập trung vào dữ liệu.

Trong phạm vi đề án, hệ thống không lựa chọn microservices vì quy mô hiện tại chưa cần tách thành nhiều dịch vụ độc lập. Nếu áp dụng microservices quá sớm, hệ thống sẽ phát sinh thêm nhiều vấn đề về triển khai, giám sát, giao tiếp giữa các dịch vụ và đồng bộ dữ liệu. Kiến trúc MVC cũng chưa đủ rõ để mô tả cách hệ thống được tổ chức, vì backend của đề án không chỉ có controller và model mà còn có service xử lý nghiệp vụ, repository truy cập dữ liệu, các thành phần xử lý media, thanh toán và AI. Vì vậy, kiến trúc phân lớp là lựa chọn phù hợp hơn: đủ rõ để tách trách nhiệm, nhưng không làm hệ thống phức tạp quá mức.

Kiến trúc phân lớp định hướng thiết kế hệ thống được trình bày trong Hình 4.1.



Hình 4.1: Kiến trúc phân lớp của hệ thống

Khi áp dụng vào hệ thống, lớp Presentation bao gồm giao diện Next.js ở phía người dùng và các route/controller Express ở phía backend. Lớp này chịu trách nhiệm hiển thị dữ liệu, nhận thao tác từ người dùng và chuyển yêu cầu vào hệ thống. Các thành phần cụ thể thuộc lớp này gồm các trang học tập, trang quản trị, form tạo khóa học, form tạo bài kiểm tra, trình phát video, các component hiển thị dữ liệu và các route tiếp nhận HTTP request.

Lớp Business được triển khai chủ yếu bằng các service trong backend. Đây là nơi xử lý các quy tắc nghiệp vụ như tạo khóa học, xuất bản khóa học, thiết lập bài kiểm tra, tạo đơn hàng mua khóa học, kiểm tra quyền truy cập bài học, ghi nhận tiến độ học tập, xử lý trạng thái media và điều phối yêu cầu hỏi đáp theo nội dung bài học. Lớp này quyết định một thao tác có hợp lệ hay không theo ngữ cảnh nghiệp vụ, thay vì chỉ kiểm tra dữ liệu ở mức định dạng.

Lớp Persistence gồm các repository, Prisma ORM và các thành phần làm việc với dịch vụ lưu trữ, xử lý video, thanh toán hoặc dịch vụ AI. Nhiệm vụ của lớp này là thực hiện truy vấn dữ liệu, cập nhật trạng thái, tạo đường dẫn tải tệp, lưu kết quả xử lý và che giấu chi tiết kỹ thuật khỏi lớp Business. Nhờ vậy, service không cần tự viết truy vấn CSDL hoặc tự xử lý chi tiết lưu trữ tệp.

Lớp Database bao gồm PostgreSQL và Cloudflare R2. PostgreSQL lưu trữ dữ liệu có cấu trúc như người dùng, khóa học, chương, bài học, bài kiểm tra, đơn hàng, giao dịch và tiến độ học tập. Cloudflare R2 lưu trữ các media dung lượng lớn như

ảnh, tài liệu và video bài giảng. Việc tách dữ liệu nghiệp vụ và media giúp backend không phải lưu trực tiếp các tệp lớn trên máy chủ, đồng thời thuận lợi hơn khi mở rộng dung lượng lưu trữ.

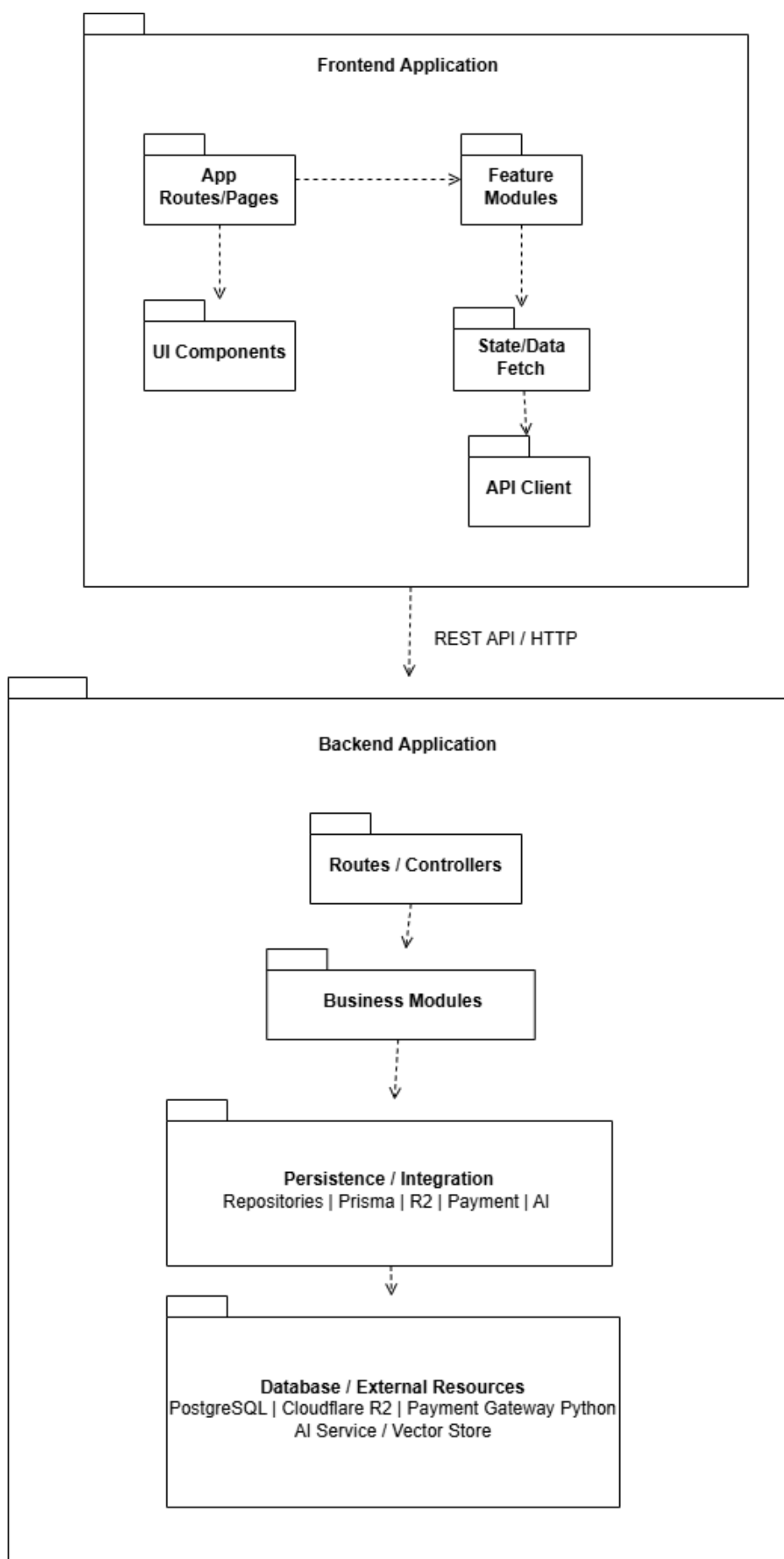
So với mô hình phân lớp lý thuyết, hệ thống có ba điều chỉnh để phù hợp với ứng dụng Web: (i) lớp Presentation gồm cả giao diện phía trình duyệt và route/controller tiếp nhận yêu cầu ở backend; (ii) lớp Persistence bao gồm CSDL quan hệ cùng các thành phần tích hợp lưu trữ media, xử lý video, thanh toán và AI; (iii) mã nguồn được chia theo module nghiệp vụ như khóa học, bài kiểm tra, đơn hàng, media và tutor. Cách tổ chức này vẫn giữ trách nhiệm riêng cho từng lớp và hạn chế ảnh hưởng lan truyền khi thay đổi thành phần kỹ thuật.

4.1.2 Thiết kế tổng quan

Để cụ thể hóa kiến trúc đã lựa chọn ở Mục 4.1.1, hệ thống được mô tả bằng biểu đồ gói UML ở mức tổng quan. Biểu đồ này tập trung vào cách chia các package chính, quan hệ phụ thuộc giữa các package và ranh giới giữa phần giao diện người dùng, phần máy chủ và các tài nguyên bên ngoài. Hình 4.2 trình bày cấu trúc tổng quan của hệ thống.

Ở mức tổng quan, hệ thống gồm hai khối phần mềm chính là Frontend Application và Backend Application. Khối Frontend Application chạy ở phía trình duyệt, được tổ chức theo các package như App Routes/Pages, Feature Modules, UI Components, State/Data Fetch và API Client. App Routes/Pages định nghĩa các tuyến màn hình cho học sinh, giáo viên và quản trị viên. Feature Modules gom các thành phần theo nhóm chức năng như khóa học, bài kiểm tra, đơn hàng, media hoặc xác thực. UI Components chứa các thành phần giao diện tái sử dụng. State/Data Fetch quản lý trạng thái giao diện và dữ liệu lấy từ máy chủ. API Client là package giao tiếp với Backend Application thông qua REST API.

Khối Backend Application chạy ở phía máy chủ và được tổ chức theo kiến trúc phân lớp. Tầng Presentation gồm Routes/Controllers, chịu trách nhiệm tiếp nhận request từ frontend, gọi middleware/validator cần thiết và chuyển yêu cầu xuống tầng nghiệp vụ. Tầng Business gồm các module nghiệp vụ chính như Auth, Users, Courses, Learning, Assessments, Orders, Payments, Media, Blogs và Tutor. Trong đó, Learning biểu diễn nhóm luồng học tập của học sinh, được hiện thực chủ yếu thông qua các route và service học tập trong module Courses, module Assessments và Tutor. Tầng Persistence/Integration gồm Repositories, Prisma ORM, R2 Storage, Video Processing, Payment Integration và AI Integration. Tầng Database/External Resources gồm PostgreSQL, Cloudflare R2, Payment Gateway và Python AI Service/Vector Store.



Hình 4.2: Biểu đồ gói tổng quan của hệ thống

Hình 4.3 trình bày chi tiết hơn cấu trúc phân tầng của khối Backend Application.

Quan hệ phụ thuộc trong thiết kế được giới hạn theo một chiều từ tầng trên xuống tầng dưới. Routes/Controllers phụ thuộc vào các module ở tầng Business để thực hiện nghiệp vụ. Các module nghiệp vụ không truy cập trực tiếp vào PostgreSQL, Cloudflare R2 hoặc dịch vụ thanh toán, mà thông qua các package ở tầng Persistence/Integration. Repositories và Prisma ORM đảm nhiệm truy vấn dữ liệu có cấu trúc trong PostgreSQL. R2 Storage và Video Processing phục vụ luồng upload, lưu trữ và xử lý video/tài liệu. Payment Integration chịu trách nhiệm kết nối với cổng thanh toán. AI Integration chịu trách nhiệm chuyển yêu cầu hỏi đáp từ Tutor sang Python AI Service.

Thiết kế này hạn chế phụ thuộc vòng bằng cách không cho tầng dưới gọi ngược lên tầng trên. Các module nghiệp vụ cũng được tách theo nhóm chức năng để giảm phụ thuộc chéo. Một số quan hệ giữa các module nghiệp vụ vẫn tồn tại do yêu cầu thực tế của hệ thống, ví dụ Orders cần thông tin khóa học để tạo đơn mua khóa học, Payments cập nhật trạng thái đơn hàng sau khi có giao dịch, Learning cần thông tin khóa học và bài kiểm tra để hiển thị lộ trình học tập, còn Tutor cần ngữ cảnh bài học để trả lời câu hỏi. Các quan hệ này được giữ ở mức một chiều và có chủ đích, nhằm đảm bảo cấu trúc tổng thể vẫn bám theo kiến trúc phân lớp.

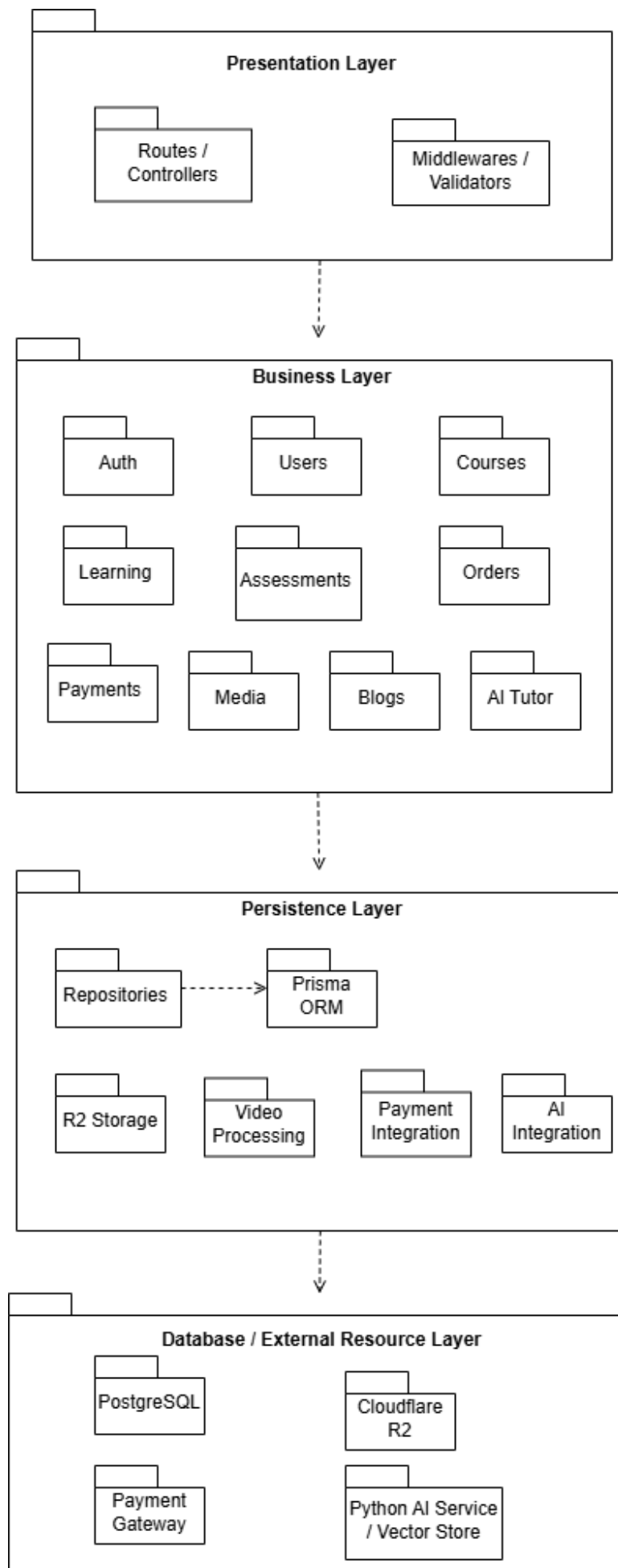
4.1.3 Thiết kế chi tiết gói

Sau khi xác định cấu trúc package tổng quan, đề án lựa chọn hai nhóm chức năng tiêu biểu để trình bày thiết kế chi tiết gói. Các biểu đồ trong mục này chỉ thể hiện tên lớp và quan hệ giữa các lớp, chưa trình bày thuộc tính hoặc phương thức. Hai nhóm chức năng được lựa chọn là tạo khóa học và upload video bài giảng, vì đây là hai luồng thể hiện rõ cách hệ thống áp dụng kiến trúc phân lớp từ giao diện, controller, service, repository đến hạ tầng lưu trữ.

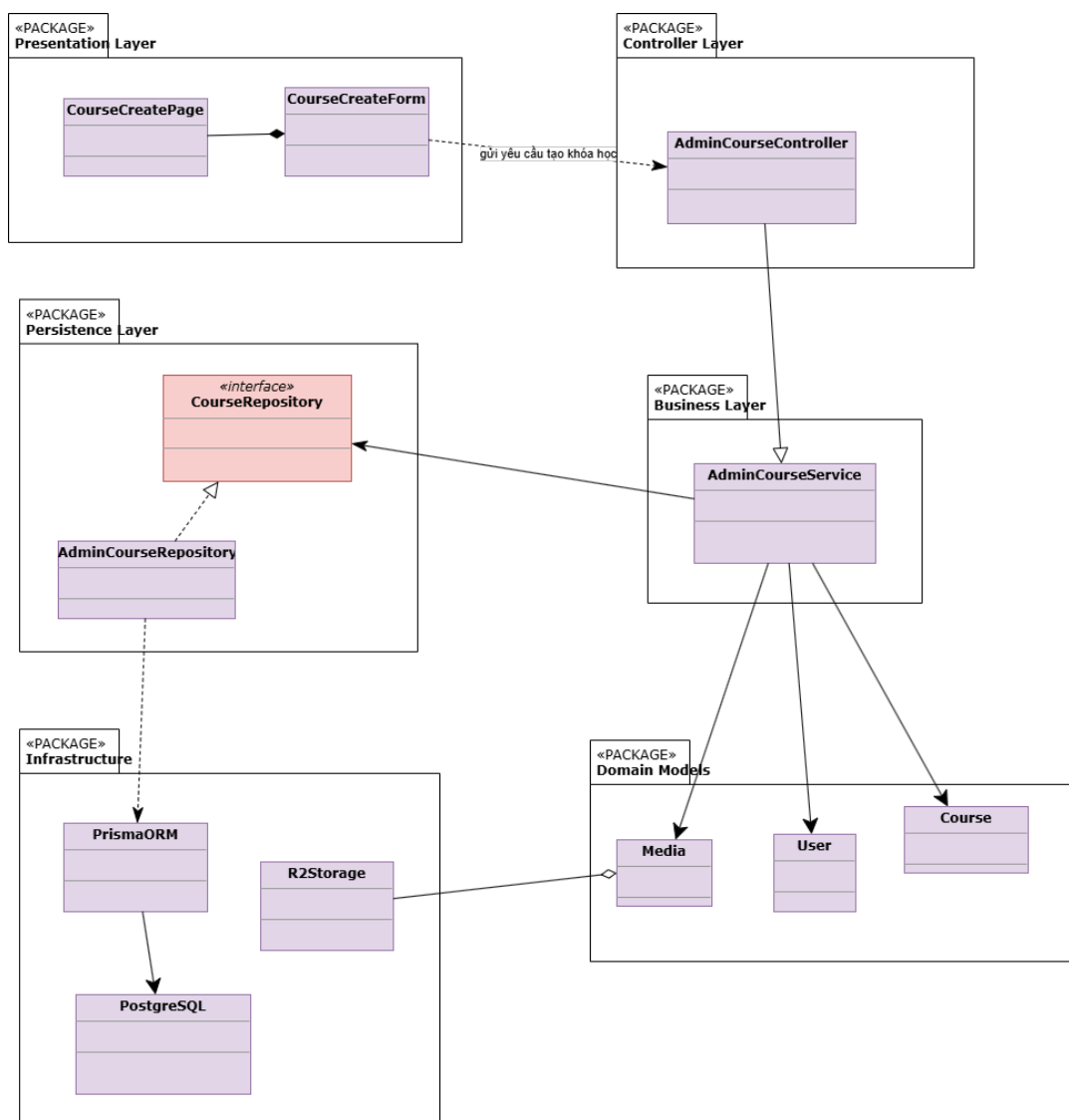
a, Thiết kế chi tiết gói chức năng tạo khóa học

Hình 4.4 trình bày thiết kế chi tiết cho chức năng tạo khóa học. Ở tầng Presentation, trang tạo khóa học chứa form nhập thông tin khóa học. Form này gửi yêu cầu tạo khóa học tới `AdminCourseController` ở tầng Controller. Controller không xử lý trực tiếp nghiệp vụ mà chuyển yêu cầu xuống `AdminCourseService`.

`AdminCourseService` là lớp xử lý nghiệp vụ chính của chức năng này, bao gồm kiểm tra dữ liệu, xác định người tạo khóa học và chuẩn bị thông tin cần lưu. Service phụ thuộc vào `CourseRepository` để thực hiện thao tác lưu trữ. `AdminCourseRepository` là lớp triển khai repository, làm việc với `PrismaORM` để ghi dữ liệu xuống PostgreSQL. Các lớp `Course`, `User` và `Media` biểu diễn



Hình 4.3: Biểu đồ gói phân tầng phía máy chủ



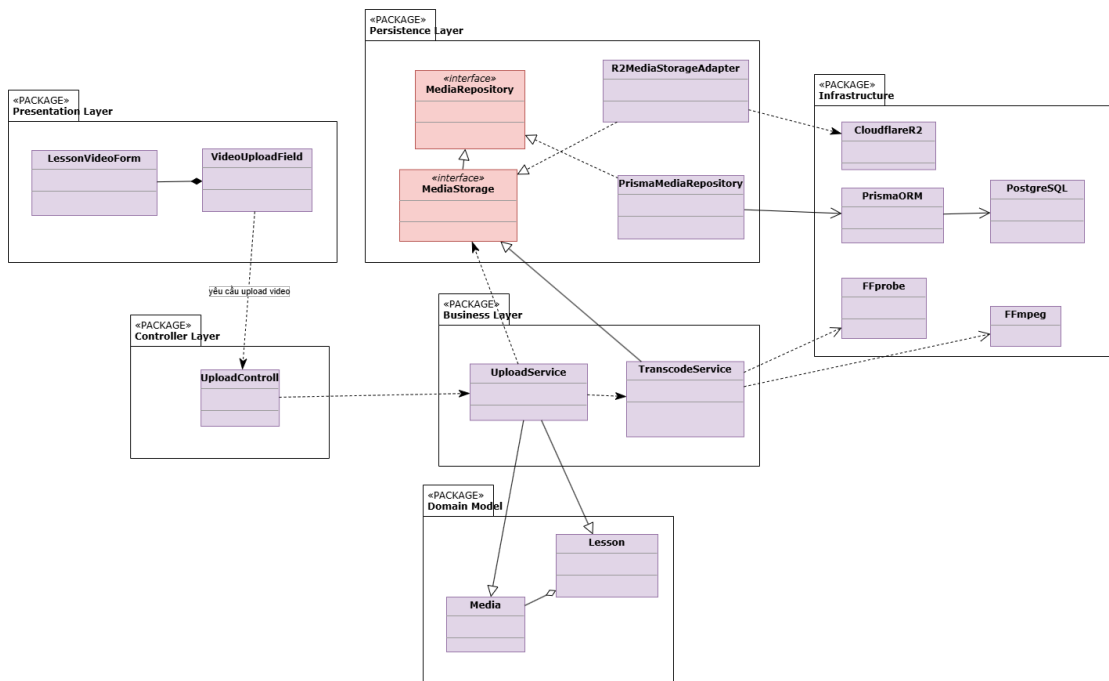
Hình 4.4: Thiết kế chi tiết gói chức năng tạo khóa học

các mô hình dữ liệu liên quan trong chức năng tạo khóa học. Trong đó, Media được sử dụng khi khóa học có ảnh đại diện hoặc tài nguyên media liên quan được lưu trên R2.

Biểu đồ thể hiện quan hệ hợp thành giữa trang tạo khóa học và form tạo khóa học, quan hệ phụ thuộc từ form tới controller, quan hệ kết hợp/phụ thuộc giữa controller, service và repository, đồng thời thể hiện quan hệ thực thi giữa Admin-CourseRepository và CourseRepository. Cách tổ chức này giữ cho phần giao diện, xử lý nghiệp vụ và truy cập dữ liệu không bị trộn lẫn với nhau.

b, Thiết kế chi tiết gói chức năng upload video bài giảng

Hình 4.5 trình bày thiết kế chi tiết cho chức năng upload video bài giảng và xử lý HLS. Đây là chức năng có nhiều thành phần kỹ thuật hơn so với tạo khóa học, vì hệ thống không chỉ lưu thông tin video trong CSDL mà còn phải đưa tệp lên Cloudflare R2 và chuyển đổi video sang định dạng phù hợp cho việc phát trên trình duyệt.



Hình 4.5: Thiết kế chi tiết gói chức năng upload video bài giảng

Ở tầng Presentation, LessonVideoForm chứa thành phần VideoUploadField để người quản lý nội dung chọn và tải video bài giảng. Thành phần này gửi yêu cầu upload tới UploadController. Controller chuyển tiếp yêu cầu cho UploadService. UploadService chịu trách nhiệm tạo bản ghi media, làm việc với kho lưu trữ thông qua MediaStorage và kích hoạt TranscodeService khi cần xử lý video.

Tầng Persistence gồm MediaRepository, MediaStorage, PrismaMe-

MediaRepository và R2MediaStorageAdapter. Trong đó, PrismaMediaRepository triển khai MediaRepository để lưu thông tin media thông qua Prisma ORM, còn R2MediaStorageAdapter triển khai MediaStorage để làm việc với Cloudflare R2. TranscodeService sử dụng FFprobe để đọc thông tin video và FFmpeg để chuyển đổi video sang HLS. Các mô hình Media và Lesson thể hiện mối liên hệ giữa tệp video và bài học sử dụng video đó.

Thiết kế này giúp tách rõ ba phần: giao diện upload video, nghiệp vụ quản lý trạng thái media và hạ tầng xử lý/lưu trữ video. Nhờ vậy, nếu thay đổi cách lưu trữ tệp hoặc thay đổi công cụ xử lý video, các thay đổi chủ yếu nằm ở tầng Persistence/Infrastructure mà không làm ảnh hưởng trực tiếp tới controller hoặc giao diện.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Hệ thống được thiết kế dưới dạng ứng dụng Web responsive, phục vụ hai nhóm thao tác chính: học sinh học tập trên trình duyệt và giáo viên/quản trị viên quản lý nội dung trên màn hình lớn. Đối với các màn hình quản trị như tạo khóa học, sắp xếp chương/bài học, upload học liệu và tạo bài kiểm tra, giao diện ưu tiên độ phân giải desktop từ 1366x768 trở lên để đảm bảo đủ không gian cho bảng dữ liệu, form nhập liệu và khu vực thao tác. Đối với các màn hình học tập, giao diện được thiết kế có khả năng thích ứng với màn hình nhỏ hơn bằng cách thu gọn sidebar, chuyển một số panel sang dạng ẩn/hiện và giữ khu vực nội dung chính để đọc.

Về định hướng thị giác, hệ thống sử dụng giao diện nền tối làm mặc định để phù hợp với các phiên học dài, đặc biệt khi học sinh xem video hoặc đọc tài liệu trong thời gian liên tục. Màu nhấn được dùng có kiểm soát cho hành động chính, trạng thái đang chọn và các thông tin cần chú ý; các trạng thái thành công, cảnh báo và lỗi được quy ước nhất quán trong thông báo, validation và trạng thái xử lý. Bên cạnh đó, hệ thống vẫn hỗ trợ chế độ sáng để phù hợp với các môi trường sử dụng khác nhau.

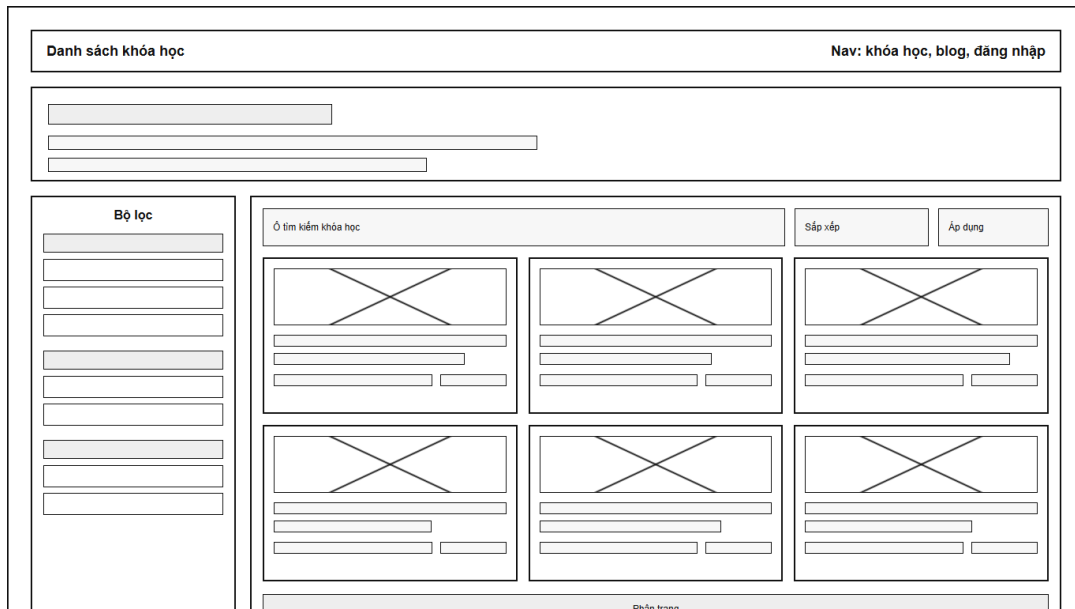
Về typography, giao diện sử dụng Be Vietnam Pro làm font chính cho nội dung, nhãn, nút bấm và dữ liệu bảng; font Quicksand được dùng cho một số tiêu đề cần nhấn mạnh. Cách lựa chọn này giúp hiển thị tiếng Việt rõ ràng, đồng thời giữ được cảm giác học thuật và thân thiện của hệ thống. Kích thước chữ trong giao diện được chia theo vai trò: tiêu đề trang có kích thước lớn hơn, nội dung thường là 16px, nhãn và nút bấm có kích thước nhỏ hơn để phù hợp với các màn hình quản trị nhiều thông tin.

Các thành phần giao diện được chuẩn hóa để người dùng không phải học lại thao

tác ở từng màn hình. Nút bấm có bốn biến thể chính gồm *primary*, *secondary*, *outline* và *ghost*; trong đó *primary* dùng cho hành động chính như lưu, tạo mới, mua khóa học hoặc bắt đầu làm bài. Trường nhập liệu có nhãn rõ ràng, viền, trạng thái focus và trạng thái lỗi. Các thao tác bất đồng bộ như lưu dữ liệu, upload media hoặc tạo bài kiểm tra có trạng thái loading và thông báo phản hồi. Hệ thống sử dụng toast ở góc trên bên phải để thông báo thành công, lỗi, cảnh báo hoặc thông tin; những thao tác có khả năng làm thay đổi dữ liệu quan trọng được kết hợp với hộp xác nhận hoặc trạng thái cảnh báo trực tiếp trên màn hình.

Về bố cục, các màn hình công khai như danh sách khóa học ưu tiên khả năng tìm kiếm và so sánh nhanh. Các màn hình học tập ưu tiên việc tập trung vào bài giảng, giữ video hoặc nội dung bài học ở vùng trung tâm, lộ trình khóa học ở một bên và công cụ hỗ trợ học tập ở bên còn lại. Các màn hình quản trị ưu tiên thao tác lặp lại: danh sách, bộ lọc, form, nút hành động và thông báo trạng thái được đặt ở vị trí nhất quán. Hai hình dưới đây là wireframe minh họa định hướng bố cục, không phải giao diện sản phẩm sau cùng.

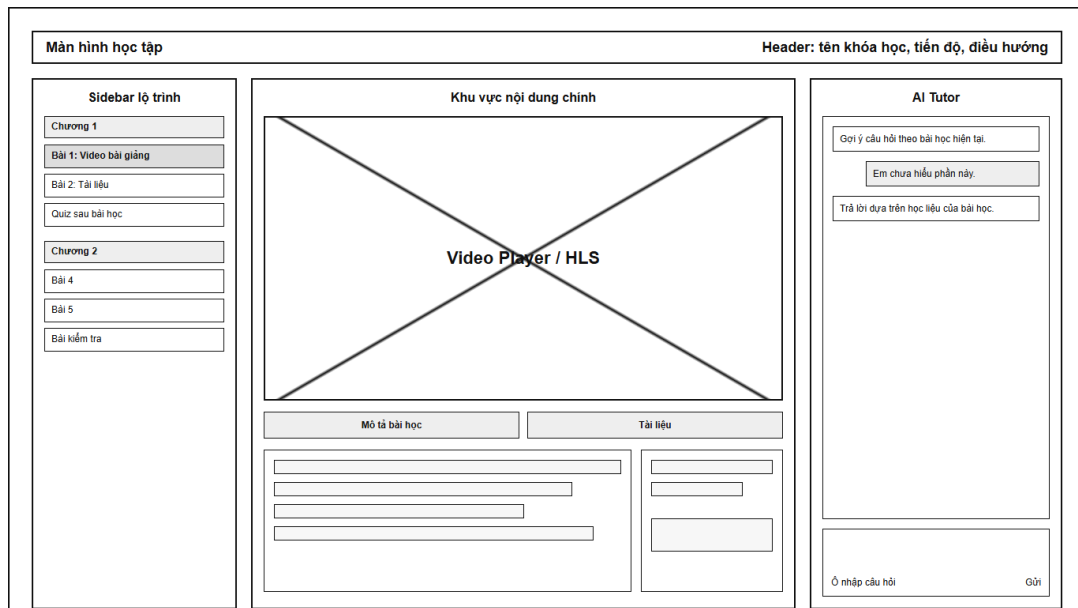
Bố cục màn hình danh sách khóa học được trình bày trong Hình 4.6. Màn hình gồm khu vực điều hướng, phần giới thiệu ngắn, bộ lọc ở bên trái và danh sách khóa học ở vùng nội dung chính. Khu vực danh sách có ô tìm kiếm, lựa chọn sắp xếp, các thẻ khóa học và phân trang. Cách bố trí này giúp học sinh hoặc khách truy cập lọc và so sánh khóa học trong cùng một màn hình.



Hình 4.6: Wireframe màn hình danh sách khóa học

Bố cục màn hình học tập trong Hình 4.7 được chia thành ba vùng: lộ trình ở bên trái, nội dung bài học ở giữa và AI Tutor ở bên phải. Khu vực trung tâm ưu tiên trình phát video/HLS, phía dưới là các tab mô tả bài học và tài liệu. Thiết kế này

cho phép học sinh xem bài giảng, truy cập học liệu và đặt câu hỏi trong cùng một ngữ cảnh.



Hình 4.7: Wireframe màn hình học tập

4.2.2 Thiết kế lớp

Phần này trình bày thiết kế của hai lớp nghiệp vụ trung tâm là Course và Assessment, cùng các biểu đồ trình tự cho bốn chức năng tiêu biểu. Các lớp được mô tả ở mức nghiệp vụ: thuộc tính phản ánh dữ liệu cần quản lý, còn phương thức thể hiện các thao tác chính của hệ thống. Khi cài đặt, các thao tác này được phân tách vào controller, service và repository theo kiến trúc phân lớp đã trình bày ở Mục 4.1.

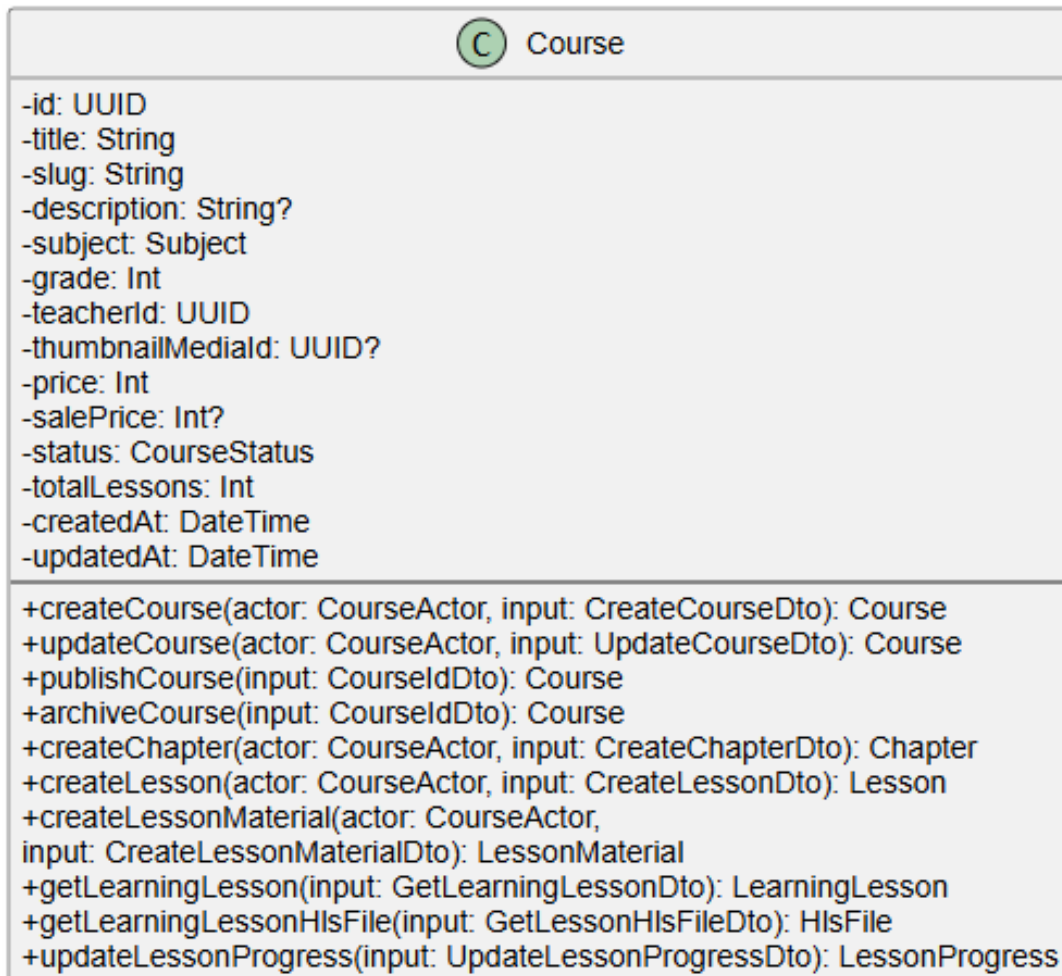
a, Thiết kế lớp Course

Lớp Course đại diện cho khóa học trong hệ thống. Lớp này quản lý thông tin nhận diện, môn học, khối lớp, giáo viên phụ trách, giá bán và trạng thái xuất bản của khóa học. Hình 4.8 mô tả các thuộc tính và phương thức chính của lớp Course.

Các phương thức createCourse, updateCourse, publishCourse và archiveCourse hỗ trợ quản lý vòng đời khóa học. createChapter, createLesson và createLessonMaterial dùng để xây dựng cấu trúc khóa học theo thứ tự khóa học - chương - bài học - học liệu. Với học sinh, getLearningLesson tải nội dung bài học, getLearningLessonHlsFile cung cấp luồng video và updateLessonProgress ghi nhận tiến độ học tập.

b, Thiết kế lớp Assessment

Lớp Assessment đại diện cho bài kiểm tra trong hệ thống. Đối tượng này được sử dụng cho nhiều ngữ cảnh khác nhau: bài kiểm tra công khai, bài kiểm tra gắn với khóa học và bài quiz gắn với bài học. Hình 4.9 mô tả các thuộc tính và phương



Hình 4.8: Biểu đồ lớp Course

thức chính của lớp Assessment.



Hình 4.9: Biểu đồ lớp Assessment

Các thuộc tính của Assessment mô tả thông tin cơ bản của bài kiểm tra như môn học, khối lớp, loại bài kiểm tra, hình thức chấm điểm, thời gian làm bài, trạng thái hiển thị, tệp nguồn và người tạo. Nhóm phương thức createAssessment, updateAssessment, createSection, createSectionItems, createPlacement và publishAssessment thể hiện quá trình xây dựng và xuất bản bài kiểm tra. Nhóm phương thức startAttempt, getAssessmentWorkspace, saveAnswers và submitAttempt thể hiện quá trình học sinh làm bài, lưu câu trả lời và nộp bài. Với các câu hỏi tự luận, hệ thống có thêm gradeEssay và finalizeManualSubmission để giáo viên chấm điểm và hoàn tất kết quả cuối cùng.

c, Biểu đồ trình tự học bài

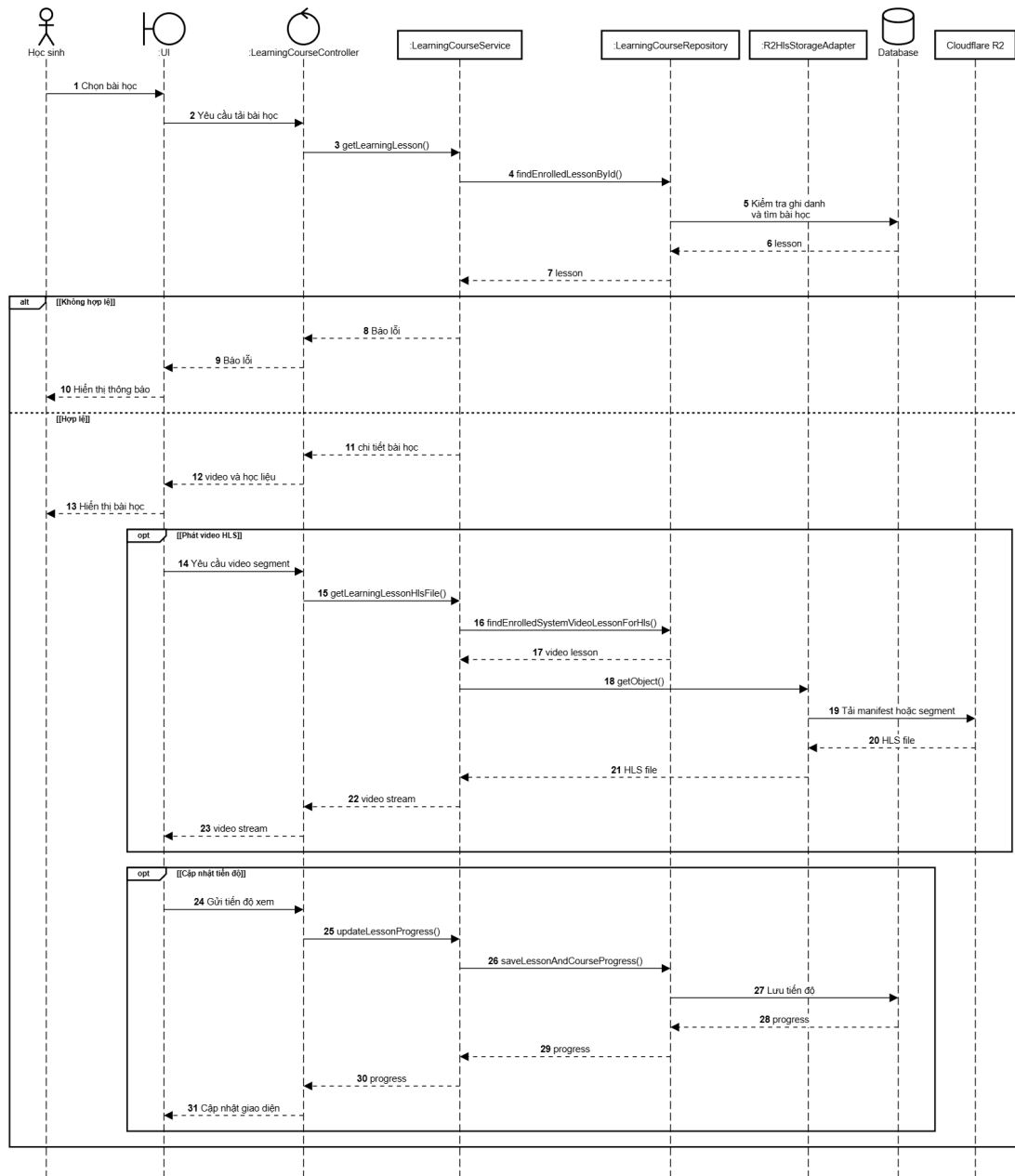
Hình 4.10 mô tả luồng xử lý khi Học sinh chọn một bài học từ màn hình học tập. Giao diện gửi yêu cầu tải bài học đến `LearningCourseController`; `controller` lấy định danh người dùng từ phiên đăng nhập và gọi `LearningCourseService.getLearningLesson`. `Service` gọi `LearningCourseRepository.findEnrolledLessonById`, nơi Prisma truy vấn dữ liệu ghi danh, bài học, video và học liệu trong PostgreSQL. Nếu học sinh chưa ghi danh hoặc bài học không tồn tại trong khóa học đã ghi danh, `service` trả lỗi để giao diện hiển thị thông báo. Ngược lại, `controller` trả chi tiết bài học để giao diện hiển thị nội dung video và tài liệu.

Khi trình phát cần một tệp HLS, giao diện tiếp tục yêu cầu manifest hoặc segment video. `Controller` gọi `getLearningLessonHlsFile`; `service` kiểm tra lại bài học video thuộc quyền học của học sinh qua repository, sau đó gọi `R2HlsStorageAdapter.getObject` để lấy tệp từ Cloudflare R2 và chuyển luồng dữ liệu về trình phát. Trong quá trình xem, giao diện gửi vị trí xem hiện tại đến `controller`. `LearningCourseService.updateLessonProgress` gọi repository lưu tiến độ bài học và tiến độ khóa học bằng Prisma, rồi trả trạng thái mới để giao diện cập nhật.

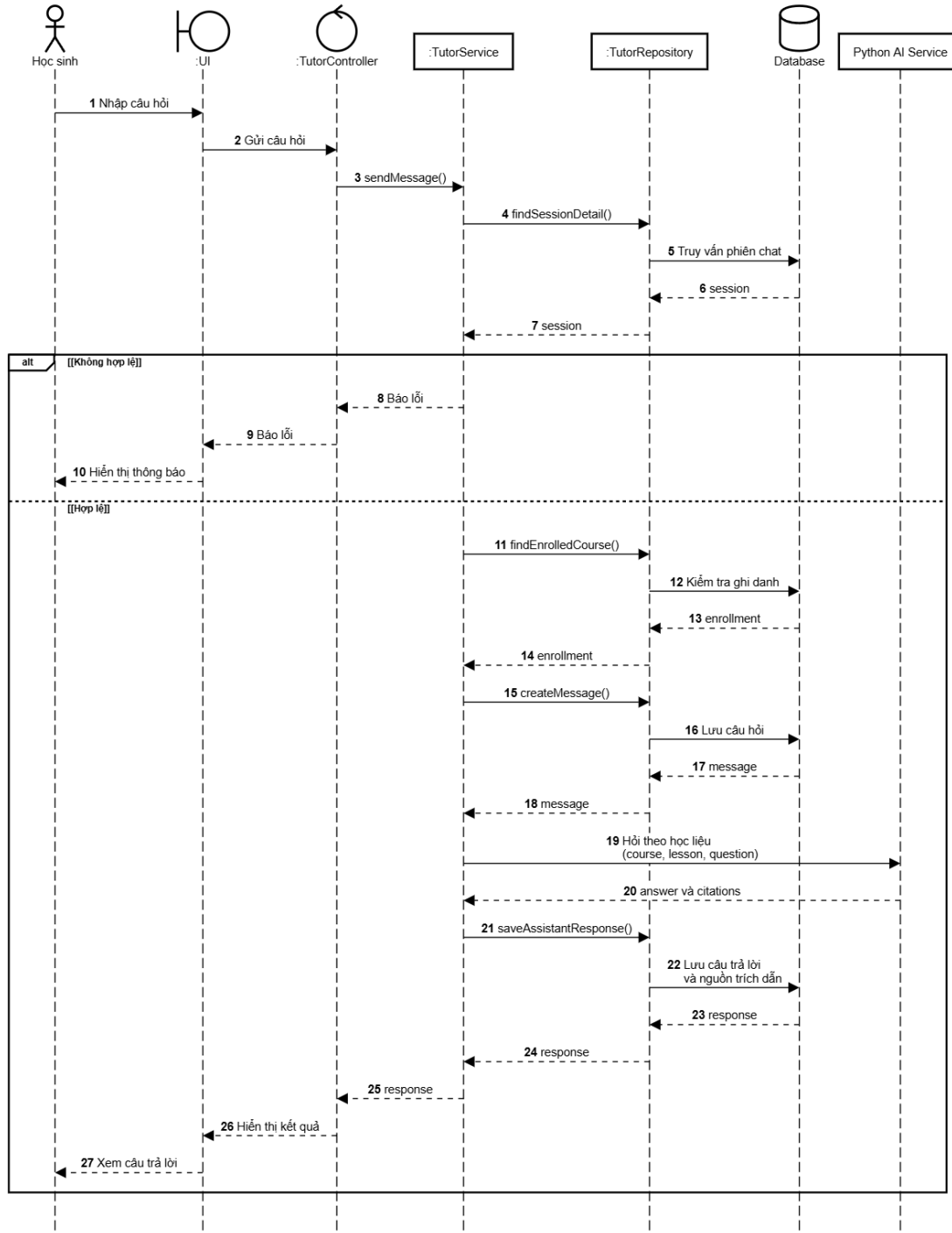
d, Biểu đồ trình tự hỏi đáp với AI Tutor

Hình 4.11 mô tả luồng Học sinh gửi câu hỏi trong khung chat của một bài học. Khi học sinh nhập và gửi câu hỏi, giao diện gọi `TutorController`; `controller` chuyển dữ liệu người dùng và nội dung câu hỏi cho `TutorService.sendMessage`. `Service` dùng `TutorRepository.findSessionDetail` để lấy phiên chat và lịch sử trao đổi bằng Prisma, sau đó dùng `findEnrolledCourse` kiểm tra học sinh có quyền học khóa học chứa bài học đang hỏi hay không. Nếu phiên chat, khóa học hoặc quyền truy cập không hợp lệ, `controller` trả lỗi để giao diện thông báo cho học sinh.

Với yêu cầu hợp lệ, `service` gọi `createMessage` để lưu câu hỏi của học sinh vào PostgreSQL trước khi gửi ngữ cảnh gồm khóa học, bài học và câu hỏi đến Python AI Service. Dịch vụ AI truy xuất học liệu liên quan và trả về nội dung phản hồi cùng các nguồn trích dẫn. `TutorService` gọi `saveAssistantResponse` để Prisma lưu tin nhắn trả lời, các nguồn trích dẫn và thời điểm trao đổi cuối của phiên chat; sau đó `controller` trả dữ liệu này về giao diện để hiển thị câu trả lời cho học sinh.



Hình 4.10: Biểu đồ trình tự học bài



Hình 4.11: Biểu đồ trình tự hỏi đáp với AI Tutor

e, Biểu đồ trình tự tạo và xuất bản bài kiểm tra

Hình 4.12 mô tả luồng Người quản lý nội dung tạo và xuất bản một bài kiểm tra. Luồng bắt đầu khi người dùng nhập thông tin cơ bản trên màn hình soạn đề và gửi yêu cầu đến `AdminAssessmentController`. Controller gọi `AdminAssessmentService.createAssessment`; service chuyển yêu cầu cho repository để Prisma tạo bản ghi bài kiểm tra ở trạng thái nháp trong PostgreSQL và trả lại mã bài kiểm tra cho giao diện.

Trong giai đoạn soạn đề, mỗi thao tác thêm phần đề hoặc câu hỏi được controller chuyển đến service qua `createSection` và `createSectionItems`. Repository lưu cấu trúc phần đề, câu hỏi và thứ tự hiển thị bằng Prisma. Tiếp theo, người quản lý nội dung thiết lập vị trí sử dụng, thời gian mở - đóng và số lần làm bài; controller gọi `createPlacement` để repository lưu cấu hình này. Khi người dùng chọn xuất bản, controller gọi `publishAssessment`. Service dùng `findAssessmentForPublish` lấy lại cấu trúc bài kiểm tra từ PostgreSQL để kiểm tra các điều kiện xuất bản. Nếu thiếu dữ liệu cần thiết, hệ thống trả lỗi về giao diện; nếu hợp lệ, service gọi repository cập nhật trạng thái bài kiểm tra sang xuất bản và trả kết quả để giao diện thông báo thành công.

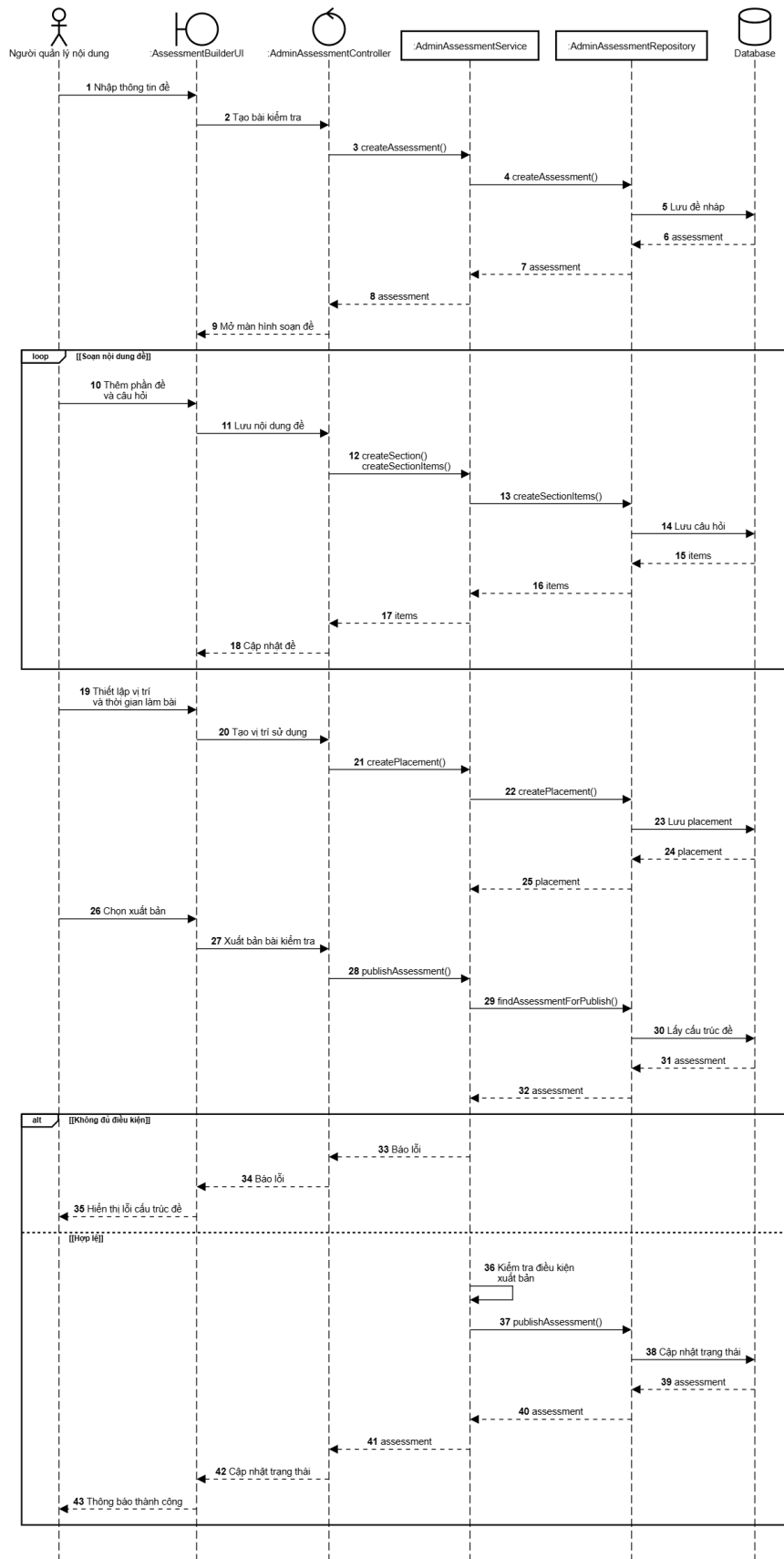
f, Biểu đồ trình tự nộp bài kiểm tra

Hình 4.13 mô tả luồng Học sinh xác nhận nộp bài kiểm tra từ màn hình làm bài. Giao diện gửi yêu cầu đến `StudentAssessmentController`; controller chuyển định danh học sinh và bài làm cho `StudentAssessmentService.submitAttempt`. Service gọi `StudentAssessmentRepository.findSubmissionForStudent`; repository dùng Prisma lấy bài làm, câu trả lời đã lưu, cấu hình chấm điểm và đáp án cần thiết từ PostgreSQL. Bước này bảo đảm bài làm thuộc về học sinh và vẫn còn ở trạng thái có thể nộp. Nếu không thỏa điều kiện, service trả lỗi để giao diện thông báo và không thay đổi dữ liệu bài làm.

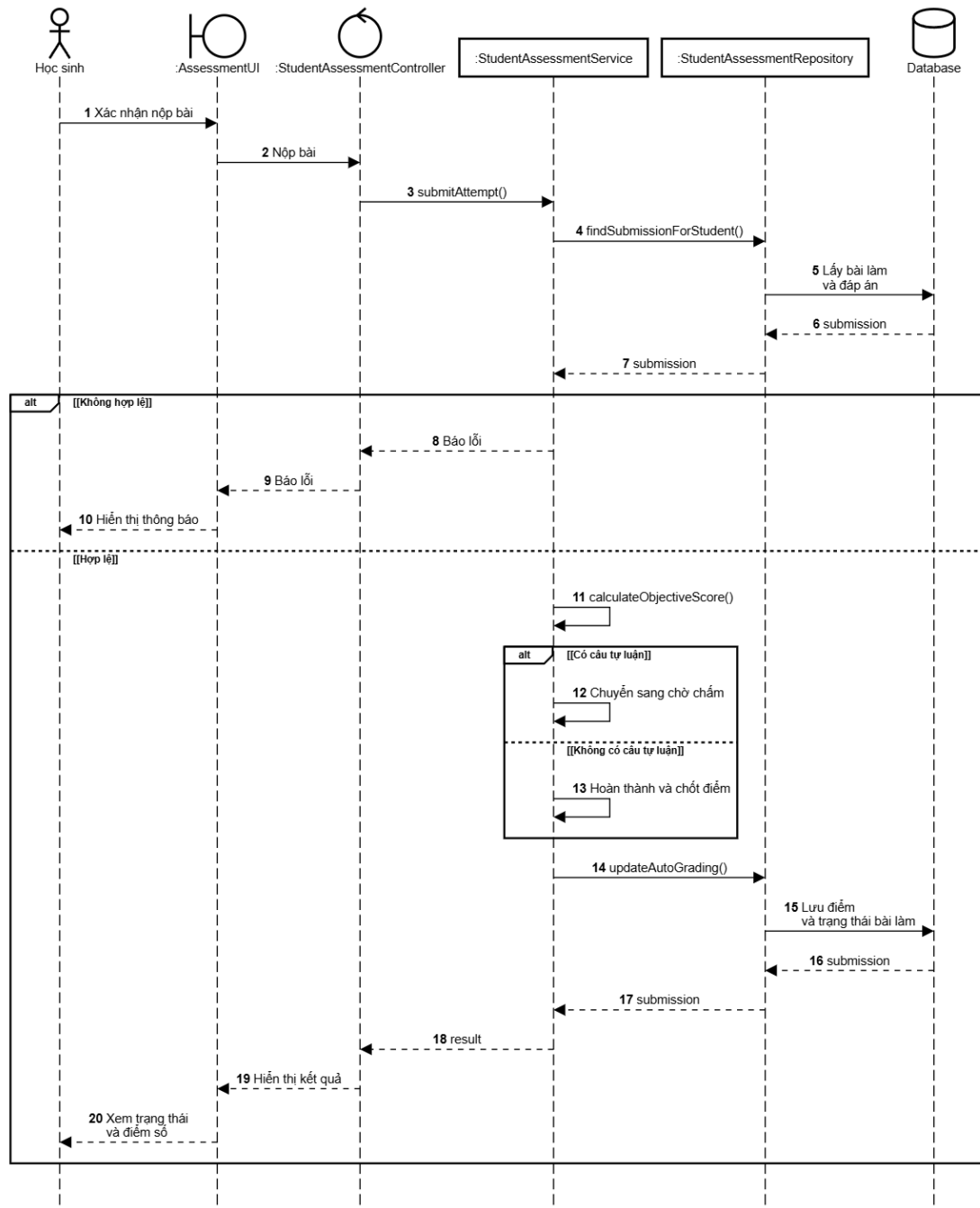
Với bài làm hợp lệ, dịch vụ `calculateObjectiveScore` tính điểm các câu trắc nghiệm, đúng sai và số. Bài có câu tự luận được chuyển sang trạng thái chờ giáo viên chấm; các trường hợp còn lại được chốt điểm tự động. Hàm `updateAutoGrading` sau đó cập nhật điểm, trạng thái và thời điểm nộp bằng Prisma. Kết quả được trả về giao diện để học sinh xem điểm hoặc trạng thái chờ chấm.

4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng PostgreSQL để lưu trữ dữ liệu nghiệp vụ có cấu trúc. Để tập trung vào luồng cốt lõi, biểu đồ thực thể liên kết mô hình hóa các nhóm bảng chính gồm người dùng, khóa học, bài học, học liệu, đơn hàng, thanh toán, bài kiểm



Hình 4.12: Biểu đồ trình tự tạo và xuất bản bài kiểm tra



Hình 4.13: Biểu đồ trình tự nộp bài kiểm tra

tra, tiến độ học tập và dữ liệu AI Tutor. Các bảng kỹ thuật như phiên đăng nhập, webhook, idempotency, thông báo, nhật ký và blog vẫn thuộc lược đồ vật lý nhưng không được đưa vào biểu đồ này.

a, Biểu đồ thực thể liên kết

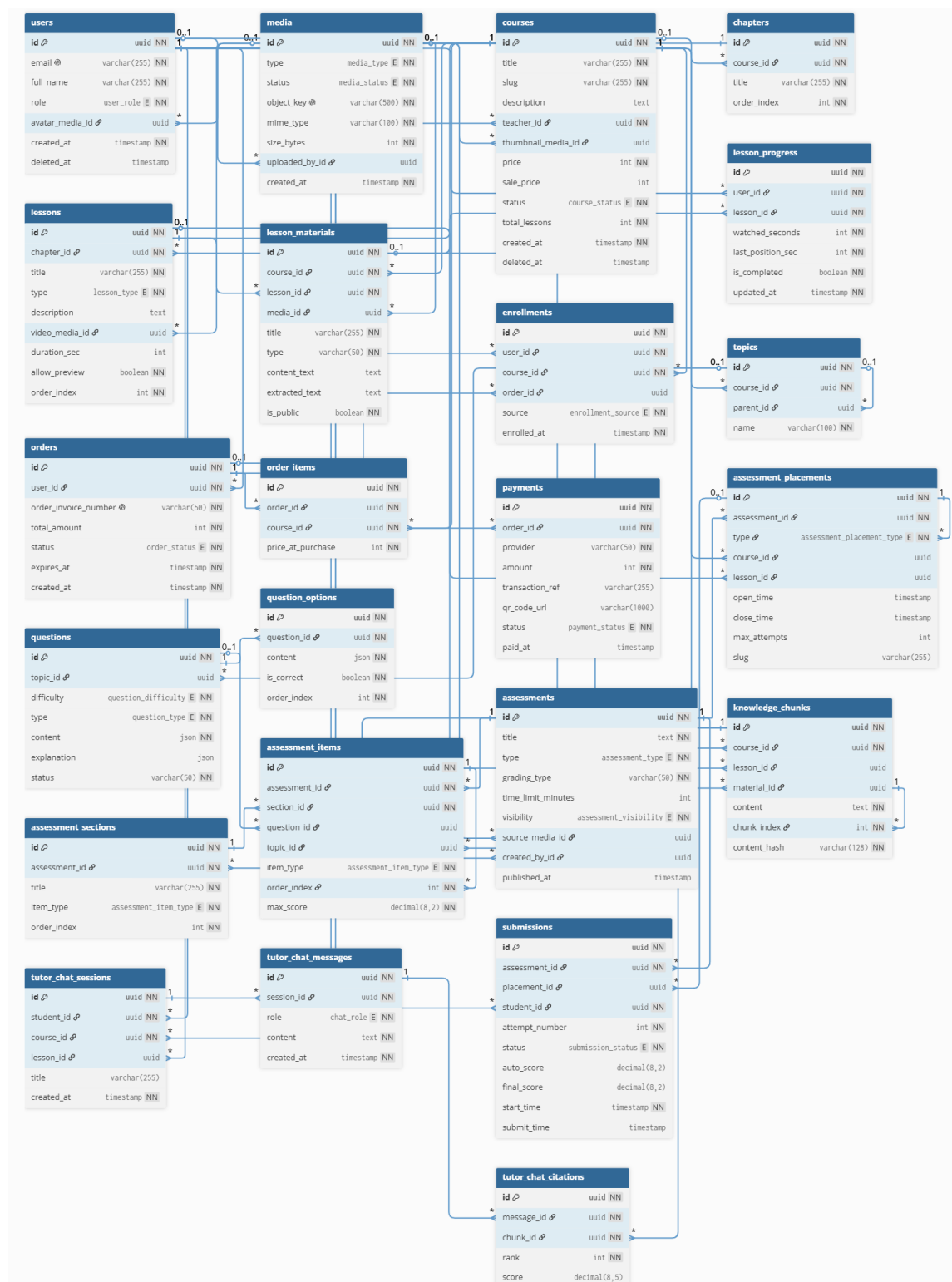
Hình 4.14 trình bày biểu đồ thực thể liên kết của cơ sở dữ liệu ở mức lõi. Biểu đồ thể hiện các quan hệ chính giữa người dùng, khóa học, bài học, thanh toán, bài kiểm tra và dữ liệu hỏi đáp theo học liệu.

Trong biểu đồ, bảng `users` là bảng trung tâm cho các vai trò học sinh, giáo viên và quản trị viên. Một giáo viên có thể phụ trách nhiều khóa học thông qua quan hệ từ `courses.teacher_id` tới `users.id`. Mỗi khóa học được tổ chức theo cấu trúc `courses - chapters - lessons`; trong đó mỗi bài học có thể có video, tài liệu đính kèm và các nội dung được trích xuất để phục vụ tìm kiếm ngữ cảnh. Bảng `media` được tách riêng để quản lý thông tin các tệp như ảnh, video và tài liệu, trong khi tệp thật được lưu ở dịch vụ lưu trữ đối tượng.

Nhóm bảng thanh toán gồm `orders`, `order_items`, `payments` và `enrollments`. Khi học sinh mua khóa học, hệ thống tạo đơn hàng, ghi nhận các khóa học trong đơn, theo dõi trạng thái thanh toán và tạo bản ghi ghi danh khi giao dịch hợp lệ. Cách tách này giúp hệ thống phân biệt rõ giữa ý định mua khóa học, giao dịch thanh toán và quyền truy cập khóa học của học sinh.

Nhóm bảng bài kiểm tra được tổ chức quanh `assessments`. Mỗi bài kiểm tra được cấu hình cho một `assessment_placement` cụ thể: công khai, thuộc khóa học hoặc gắn với bài học dạng quiz. Khi cần dùng lại nội dung đề ở vị trí khác, giáo viên tạo bản sao thay vì dùng chung một `assessment`, nhờ đó lịch mở bài, lượt làm và kết quả không bị trộn giữa các đợt. Nội dung đề được chia thành `assessment_sections` và `assessment_items`; các câu hỏi có thể lấy từ ngân hàng `questions` và `question_options`. Bài làm của học sinh được lưu trong `submissions`, liên kết với bài kiểm tra, vị trí sử dụng và học sinh thực hiện.

Nhóm bảng AI Tutor gồm `knowledge_chunks`, `tutor_chat_sessions`, `tutor_chat_messages` và `tutor_chat_citations`. `knowledge_chunks` lưu các đoạn kiến thức được tách từ mô tả bài học hoặc tài liệu bài học. Khi học sinh đặt câu hỏi, hệ thống tạo phiên chat, lưu các tin nhắn và ghi lại các đoạn kiến thức được sử dụng làm căn cứ trả lời. Thiết kế này giúp câu trả lời của AI Tutor gắn với học liệu nội bộ thay vì trả lời hoàn toàn độc lập với nội dung khóa học.



Hình 4.14: Biểu đồ thực thể liên kết của cơ sở dữ liệu

b, Danh sách các bảng dữ liệu

Bảng 4.1 trình bày các bảng dữ liệu chính được sử dụng trong biểu đồ E-R. Các bảng kỹ thuật phụ như `user_sessions`, `webhook_events`, `idempotency_keys`, `audit_logs`, `notifications` và các bảng blog vẫn tồn tại trong cơ sở dữ liệu vật lý, nhưng không được đưa vào biểu đồ lỗi để tránh làm biểu đồ quá phức tạp.

Bảng 4.1: Danh sách các bảng dữ liệu chính

Tên bảng dữ liệu	Mô tả
<code>users</code>	Lưu thông tin tài khoản người dùng, vai trò và trạng thái tài khoản.
<code>media</code>	Lưu metadata của ảnh, video và tài liệu được tải lên hệ thống.
<code>courses</code>	Lưu thông tin khóa học, giáo viên phụ trách, giá bán và trạng thái xuất bản.
<code>chapters</code>	Lưu các chương thuộc một khóa học.
<code>lessons</code>	Lưu các bài học thuộc từng chương, bao gồm bài học video, tài liệu hoặc quiz.
<code>lesson_materials</code>	Lưu tài liệu bài học, nội dung trích xuất và trạng thái xử lý tài liệu.
<code>enrollments</code>	Lưu quyền truy cập khóa học của học sinh sau khi mua, được cấp thủ công hoặc khóa học miễn phí.
<code>lesson_progress</code>	Lưu tiến độ xem bài học của từng học sinh.
<code>orders</code>	Lưu đơn mua khóa học của học sinh.
<code>order_items</code>	Lưu các khóa học nằm trong một đơn hàng.
<code>payments</code>	Lưu trạng thái thanh toán, mã giao dịch và thông tin thanh toán của đơn hàng.
<code>topics</code>	Lưu chủ đề kiến thức dùng cho ngân hàng câu hỏi và thống kê kết quả học tập.
<code>questions</code>	Lưu câu hỏi trong ngân hàng câu hỏi.
<code>question_options</code>	Lưu các phương án trả lời của câu hỏi trắc nghiệm hoặc đúng/sai.
<code>assessments</code>	Lưu thông tin chung của bài kiểm tra.
<code>assessment_placements</code>	Lưu vị trí sử dụng bài kiểm tra, ví dụ công khai, trong khóa học hoặc trong bài học.
<code>assessment_sections</code>	Lưu các phần trong một bài kiểm tra.

Bảng 4.1: Danh sách các bảng dữ liệu chính - tiếp theo

Tên bảng dữ liệu	Mô tả
assessment_items	Lưu các câu hỏi hoặc mục chấm điểm trong bài kiểm tra.
submissions	Lưu lượt làm bài của học sinh.
knowledge_chunks	Lưu các đoạn kiến thức được tách từ học liệu để phục vụ AI Tutor.
tutor_chat_sessions	Lưu phiên hỏi đáp giữa học sinh và AI Tutor.
tutor_chat_messages	Lưu từng tin nhắn trong phiên hỏi đáp.
tutor_chat_citations	Lưu các đoạn kiến thức được trích dẫn trong câu trả lời của AI Tutor.

c, Chi tiết các bảng dữ liệu chính

Các bảng dưới đây trình bày chi tiết một số bảng dữ liệu chính của hệ thống. Với các trường cho phép giá trị rỗng, kiểu dữ liệu được ký hiệu bằng dấu hỏi, ví dụ `uuid?` hoặc `timestamp?`. Với kiểu dữ liệu dạng enum, cột kiểu dữ liệu ghi tên enum kèm (ENUM) và phần mô tả nêu rõ các giá trị có thể nhận.

Bảng `users` lưu trữ thông tin đăng nhập, họ tên, vai trò và trạng thái tài khoản. Cấu trúc chi tiết của bảng này được trình bày tại Bảng 4.2.

Bảng 4.2: Chi tiết bảng `users`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của người dùng.
2	<code>email</code>	<code>varchar(255)</code>	Email đăng nhập, không được trùng.
3	<code>password_hash</code>	<code>text</code>	Mật khẩu đã được băm.
4	<code>full_name</code>	<code>varchar(255)</code>	Họ tên hiển thị của người dùng.
5	<code>avatar_media_id</code>	<code>uuid?</code>	Ảnh đại diện, tham chiếu tới bảng <code>media</code> .
6	<code>role</code>	<code>user_role (ENUM)</code>	Vai trò người dùng: <code>admin</code> , <code>teacher</code> , <code>student</code> .

Bảng 4.2: Chi tiết bảng users - tiếp theo

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
7	status	user_status (ENUM)	Trạng thái tài khoản: pending_verification, active, banned.
8	created_at	timestamp	Thời điểm tạo tài khoản.
9	updated_at	timestamp	Thời điểm cập nhật tài khoản gần nhất.
10	deleted_at	timestamp?	Thời điểm xóa mềm tài khoản, nếu có.

Để quản lý ảnh, video và tài liệu, hệ thống sử dụng bảng media với cấu trúc tại Bảng 4.3. Bảng chỉ lưu metadata, trạng thái xử lý và liên kết tới người tải lên, không lưu trực tiếp nội dung tệp.

Bảng 4.3: Chi tiết bảng media

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	id	uuid	Khóa chính của tệp media.
2	type	media_type (ENUM)	Loại media: image, video, document.
3	status	media_status (ENUM)	Trạng thái xử lý: pending_upload, uploaded, processing, ready, failed, deleted.
4	visibility	media_visibility (ENUM)	Phạm vi truy cập: private, public.
5	original_name	varchar(255)?	Tên tệp gốc khi người dùng tải lên.
6	object_key	varchar(500)	Khóa đối tượng trong dịch vụ lưu trữ.
7	mime_type	varchar(100)	Kiểu MIME của tệp.
8	size_bytes	int	Kích thước tệp theo byte.
9	duration_sec	int?	Thời lượng video, nếu media là video.

Bảng 4.3: Chi tiết bảng `media` - tiếp theo

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
10	<code>uploaded_by_id</code>	<code>uuid?</code>	Người tải tệp lên hệ thống.

Thông tin cốt lõi của khóa học như tên, môn học, giáo viên, giá bán, ảnh đại diện và trạng thái xuất bản được tổ chức trong bảng `courses` (Bảng 4.4).

Bảng 4.4: Chi tiết bảng `courses`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của khóa học.
2	<code>title</code>	<code>varchar(255)</code>	Tên khóa học.
3	<code>slug</code>	<code>varchar(255)</code>	Định danh khóa học trên URL.
4	<code>description</code>	<code>text?</code>	Mô tả khóa học.
5	<code>subject</code>	<code>subject (ENUM)</code>	Môn học: <code>math</code> , <code>physics</code> , <code>chemistry</code> , <code>literature</code> , <code>english</code> , <code>biology</code> , <code>history</code> , <code>geography</code> .
6	<code>grade</code>	<code>int</code>	Khối lớp mục tiêu của khóa học.
7	<code>teacher_id</code>	<code>uuid</code>	Giáo viên phụ trách khóa học.
8	<code>thumbnail_media_id</code>	<code>uuid?</code>	Ảnh đại diện khóa học.
9	<code>price</code>	<code>int</code>	Giá gốc của khóa học.
10	<code>sale_price</code>	<code>int?</code>	Giá khuyến mại, nếu có.
11	<code>status</code>	<code>course_status (ENUM)</code>	Trạng thái khóa học: <code>draft</code> , <code>published</code> , <code>archived</code> .
12	<code>total_lessons</code>	<code>int</code>	Tổng số bài học trong khóa học.

Các chương thuộc khóa học được sắp xếp trong bảng `chapters`. Những trường dùng để xác định khóa học, tiêu đề và thứ tự chương được liệt kê tại Bảng 4.5.

Bảng 4.5: Chi tiết bảng chapters

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	id	uuid	Khóa chính của chương.
2	course_id	uuid	Khóa học chứa chương.
3	title	varchar(255)	Tên chương.
4	order_index	int	Thứ tự hiển thị của chương trong khóa học.
5	deleted_at	timestamp?	Thời điểm xóa mềm chương, nếu có.

Mỗi bài học trong chương được biểu diễn bởi một bản ghi lessons, gồm loại bài, mô tả, video, thời lượng và quyền xem thử. Bảng 4.6 trình bày các trường tương ứng.

Bảng 4.6: Chi tiết bảng lessons

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	id	uuid	Khóa chính của bài học.
2	chapter_id	uuid	Chương chứa bài học.
3	title	varchar(255)	Tên bài học.
4	type	lesson_type (ENUM)	Loại bài học: video, quiz, document.
5	description	text?	Mô tả bài học.
6	video_type	video_type (ENUM) ?	Loại video: system hoặc youtube.
7	video_media_id	uuid?	Video bài giảng lưu trong hệ thống.
8	youtube_url	text?	Đường dẫn YouTube nếu bài học dùng video ngoài.
9	duration_sec	int?	Thời lượng bài học theo giây.
10	allow_preview	boolean	Cho phép xem thử bài học hay không.
11	order_index	int	Thứ tự hiển thị của bài học trong chương.

Học liệu tải lên, văn bản nhập trực tiếp và nội dung trích xuất từ tệp được lưu trong bảng lesson_materials theo cấu trúc ở Bảng 4.7. Đây cũng là nguồn dữ liệu cho quá trình lập chỉ mục của AI Tutor.

Bảng 4.7: Chi tiết bảng `lesson_materials`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của học liệu.
2	<code>course_id</code>	<code>uuid</code>	Khóa học chứa học liệu.
3	<code>lesson_id</code>	<code>uuid</code>	Bài học chứa học liệu.
4	<code>media_id</code>	<code>uuid?</code>	Tệp tài liệu đính kèm.
5	<code>created_by_id</code>	<code>uuid?</code>	Người tạo học liệu.
6	<code>title</code>	<code>varchar(255)</code>	Tên học liệu.
7	<code>type</code>	<code>lesson_</code> <code>material_type</code> (ENUM)	Loại học liệu: <code>text</code> , <code>markdown</code> , <code>pdf</code> , <code>docx</code> , <code>pptx</code> .
8	<code>content_text</code>	<code>text?</code>	Nội dung nhập trực tiếp nếu học liệu là văn bản.
9	<code>extracted_</code> <code>text</code>	<code>text?</code>	Nội dung được trích xuất từ tệp tài liệu.
10	<code>processing_</code> <code>status</code>	<code>lesson_</code> <code>material_</code> <code>processing_</code> <code>status</code> (ENUM)	Trạng thái xử lý: <code>pend-</code> <code>ing</code> , <code>processing</code> , <code>ready</code> , <code>failed</code> .
11	<code>is_public</code>	<code>boolean</code>	Xác định học liệu có được hiển thị cho học sinh hay không.

Quyền truy cập khóa học của học sinh được lưu trong bảng `enrollments`. Cấu trúc tại Bảng 4.8 cho phép phân biệt ghi danh qua thanh toán, cấp thử công hoặc khóa học miễn phí.

Bảng 4.8: Chi tiết bảng `enrollments`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của bản ghi ghi danh.
2	<code>user_id</code>	<code>uuid</code>	Học sinh được cấp quyền học.
3	<code>course_id</code>	<code>uuid</code>	Khóa học được cấp quyền truy cập.
4	<code>order_id</code>	<code>uuid?</code>	Đơn hàng tạo ra quyền truy cập, nếu có.
5	<code>source</code>	<code>enrollment_</code> <code>source</code> (ENUM)	Nguồn ghi danh: <code>pay-</code> <code>ment</code> , <code>manual</code> , <code>free</code> .

Bảng 4.8: Chi tiết bảng `enrollments` - tiếp theo

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
6	<code>manual_reason</code>	<code>varchar(500) ?</code>	Lý do cấp quyền thủ công.
7	<code>enrolled_at</code>	<code>timestamp</code>	Thời điểm học sinh được ghi danh vào khóa học.

Đơn mua khóa học được quản lý bằng bảng `orders`; các trường về mã đơn, tổng tiền, trạng thái và hạn thanh toán được trình bày tại Bảng 4.9.

Bảng 4.9: Chi tiết bảng `orders`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của đơn hàng.
2	<code>user_id</code>	<code>uuid</code>	Học sinh tạo đơn hàng.
3	<code>order_invoice_number</code>	<code>varchar(50)</code>	Mã đơn hàng dùng để đối soát thanh toán.
4	<code>total_amount</code>	<code>int</code>	Tổng số tiền của đơn hàng.
5	<code>currency</code>	<code>varchar(3)</code>	Đơn vị tiền tệ, mặc định là VND.
6	<code>status</code>	<code>order_status (ENUM)</code>	Trạng thái đơn hàng: <code>pending</code> , <code>completed</code> , <code>cancelled</code> , <code>expired</code> .
7	<code>expires_at</code>	<code>timestamp</code>	Thời điểm hết hạn thanh toán đơn hàng.

Thông tin thanh toán của đơn hàng được tách sang bảng `payments`, bao gồm nhà cung cấp, số tiền, mã giao dịch, đường dẫn QR và trạng thái. Chi tiết các trường được nêu tại Bảng 4.10.

Bảng 4.10: Chi tiết bảng `payments`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của bản ghi thanh toán.
2	<code>order_id</code>	<code>uuid</code>	Đơn hàng tương ứng.
3	<code>provider</code>	<code>varchar(50)</code>	Nhà cung cấp thanh toán.
4	<code>amount</code>	<code>int</code>	Số tiền cần thanh toán.
5	<code>transaction_ref</code>	<code>varchar(255) ?</code>	Mã tham chiếu giao dịch.

Bảng 4.10: Chi tiết bảng `payments` - tiếp theo

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
6	<code>qr_code_url</code>	<code>varchar(1000)?</code>	Đường dẫn mã QR thanh toán.
7	<code>status</code>	<code>payment_status</code> (ENUM)	Trạng thái thanh toán: <code>pending</code> , <code>success</code> , <code>failed</code> , <code>cancelled</code> , <code>late_success</code> , <code>manual_review</code> .
8	<code>paid_at</code>	<code>timestamp?</code>	Thời điểm thanh toán thành công.

Các thuộc tính chung của bài kiểm tra được lưu trong bảng `assessments`, từ môn học, khối lớp đến cách chấm, thời gian làm bài và trạng thái hiển thị (Bảng 4.11).

Bảng 4.11: Chi tiết bảng `assessments`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của bài kiểm tra.
2	<code>subject</code>	<code>subject</code> (ENUM)	Môn học của bài kiểm tra.
3	<code>grade</code>	<code>int</code>	Khối lớp mục tiêu.
4	<code>type</code>	<code>assessment_type</code> (ENUM)	Loại bài kiểm tra: <code>quiz</code> , <code>exam</code> .
5	<code>title</code>	<code>text</code>	Tên bài kiểm tra.
6	<code>grading_type</code>	<code>grading_type</code> (ENUM)	Hình thức chấm điểm: <code>auto</code> , <code>manual</code> , <code>mixed</code> .
7	<code>time_limit_minutes</code>	<code>int?</code>	Thời gian làm bài nếu có giới hạn.
8	<code>visibility</code>	<code>assessment_visibility</code> (ENUM)	Trạng thái hiển thị: <code>draft</code> , <code>published</code> , <code>hidden</code> .
9	<code>source_media_id</code>	<code>uuid?</code>	Tệp đề gốc nếu bài kiểm tra được import từ media.
10	<code>created_by_id</code>	<code>uuid?</code>	Người tạo bài kiểm tra.

Phạm vi phát hành bài kiểm tra được xác định bởi bảng `assessment_placements`. Bảng 4.12 thể hiện ba trường hợp sử dụng: công khai, trong khóa học hoặc tại một bài học cụ thể.

Bảng 4.12: Chi tiết bảng `assessment_placements`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của vị trí sử dụng bài kiểm tra.
2	<code>assessment_id</code>	<code>uuid</code>	Bài kiểm tra được đặt vào ngữ cảnh sử dụng.
3	<code>type</code>	<code>assessment_placement_type</code> (ENUM)	Ngữ cảnh sử dụng: <code>public_practice</code> , <code>course</code> , <code>lesson</code> .
4	<code>course_id</code>	<code>uuid?</code>	Khóa học chứa bài kiểm tra, nếu có.
5	<code>lesson_id</code>	<code>uuid?</code>	Bài học chứa bài kiểm tra, nếu có.
6	<code>open_time</code>	<code>timestamp?</code>	Thời điểm mở bài kiểm tra.
7	<code>close_time</code>	<code>timestamp?</code>	Thời điểm đóng bài kiểm tra.
8	<code>max_attempts</code>	<code>int?</code>	Số lần làm bài tối đa.
9	<code>slug</code>	<code>varchar(255)?</code>	Đường dẫn công khai nếu bài kiểm tra được phát hành dạng public.

Các câu hỏi của bài kiểm tra được sắp xếp trong bảng `assessment_items`. Loại câu hỏi, thứ tự, điểm tối đa và liên kết tới ngân hàng câu hỏi được mô tả tại Bảng 4.13.

Bảng 4.13: Chi tiết bảng `assessment_items`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của mục trong bài kiểm tra.
2	<code>assessment_id</code>	<code>uuid</code>	Bài kiểm tra chứa mục này.
3	<code>section_id</code>	<code>uuid</code>	Phần chứa mục này.
4	<code>item_type</code>	<code>assessment_item_type</code> (ENUM)	Loại mục: <code>mcq</code> , <code>true_false</code> , <code>numeric</code> , <code>essay</code> .
5	<code>question_id</code>	<code>uuid?</code>	Câu hỏi trong ngân hàng câu hỏi nếu mục được liên kết với câu hỏi có sẵn.

Bảng 4.13: Chi tiết bảng `assessment_items` - tiếp theo

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
6	<code>topic_id</code>	<code>uuid?</code>	Chủ đề kiến thức liên quan.
7	<code>order_index</code>	<code>int</code>	Thứ tự của mục trong phần.
8	<code>max_score</code>	<code>decimal(8,2)</code>	Điểm tối đa của mục.

Mỗi lượt làm bài của học sinh tạo một bản ghi `submissions`. Bảng 4.14 trình bày các trường về bài kiểm tra, số lần làm, trạng thái, thời điểm và điểm số.

Bảng 4.14: Chi tiết bảng `submissions`

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	<code>id</code>	<code>uuid</code>	Khóa chính của lượt làm bài.
2	<code>assessment_id</code>	<code>uuid</code>	Bài kiểm tra được thực hiện.
3	<code>placement_id</code>	<code>uuid?</code>	Vị trí sử dụng bài kiểm tra.
4	<code>student_id</code>	<code>uuid</code>	Học sinh làm bài.
5	<code>attempt_number</code>	<code>int</code>	Số thứ tự lần làm bài.
6	<code>status</code>	<code>submission_status (ENUM)</code>	Trạng thái lượt làm: <code>doing</code> , <code>submitted</code> , <code>auto_submitted</code> , <code>completed</code> .
7	<code>auto_score</code>	<code>decimal(8,2)?</code>	Điểm tự động của các câu có thể chấm tự động.
8	<code>final_score</code>	<code>decimal(8,2)?</code>	Điểm cuối cùng sau khi chấm.
9	<code>start_time</code>	<code>timestamp</code>	Thời điểm bắt đầu làm bài.
10	<code>submit_time</code>	<code>timestamp?</code>	Thời điểm nộp bài.

Các đoạn kiến thức tách từ mô tả bài học và học liệu được lưu trong bảng `knowledge_chunks` để phục vụ truy xuất ngữ cảnh. Cấu trúc dữ liệu vector và thông tin nguồn được trình bày tại Bảng 4.15.

Bảng 4.15: Chi tiết bảng knowledge_chunks

STT	Trường dữ liệu	Kiểu dữ liệu	Mô tả
1	id	uuid	Khóa chính của đoạn kiến thức.
2	course_id	uuid	Khóa học chứa đoạn kiến thức.
3	lesson_id	uuid?	Bài học chứa đoạn kiến thức.
4	material_id	uuid?	Học liệu sinh ra đoạn kiến thức.
5	source_type	chunk_source_type (ENUM)	Nguồn sinh đoạn kiến thức: lesson_description, material.
6	chunk_index	int	Thứ tự của đoạn kiến thức trong nguồn.
7	content	text	Nội dung đoạn kiến thức.
8	content_hash	varchar(128)	Mã băm nội dung dùng để phát hiện trùng lặp hoặc thay đổi.
9	indexed_at	timestamp?	Thời điểm đoạn kiến thức được lập chỉ mục.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Các công cụ, thư viện và dịch vụ được lựa chọn theo từng nhóm công việc của hệ thống, từ phát triển ứng dụng web, quản lý dữ liệu, xử lý media đến xây dựng AI Tutor. Bảng 4.16 tổng hợp mục đích sử dụng và địa chỉ tài liệu của các thành phần chính.

Bảng 4.16: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ / Thư viện / API	Địa chỉ URL
IDE lập trình	Visual Studio Code	https://code.visualstudio.com/
Môi trường chạy JavaScript	Node.js	https://nodejs.org/
Quản lý gói JavaScript	npm	https://www.npmjs.com/

Bảng 4.16: Danh sách thư viện và công cụ sử dụng - tiếp theo

Mục đích	Công cụ / Thư viện / API	Địa chỉ URL
Ngôn ngữ lập trình web	TypeScript	https://www.typescriptlang.org/
Xây dựng REST API	Express	https://expressjs.com/
Cơ sở dữ liệu quan hệ	PostgreSQL	https://www.postgresql.org/
Tìm kiếm vector trong PostgreSQL	pgvector	https://github.com/pgvector/pgvector
ORM và migration CSDL	Prisma, @prisma/client	https://www.prisma.io/
Kiểm tra dữ liệu đầu vào	Zod	https://zod.dev/
Xác thực người dùng	jsonwebtoken, bcrypt	https://www.npmjs.com/package/jsonwebtoken
Tính toán điểm và tiền tệ	decimal.js	https://mikemcl.github.io/decimal.js/
Tài liệu hóa API	swagger-ui-express, yaml	https://swagger.io/specification/
Lưu trữ media	Cloudflare R2	https://developers.cloudflare.com/r2/
Kết nối kho lưu trữ tương thích S3	AWS SDK for JavaScript	https://docs.aws.amazon.com/AWSJavaScriptSDK/v3/latest/
Xử lý video bài giảng	FFmpeg, FFprobe	https://ffmpeg.org/
Thanh toán khóa học	SePay Webhook/API	https://sepay.vn/
Xây dựng ứng dụng web	Next.js	https://nextjs.org/
Xây dựng giao diện	React, React DOM	https://react.dev/
Thiết kế giao diện bằng lớp tiện ích	Tailwind CSS	https://tailwindcss.com/
Quản lý dữ liệu từ máy chủ	TanStack Query	https://tanstack.com/query/latest
Gọi HTTP API	Axios	https://axios-http.com/
Quản lý trạng thái giao diện	Zustand	https://zustand-demo.pmnd.rs/

Bảng 4.16: Danh sách thư viện và công cụ sử dụng - tiếp theo

Mục đích	Công cụ / Thư viện / API	Địa chỉ URL
Quản lý biểu mẫu	React Hook Form	https://react-hook-form.com/
Soạn thảo nội dung giàu định dạng	Tiptap	https://tiptap.dev/
Kéo thả cấu trúc khóa học	@hello-pangea/dnd	https://github.com/hello-pangea/dnd
Phát video HLS trên trình duyệt	Video.js, hls.js	https://videojs.com/
Biểu tượng và thông báo giao diện	Lucide, Sonner	https://lucide.dev/
Ngôn ngữ của dịch vụ AI	Python	https://www.python.org/
Xây dựng API cho dịch vụ AI	FastAPI, Uvicorn	https://fastapi.tiangolo.com/
Kiểm tra dữ liệu dịch vụ AI	Pydantic	https://docs.pydantic.dev/
Kết nối PostgreSQL từ Python	psycopg2-binary	https://www.psycopg.org/
Chia nhỏ văn bản học liệu	langchain-text-splitters	https://python.langchain.com/
Trích xuất nội dung tài liệu	pypdf, python-docx, python-pptx	https://pypdf.readthedocs.io/
Kết nối mô hình ngôn ngữ	OpenAI Python SDK	https://github.com/openai/openai-python
Kiểm tra chất lượng mã nguồn	ESLint, Prettier	https://eslint.org/
Chạy máy chủ khi phát triển	Nodemon, TSX	https://github.com/remy/nodemon

4.3.2 Kết quả đạt được

Sau quá trình thiết kế và phát triển, đồ án đã xây dựng được ba khối phần mềm chính gồm ứng dụng phía máy chủ, ứng dụng giao diện và dịch vụ AI. Ứng dụng phía máy chủ xử lý các nghiệp vụ cốt lõi như xác thực, quản lý khóa học, học tập, bài kiểm tra, đơn hàng, thanh toán, media, blog và tutor. Ứng dụng giao diện cung cấp các màn hình dành cho khách truy cập, học sinh, giáo viên và quản trị viên. Dịch vụ AI đảm nhận việc xử lý tài liệu học tập, tạo embedding, truy xuất ngữ cảnh và trả lời câu hỏi theo nội dung bài học.

Về mặt dữ liệu, hệ thống đã xây dựng lược đồ PostgreSQL gồm 42 model và

33 enum trong Prisma schema. Cấu trúc dữ liệu này bao phủ các nhóm bảng chính như người dùng, phiên đăng nhập, khóa học, chương, bài học, học liệu, tiến độ học tập, bài kiểm tra, bài làm, đơn hàng, giao dịch thanh toán, media, blog, thông báo và dữ liệu phục vụ RAG. Cách tổ chức này giúp hệ thống không chỉ lưu trữ nội dung khóa học mà còn ghi nhận được các trạng thái vận hành quan trọng như thanh toán, quyền truy cập, tiến độ học tập và lịch sử hỏi đáp.

Về mặt mã nguồn, backend được tổ chức thành mười module chính gồm auth, users, courses, assessments, enrollments, media, orders, payments, tutor và blogs. Frontend được chia theo các nhóm chức năng như admin, assessments, auth, blog, courses, landing, media, orders, payments và users. Python AI Service có các tệp xử lý chính cho API, kết nối CSDL, nạp tài liệu, đánh chỉ mục tri thức, truy xuất RAG và đánh giá chất lượng truy xuất.

Các số liệu thống kê trong Bảng 4.17 được lấy từ mã nguồn của dự án, không tính thư mục thư viện ngoài như `node_modules`, thư mục build hoặc môi trường ảo Python. Các số liệu này phản ánh quy mô mã nguồn của ba thành phần trong hệ thống.

Bảng 4.17: Thống kê quy mô mã nguồn của hệ thống

Thông tin	Frontend	Backend	AI Service
Số tệp mã nguồn	246	290	10
Số dòng mã nguồn	35.066	26.858	1.031
Số nhóm chức năng chính	11	11	6
Dung lượng mã nguồn	1,30 MB	0,79 MB	0,036 MB

Các số liệu được đếm tại thời điểm hoàn thiện báo cáo trên các tệp mã nguồn thuộc thư mục `src` (và các tệp Python của AI Service), không gồm thư viện cài đặt, tệp build và tài nguyên nhị phân. Frontend có số dòng lớn nhất do gồm nhiều màn hình, form và trạng thái hiển thị; backend có nhiều tệp hơn do tách module, service, repository, DTO, validator và kiểm thử. Dịch vụ AI nhỏ hơn vì tập trung vào ingest tài liệu, truy xuất ngữ cảnh, sinh câu trả lời và đánh giá RAG.

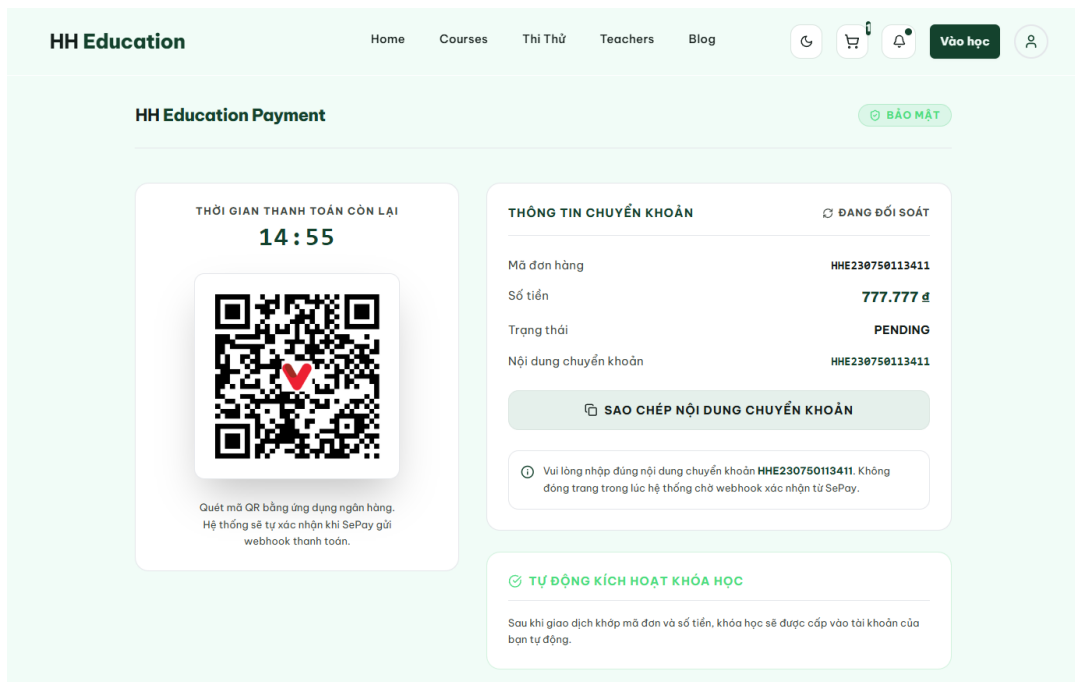
4.3.3 Minh họa các chức năng chính

Phần này minh họa một số màn hình thể hiện rõ các luồng nghiệp vụ quan trọng của hệ thống. Các ảnh được chụp trên bộ dữ liệu thử nghiệm, trong đó thông tin đơn hàng, giao dịch và học liệu được liên kết xuyên suốt để làm rõ cách hệ thống vận hành từ phía học sinh đến phía quản trị.

a, Màn hình thanh toán khóa học

Màn hình thanh toán bằng chuyển khoản được trình bày trong Hình 4.15. Khi học sinh mở trang, frontend lấy thông tin theo mã đơn hàng; nếu đơn đang chờ và chưa có lần thanh toán, hệ thống tạo lần thanh toán với một idempotency key được lưu theo phiên trình duyệt. Cách làm này hạn chế bản ghi trùng khi người dùng tải lại trang hoặc mở lại đường dẫn.

Mã QR được tạo theo số tiền và nội dung chuyển khoản là mã hóa đơn, vì vậy mỗi đơn hàng có thông tin thanh toán riêng. Giao diện hiển thị đồng hồ đếm ngược, trạng thái đối soát và cho phép sao chép nội dung chuyển khoản để giảm sai sót khi nhập mã. Trang thanh toán tự tải lại thông tin đơn hàng theo chu kỳ; khi webhook từ SePay được đối chiếu thành công và backend hoàn tất đơn hàng kèm ghi danh khóa học, trạng thái đơn hàng chuyển sang `completed` và giao diện điều hướng học sinh sang trang thanh toán thành công.



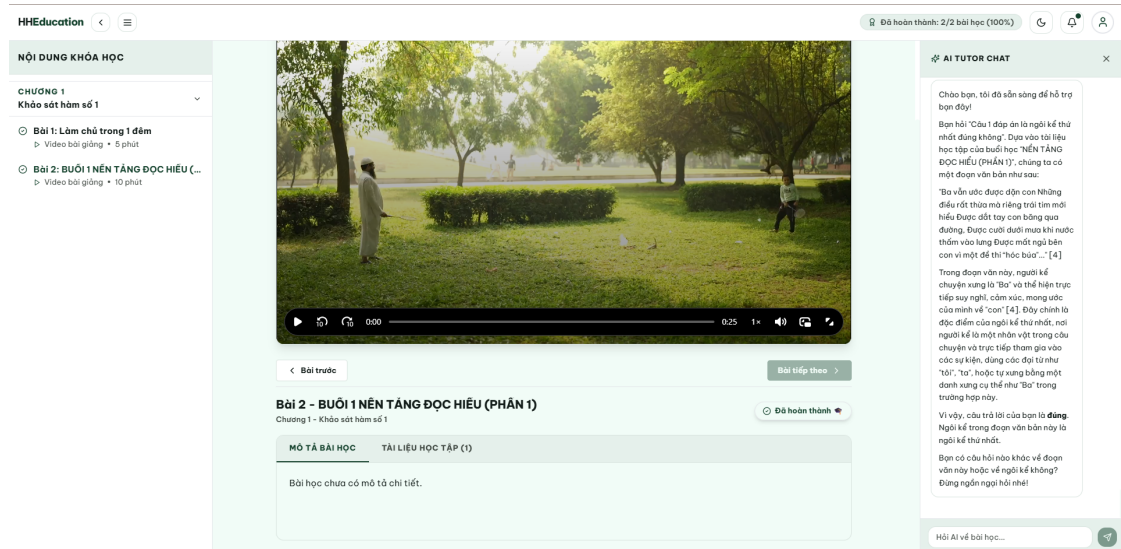
Hình 4.15: Màn hình thanh toán khóa học bằng mã QR

b, Không gian học tập và AI Tutor

Không gian học tập trong Hình 4.16 gồm danh sách bài học ở bên trái, video và học liệu ở giữa, cùng AI Tutor ở bên phải. Khi học sinh mở bài, backend kiểm tra quyền ghi danh trước khi trả về nội dung. Với video HLS, trình phát chỉ yêu cầu playlist và các segment cần thiết; mỗi yêu cầu đều được kiểm tra quyền trước khi backend lấy tệp từ Cloudflare R2.

Giao diện cũng ghi nhận tiến độ khi học sinh xem, tạm dừng hoặc tua video; dữ liệu này được lưu thành tiến độ bài học và tổng hợp lại ở mức khóa học. AI Tutor

nhận câu hỏi trong đúng ngữ cảnh bài học đang mở. Sau khi kiểm tra phiên chat và quyền học, hệ thống chỉ truy xuất các chunk thuộc bài học đó, trả lời kèm nguồn trích dẫn và lưu lại lịch sử trao đổi. Vì vậy, khung chat không phải chatbot độc lập mà gắn trực tiếp với bài học học sinh đang theo dõi.



Hình 4.16: Không gian học tập tích hợp video, học liệu và AI Tutor

c, Trình xây dựng cấu trúc khóa học

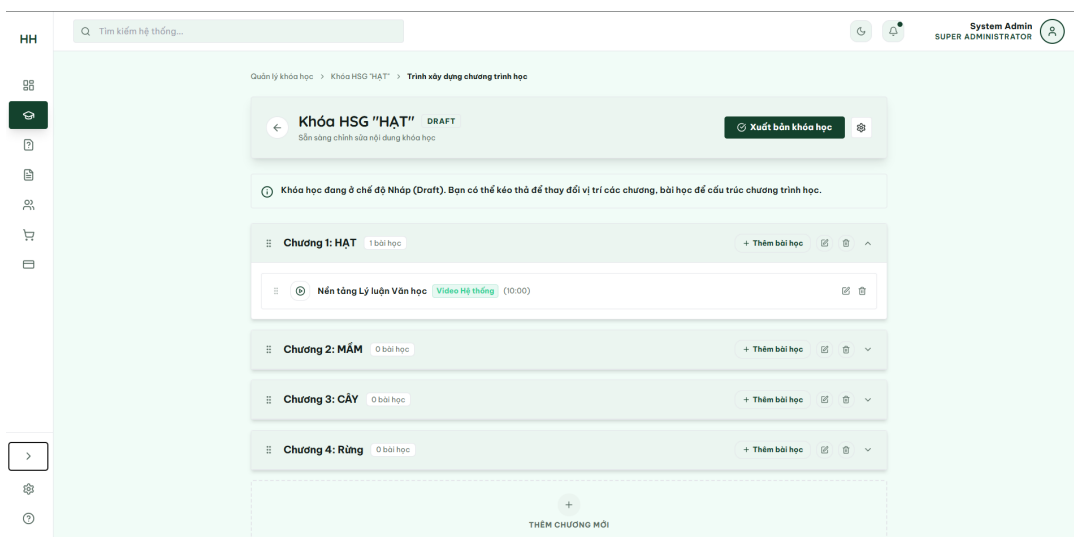
Màn hình xây dựng chương trình học trong Hình 4.17 tổ chức khóa học theo chương và bài học. Mỗi chương có thể thu gọn, thêm bài, sửa hoặc xóa; từng bài học hiển thị loại nội dung và thời lượng. Trạng thái nhấp cho biết nội dung chưa được công khai cho học sinh.

Việc thay đổi thứ tự được thực hiện bằng thao tác kéo thả của thư viện @hello-pangea/dnd. Sau khi người dùng thả một chương hoặc bài học vào vị trí mới, frontend tính lại `orderIndex`, cập nhật ngay danh sách đang hiển thị và gửi danh sách mã chương hoặc bài học theo thứ tự mới đến backend để lưu. Hệ thống chỉ cho phép đổi thứ tự bài học trong cùng một chương; nếu kéo sang chương khác, giao diện thông báo giới hạn này thay vì tự thay đổi cấu trúc ngoài ý muốn. Khi có yêu cầu đang lưu, thao tác kéo thả được tạm khóa để tránh xung đột thứ tự.

d, Trình xây dựng bài kiểm tra

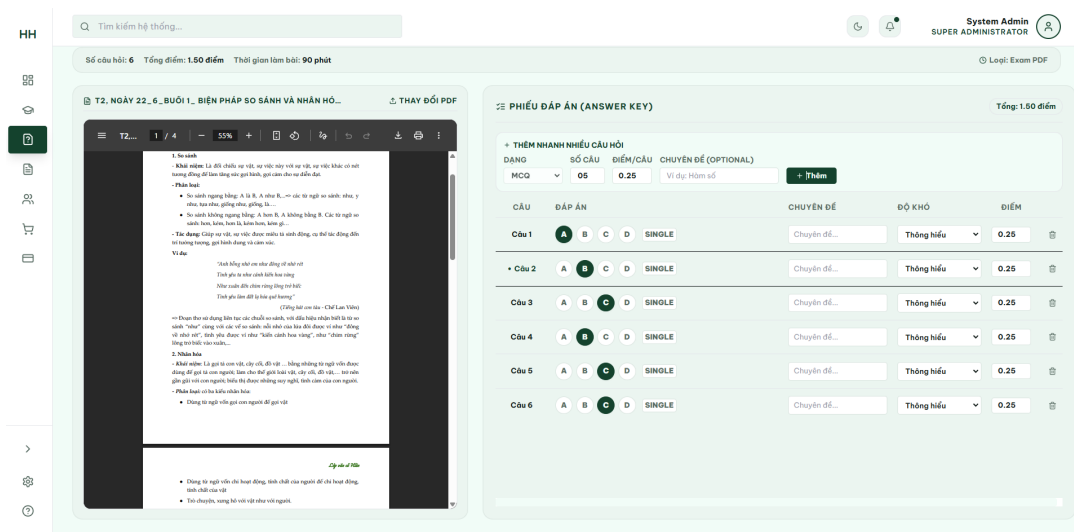
Màn hình xây dựng bài kiểm tra dạng đề PDF được trình bày trong Hình 4.18. Tài liệu nguồn nằm bên trái để đối chiếu; phiếu đáp án bên phải hỗ trợ thêm câu hỏi, chọn đáp án đúng, gán chuyên đề, độ khó và điểm. Người quản lý có thể nhập đáp án mà không phải chuyển qua lại giữa nhiều màn hình.

Khi tạo bài kiểm tra, hệ thống lưu đề nhấp, các phần đề, câu hỏi và vị trí sử dụng riêng. Người quản lý có thể thiết lập bài kiểm tra công khai, gán với khóa học hoặc



Hình 4.17: Màn hình xây dựng cấu trúc khóa học

gắn với một bài học dạng quiz, đồng thời đặt thời gian mở - đóng và số lượt làm. Trước khi xuất bản, backend kiểm tra cấu trúc đề và cấu hình sử dụng. Sau khi đã có lượt nộp, nội dung câu hỏi được khóa để không làm thay đổi đáp án hoặc điểm của các bài làm đã tồn tại.



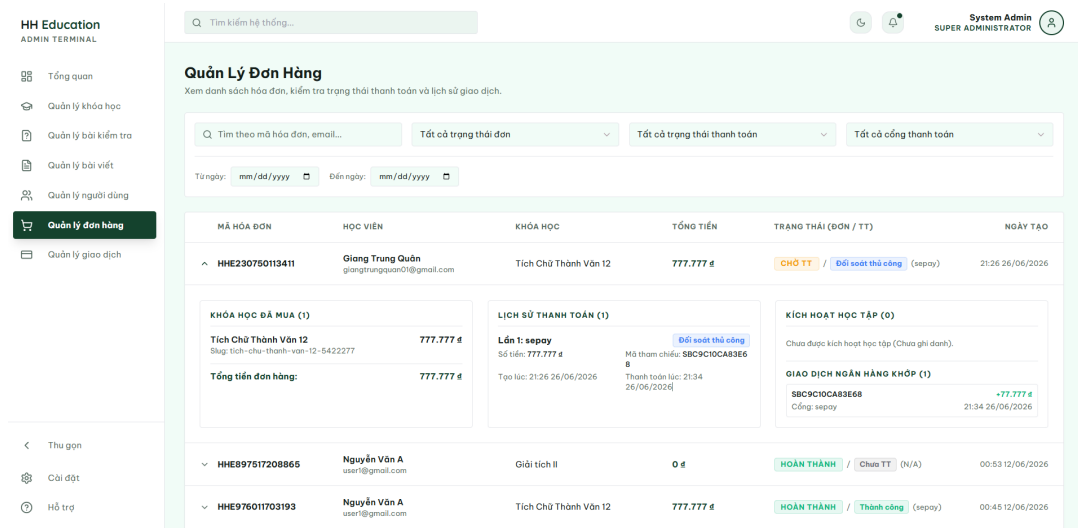
Hình 4.18: Màn hình xây dựng bài kiểm tra từ đề PDF

e, Quản lý đơn hàng và quyền học

Màn hình quản lý đơn hàng trong Hình 4.19 hỗ trợ tìm theo mã hóa đơn hoặc email và lọc theo trạng thái, cổng thanh toán hoặc thời gian tạo. Phần chi tiết hiển thị khóa học đã mua, lịch sử thanh toán, giao dịch ngân hàng đã khớp và các bản ghi ghi danh.

Thiết kế này giúp việc cấp quyền học có thể được kiểm tra từ cùng một màn hình. Với giao dịch hợp lệ, backend cập nhật trạng thái thanh toán, hoàn tất đơn

hàng và tạo ghi danh trong một transaction. Nếu giao dịch ở trạng thái cần đối soát thủ công, hệ thống vẫn lưu đầy đủ lịch sử nhưng không tự cấp quyền học. Nhờ đó, người quản trị có thể phân biệt rõ đơn đã hoàn thành, đơn đang chờ thanh toán và các trường hợp cần kiểm tra thêm.



Hình 4.19: Màn hình quản lý đơn hàng, thanh toán và quyền học

f, Quản lý giao dịch và đối soát webhook

Màn hình kiểm tra giao dịch webhook trong Hình 4.20 cho phép lọc theo mã hóa đơn, mã tham chiếu ngân hàng và khoảng thời gian. Chi tiết giao dịch gồm payload gốc từ nhà cung cấp, dữ liệu đã chuẩn hóa, trạng thái đối soát, đơn hàng liên kết và lần thanh toán tương ứng.

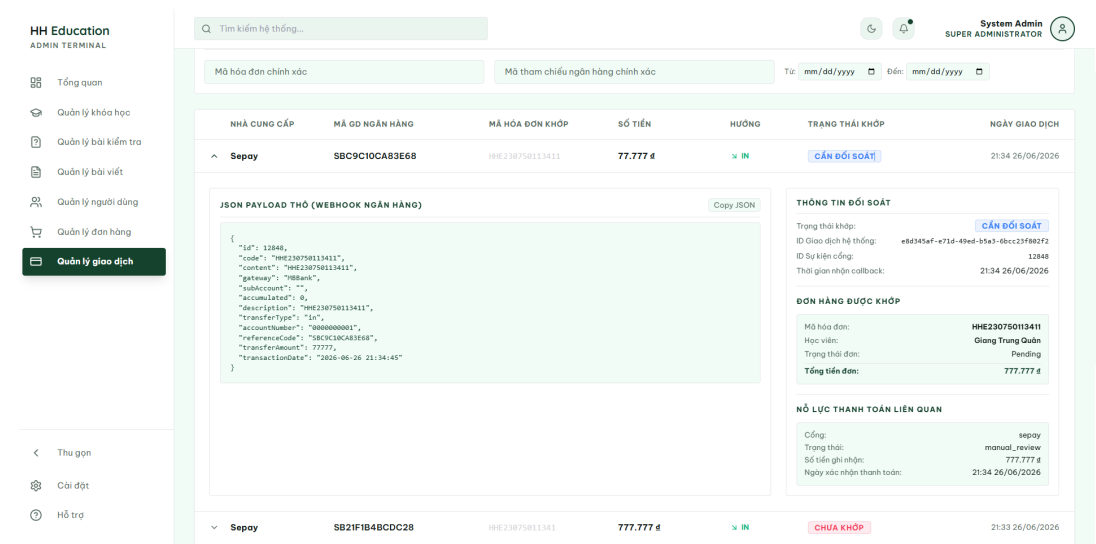
Các trạng thái `matched`, `unmatched`, `manual_review` và `ignored` phân biệt các trường hợp thiếu mã hóa đơn, sai số tiền, đơn hàng hết hạn hoặc giao dịch chi tiền. Việc lưu cả payload và kết quả đối soát cung cấp căn cứ kiểm tra quyết định cấp quyền; mã sự kiện từ nhà cung cấp giúp backend phát hiện webhook gửi lặp và tránh tạo ghi danh trùng.

4.4 Kiểm thử

Phần này kết hợp kiểm thử hộp đen các luồng người dùng với kiểm tra tự động ở mức mã nguồn. Các ca kiểm thử chức năng bao phủ xác thực, mua và cấp quyền khóa học, học bài, AI Tutor, theo dõi học viên và bài kiểm tra. Mỗi ca nêu thao tác, kết quả mong đợi và kết quả thực tế quan sát trong môi trường phát triển; các kiểm tra tự động được dùng để xác nhận thêm các quy tắc khó quan sát chỉ qua giao diện.

4.4.1 Kiểm thử chức năng đăng nhập

Chức năng đăng nhập là điểm vào của các tác nhân đã có tài khoản trong hệ thống. Bảng 4.18 trình bày các test case cho chức năng đăng nhập, bao gồm luồng



Hình 4.20: Màn hình quản lý giao dịch và đối soát webhook thanh toán

đăng nhập thành công, dữ liệu không hợp lệ và kiểm soát truy cập với người dùng chưa xác thực.

Bảng 4.18: Test case cho chức năng đăng nhập

Mã	Mục tiêu kiểm thử	Input/Thao tác	Kết quả mong đợi	Kết quả
TC001	Đăng nhập thành công với tài khoản hợp lệ	Nhập đúng email và mật khẩu của học sinh, sau đó nhấn đăng nhập.	Hệ thống xác thực thành công, lưu phiên đăng nhập và chuyển người dùng vào khu vực học tập.	Đạt
TC002	Kiểm tra đăng nhập sai mật khẩu	Nhập email hợp lệ nhưng mật khẩu không đúng.	Hệ thống từ chối đăng nhập và hiển thị thông báo lỗi phù hợp, không tạo phiên đăng nhập.	Đạt
TC003	Kiểm tra định dạng email	Nhập email sai định dạng và mật khẩu bất kỳ.	Form báo lỗi dữ liệu đầu vào, request không được gửi hoặc bị backend từ chối theo schema kiểm tra dữ liệu.	Đạt

Bảng 4.18 - Tiếp trang trước

Mã	Mục tiêu kiểm thử	Input/Thao tác	Kết quả mong đợi	Kết quả
TC004	Kiểm tra truy cập trang cần đăng nhập	Truy cập trực tiếp trang học tập khi chưa có phiên đăng nhập hợp lệ.	Hệ thống không cho truy cập nội dung riêng tư và chuyển người dùng về trang đăng nhập.	Đạt

4.4.2 Kiểm thử chức năng mua khóa học

Chức năng mua khóa học kết hợp nhiều bước gồm chọn khóa học, tạo đơn hàng, hiển thị mã VietQR, tiếp nhận webhook thanh toán và tạo bản ghi ghi danh. Bảng 4.19 trình bày các test case cho chức năng này.

Bảng 4.19: Test case cho chức năng mua khóa học

Mã	Mục tiêu kiểm thử	Input/Thao tác	Kết quả mong đợi	Kết quả
TC005	Tạo đơn hàng mua khóa học	Học sinh chọn một khóa học chưa ghi danh và xác nhận mua.	Hệ thống tạo đơn hàng trạng thái pending, sinh mã hóa đơn và tạo thông tin thanh toán.	Đạt
TC006	Hiển thị mã thanh toán VietQR	Mở màn hình thanh toán của đơn hàng đang chờ thanh toán.	Hệ thống hiển thị đúng số tiền, mã đơn hàng và mã QR tương ứng với thông tin thanh toán.	Đạt
TC007	Xử lý webhook thanh toán hợp lệ	Gửi webhook giao dịch tiền vào có mã đơn hàng, số tiền và đơn vị tiền tệ khớp với đơn hàng.	Hệ thống cập nhật đơn hàng sang completed, đánh dấu giao dịch matched và tạo bản ghi enrollments.	Đạt
TC008	Xử lý webhook thiếu mã đơn hàng	Gửi webhook giao dịch tiền vào nhưng không có mã đơn hàng trong nội dung giao dịch.	Hệ thống không cấp quyền học tự động, lưu giao dịch ở trạng thái cần đối soát thủ công.	Đạt

Bảng 4.19 - Tiếp trang trước

Mã	Mục tiêu kiểm thử	Input/Thao tác	Kết quả mong đợi	Kết quả
TC009	Ngăn mua lại khóa học đã ghi danh	Học sinh đã có quyền học tiếp tục tạo đơn hàng cho cùng khóa học.	Hệ thống từ chối thao tác mua lại hoặc không tạo thêm quyền học trùng lặp.	Đạt

4.4.3 Kiểm thử chức năng học bài và hỏi AI

Chức năng học bài và hỏi AI là luồng trải nghiệm chính của học sinh sau khi đã mua khóa học. Bảng 4.20 trình bày các test case cho chức năng này, tập trung vào quyền truy cập bài học, phát video HLS, xem tài liệu và hỏi đáp theo học liệu.

Bảng 4.20: Test case cho chức năng học bài và hỏi AI

Mã	Mục tiêu kiểm thử	Input/Thao tác	Kết quả mong đợi	Kết quả
TC010	Truy cập bài học đã có quyền học	Học sinh đã ghi danh mở một bài học trong khóa học.	Hệ thống hiển thị màn hình học tập gồm danh sách bài học, video hoặc nội dung bài học, mô tả và tài liệu.	Đạt
TC011	Chặn truy cập bài học chưa có quyền	Học sinh chưa ghi danh truy cập trực tiếp URL của bài học không cho xem thử.	Hệ thống từ chối truy cập nội dung học tập và yêu cầu mua hoặc đăng ký khóa học.	Đạt
TC012	Phát video bài giảng HLS	Học sinh mở bài học có video đã xử lý HLS.	Trình phát tải được playlist <code>index.m3u8</code> , phát các segment video và không cho truy cập tên tệp HLS sai định dạng.	Đạt
TC013	Hỏi AI khi có học liệu liên quan	Học sinh đặt câu hỏi về nội dung bài học đã có tài liệu được đánh chỉ mục.	AI trả lời bằng tiếng Việt, sử dụng ngữ cảnh truy xuất được và trả về citation tương ứng.	Đạt

Bảng 4.20 - Tiếp trang trước

Mã	Mục tiêu kiểm thử	Input/Thao tác	Kết quả mong đợi	Kết quả
TC014	Hỏi AI khi bài học chưa có tài liệu	Học sinh hỏi về nội dung bài học hiện tại nhưng bài học chưa có tài liệu học tập.	AI không tự bịa nội dung, phản hồi rằng tài liệu hiện tại chưa đủ để trả lời chắc chắn.	Đạt

4.4.4 Kiểm thử theo dõi học viên và bài kiểm tra

Nhóm kiểm thử này đối chiếu trực tiếp với yêu cầu quản lý học tập: giáo viên phải nhận biết học sinh nào chưa học, chưa nộp bài hoặc có bài tự luận chờ chấm, thay vì chỉ quản lý nội dung khóa học.

Bảng 4.21: Test case theo dõi học viên và bài kiểm tra

Mã	Mục tiêu kiểm thử	Input/Thao tác	Kết quả mong đợi	Kết quả
TC015	Xem danh sách học viên của khóa học	Mở tab Quản lý học viên của một khóa học có nhiều bản ghi ghi danh.	Hiển thị đúng học viên, trạng thái ghi danh và phần trăm tiến độ tổng quan.	Đạt
TC016	Xem chi tiết tiến độ một học sinh	Chọn một học sinh trong danh sách.	Hiển thị curriculum theo thứ tự chương-bài, bài đã hoàn thành, chưa hoàn thành và tiến độ tổng.	Đạt
TC017	Phân biệt chưa nộp bài	Mở kết quả một bài kiểm tra có học sinh ghi danh nhưng không có submission.	Học sinh được hiển thị ở trạng thái chưa nộp, không bị bỏ khỏi danh sách.	Đạt
TC018	Tổng hợp nhiều lượt làm	Học sinh có nhiều submission đã hoàn tất cho cùng assessment.	Bảng tổng hợp hiển thị số lượt làm và điểm cao nhất; chi tiết vẫn giữ từng lượt.	Đạt
TC019	Chấm bài tự luận	Mở submission có câu tự luận ở trạng thái chờ chấm, nhập điểm và hoàn tất.	Điểm cuối cùng được cập nhật và học sinh nhận thông báo kết quả.	Đạt

Mã	Mục tiêu kiểm thử	Input/Thao tác	Kết quả mong đợi	Kết quả
TC020	Không lộ đáp án trước khi hoàn tất	Mở preview hoặc kết quả của submission đang làm/chờ chấm.	API không trả đáp án đúng và giải thích; dữ liệu này chỉ xuất hiện sau khi hoàn tất chấm.	Đạt
TC021	Tự nộp khi hết hạn phía máy chủ	Đề một submission ở trạng thái đang làm vượt quá hạn cá nhân hoặc giờ đóng bài.	Backend chuyển bài sang tự nộp; các câu có đáp án xác định được chấm tự động, câu tự luận chuyển chờ chấm.	Đạt
TC022	Khôi phục lượt làm đang dở	Đóng trang rồi mở lại bài khi submission còn hiệu lực.	Hệ thống trả đúng submission cũ, đáp án đã lưu và thời gian còn lại, không tạo lượt mới.	Đạt

4.4.5 Đánh giá định lượng AI Tutor RAG

Đánh giá định lượng được thực hiện trên khóa HSG “HẠT” sau khi khóa học có hai bài học và bốn tài liệu đã xử lý thành công. Bài 1 có 39 chunk, Bài 2 có 46 chunk, tổng cộng 85 chunk từ slide, tài liệu đọc và transcript bài giảng. Phạm vi truy xuất được giới hạn theo `lessonId`: khi học sinh đang ở bài nào, hệ thống chỉ xét các chunk thuộc bài đó. Cấu hình đánh giá dùng mô hình `text-embedding-3-small` để tạo embedding, `gpt-4.1-mini` để sinh câu trả lời, ngưỡng tương đồng 0,45 và tối đa 6 đoạn ngữ cảnh.

Bộ dữ liệu gồm 25 câu hỏi: 15 câu có đáp án tập trung trong một tài liệu, 5 câu cần tổng hợp từ hai tài liệu trong cùng bài học và 5 câu không có đáp án trong học liệu. Với 20 câu có đáp án, các chunk đúng, tài liệu đúng và câu trả lời tham chiếu được đối chiếu thủ công từ corpus trước khi chạy đánh giá. Năm câu còn lại kiểm tra khả năng từ chối thay vì dùng kiến thức nền của mô hình để suy đoán. Mỗi câu được chạy với lịch sử hội thoại rỗng nhằm tránh ảnh hưởng từ các lượt hỏi trước.

Ngoài `Recall@6` và `MRR` đã trình bày ở Chương 5, phép đo còn sử dụng `Source Recall@6`, tức tỷ lệ tài liệu đúng xuất hiện trong tối đa 6 kết quả truy xuất. Tỷ lệ lấy đủ nguồn cho biết hệ thống có tìm được toàn bộ tài liệu cần thiết hay không; tỷ lệ từ chối đo số câu ngoài tài liệu được nhận diện đúng. `Token F1` đo mức trùng khớp từ vựng với đáp án tham chiếu, còn tỷ lệ bao phủ citation đo phần câu trong

câu trả lời có ký hiệu nguồn.

Bảng 4.22: Kết quả đánh giá định lượng AI Tutor RAG

Chỉ số	Kết quả	Ý nghĩa
Recall@6 theo chunk	0,771	Mức thu hồi các chunk đúng trong top 6
Hit rate truy xuất	90,0% (18/20)	Câu có ít nhất một chunk đúng
MRR	0,746	Vị trí trung bình của chunk đúng đầu tiên
Source Recall@6	0,875	Mức thu hồi các tài liệu nguồn đúng
Lấy đủ toàn bộ nguồn	80,0% (16/20)	Câu lấy được mọi tài liệu cần thiết
Tỷ lệ trả lời câu có đáp án	85,0% (17/20)	Câu có đáp án không bị từ chối
Độ chính xác từ chối	100% (5/5)	Câu ngoài tài liệu được từ chối đúng
Token F1	0,532	Mức trùng khớp từ vựng với đáp án tham chiếu
Bao phủ citation theo câu	0,323	Tỷ lệ câu trong câu trả lời có gắn nguồn
Độ trễ trung bình	3,99 giây	Thời gian truy xuất và sinh câu trả lời
Lỗi gọi mô hình	0/25	Số mẫu không hoàn tất do lỗi provider

Bảng 4.23: Kết quả truy xuất theo nhóm câu hỏi

Nhóm câu hỏi	Recall@6	Source Recall@6
Một tài liệu	0,811	0,933
Nhiều tài liệu trong cùng bài	0,650	0,700

Kết quả cho thấy RAG hoạt động tốt hơn khi đáp án tập trung trong một tài liệu. Với câu cần tổng hợp nhiều nguồn, top 6 có thể bị chiếm bởi các chunk tương tự của cùng một tài liệu nên chưa lấy đủ nguồn còn lại. Ba câu có đáp án nhưng hệ thống từ chối là HAT-D07, HAT-D12 và HAT-M05; hai câu đầu không truy xuất được chunk chuẩn, câu cuối chỉ lấy được một phần nguồn cần tổng hợp. Ngược lại, cả 5 câu không có đáp án đều được từ chối, kể cả khi hệ thống lấy được đoạn gần chủ đề. Điều này cho thấy prompt kiểm tra bằng chứng trực tiếp đã hạn chế được việc mô hình tự bổ sung kiến thức ngoài học liệu.

Token F1 bằng 0,532 không đồng nghĩa với độ chính xác ngữ nghĩa chỉ đạt 53,2%, vì chỉ số này nhạy với cách diễn đạt. Tỷ lệ bao phủ citation 0,323 cho thấy mô hình đã trả nguồn nhưng chưa gắn citation đều vào mọi câu chứa thông tin. Hai điểm cần cải thiện tiếp theo là kết hợp truy xuất vector với từ khóa, đa dạng hóa tài liệu trong top-K hoặc bổ sung reranking, đồng thời kiểm soát chặt hơn vị trí gắn citation. Bộ dữ liệu và kết quả được lưu trong `rag_benchmark_hat.jsonl`, `rag_evaluation_hat_retrieval.json` và `rag_evaluation_hat_generation.json` để có thể chạy lại. Do corpus mới giới hạn ở một khóa học và hai bài học, kết quả này chỉ phản ánh bộ đánh giá nội bộ, chưa đại diện cho mọi môn học hoặc mọi dạng học liệu.

4.4.6 Kết quả kiểm tra tự động

Bảng 4.24: Kết quả kiểm tra kỹ thuật trên mã nguồn

Thành phần	Lệnh/Phạm vi kiểm tra	Kết quả
Frontend	<code>npm run lint</code> ; <code>npm run build</code>	Lint và build thành công
Backend	<code>npm run lint</code> ; <code>npm run build</code> ; <code>npm test</code>	Lint, build thành công; 22/22 test đạt
Cơ sở dữ liệu	<code>npx prisma validate</code>	Prisma schema hợp lệ
Tài liệu API	Kiểm tra parse OpenAPI và toàn bộ <code>\$ref</code>	83 path; không có tham chiếu lỗi
AI Service	Kiểm thử evaluator; chạy benchmark 25 câu	4/4 test đạt; hoàn tất 25/25 mẫu, không lỗi provider

Tổng cộng, báo cáo trình bày 22 test case hộp đen cho năm nhóm chức năng, một benchmark RAG gồm 25 câu và các bước kiểm tra tự động cho các thành phần kỹ thuật. Kết quả này chứng minh các luồng chính hoạt động trong môi trường phát triển, nhưng chưa thay thế cho kiểm thử tải, kiểm thử bảo mật chuyên sâu và đánh giá chấp nhận với người dùng thật.

4.5 Triển khai

Hệ thống được triển khai trên Cloud Server của CloudFly, sử dụng Ubuntu 24.04 LTS, 2 vCPU, 3,8 GiB RAM, ổ đĩa 77 GiB và 2 GiB swap. Phần mềm được truy cập tại <https://heducation.io.vn>; API được cung cấp tại <https://api.heducation.io.vn>. Hai tên miền cùng trỏ tới địa chỉ IP của máy chủ và sử dụng chứng chỉ TLS do Let's Encrypt cấp.

Các thành phần Next.js, Express, Python AI Service và PostgreSQL được đóng gói bằng Docker và vận hành bằng Docker Compose. Caddy tiếp nhận yêu cầu từ

Internet và chuyển tiếp tới *frontend* hoặc *backend* theo tên miền. PostgreSQL và AI Service chỉ trao đổi dữ liệu trong mạng Docker. Ảnh, video HLS và tài liệu được lưu trên Cloudflare R2, còn volume PostgreSQL lưu dữ liệu nghiệp vụ và dữ liệu vector.

Quá trình triển khai gồm bốn bước: (i) đóng gói mã nguồn thành ba image cho *frontend*, *backend* và AI Service; (ii) khởi tạo PostgreSQL trên volume mới và áp dụng 17 migration bằng Prisma; (iii) cấu hình riêng các biến môi trường về CSDL, JWT, R2, SePay và mô hình AI trên máy chủ; và (iv) khởi động các thành phần bằng Docker Compose trước khi Caddy tiếp nhận yêu cầu. Máy chủ chỉ cho phép kết nối SSH bằng khóa công khai và lưu lượng web qua HTTP/HTTPS.

Sau khi triển khai, hệ thống được kiểm tra từ một máy tính bên ngoài VPS. Phép đo tải nhẹ gửi 100 yêu cầu cho mỗi địa chỉ với 10 luồng đồng thời. Thời gian trong Bảng 4.25 là thời gian hoàn thành một yêu cầu HTTP, không bao gồm thời gian trình duyệt dựng giao diện.

Bảng 4.25: Kết quả triển khai và đo thử trên môi trường public

Hạng mục	Kết quả quan sát
Truy cập công khai	Trang web và API cùng trả mã HTTP 200; chứng chỉ TLS được xác minh hợp lệ.
Frontend	100/100 yêu cầu thành công; thời gian trung bình 285,0 ms; P95 đạt 409,5 ms; thông lượng 33,6 yêu cầu/giây.
API healthcheck	100/100 yêu cầu thành công; thời gian trung bình 78,2 ms; P95 đạt 295,1 ms; thông lượng 123,3 yêu cầu/giây.
Cơ sở dữ liệu	17 migration được áp dụng thành công; lần khởi động lại không còn migration chờ xử lý.
Đăng nhập	Tài khoản admin được tạo trên CSDL mới và đăng nhập qua địa chỉ HTTPS thành công.
Sao lưu và phục hồi	Backup PostgreSQL được phục hồi vào CSDL tạm; số tài khoản sau phục hồi khớp với CSDL nguồn.
Khởi động lại	Các container hoạt động trở lại, dữ liệu PostgreSQL và chứng chỉ HTTPS được giữ nguyên.

Trong phép đo trên, hệ thống không ghi nhận yêu cầu lỗi. Kết quả này phản ánh hoạt động của hai endpoint với 10 luồng đồng thời, chưa phải giới hạn tải tối đa của toàn hệ thống. Môi trường hiện chưa có người dùng thật nên chưa có số liệu về số người truy cập hoặc phản hồi người dùng. PostgreSQL được sao lưu tự động hằng ngày; các bản sao được giữ trong bảy ngày.

Chương này đã trình bày thiết kế, kết quả xây dựng, kiểm thử và quá trình triển khai hệ thống trên máy chủ công khai. Chương 5 tiếp tục phân tích các giải pháp kỹ thuật chính gồm RAG, kiểm soát làm bài, HLS và xử lý *webhook* thanh toán.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày quá trình phân tích, thiết kế, xây dựng và kiểm thử hệ thống. Giá trị chính của đề án nằm ở việc kết nối các nghiệp vụ vốn phân tán thành một quy trình thống nhất từ bán khóa học, cấp quyền, tổ chức học tập đến theo dõi và đánh giá học sinh. Trên cơ sở đó, chương này chọn bốn bài toán triển khai tiêu biểu để phân tích sâu: hỏi đáp theo học liệu nội bộ, kiểm soát quá trình làm bài, xử lý video dung lượng lớn và tự động hóa ghi danh sau thanh toán. Bốn nội dung này cùng hỗ trợ giải pháp hệ thống, không đại diện cho những chức năng tách rời hoặc một kỹ thuật duy nhất. Mỗi nội dung được trình bày theo ba phần: bài toán, giải pháp đã triển khai, kết quả và giới hạn đánh giá.

5.1 Trợ lý AI hỏi đáp theo học liệu nội bộ

Trợ lý AI là một chức năng hỗ trợ trong không gian học tập, giúp học sinh tra cứu lại nội dung khi giáo viên chưa thể phản hồi ngay. Trong phạm vi đề án, mục tiêu không phải nghiên cứu hoặc huấn luyện mô hình ngôn ngữ mới, mà là tích hợp một luồng hỏi đáp bám vào học liệu do giáo viên cung cấp. Quy trình bắt đầu khi học liệu được tải lên và kết thúc khi học sinh nhận câu trả lời có nguồn dẫn.

5.1.1 Bài toán và vấn đề

Trong lớp học trực tuyến, học sinh thường xem lại bài giảng vào thời điểm giáo viên không thể trả lời ngay. Một trợ lý AI đặt trực tiếp trong màn hình học bài có thể hỗ trợ các thắc mắc cơ bản. Tuy nhiên, nếu hệ thống chỉ gửi câu hỏi tới một mô hình ngôn ngữ lớn, câu trả lời có thể dựa trên tri thức tổng quát thay vì nội dung bài giảng. RAG khắc phục hạn chế này bằng cách truy xuất tài liệu liên quan làm ngữ cảnh trước khi sinh câu trả lời [27]. Ví dụ, câu hỏi “phần này cần nhớ gì” chỉ có ý nghĩa khi hệ thống biết học sinh đang học bài nào và bài đó có tài liệu gì.

Bài toán đặt ra gồm ba thách thức kỹ thuật: (i) học liệu có nhiều dạng như mô tả bài học, nội dung nhập trực tiếp, PDF, DOCX và PPTX nên cần được chuẩn hóa trước khi lập chỉ mục; (ii) kết quả truy xuất phải được giới hạn đúng bài học học sinh đang mở để tránh trộn kiến thức giữa các bài; (iii) hệ thống cần từ chối trả lời khi không tìm thấy tài liệu phù hợp. Trong bối cảnh giáo dục, một câu trả lời có vẻ hợp lý nhưng không dựa trên học liệu tiềm ẩn nhiều rủi ro hơn việc thông báo chưa đủ thông tin.

Do đó, trọng tâm triển khai của phần này không nằm ở thao tác gọi API mô hình ngôn ngữ, mà ở việc kiểm soát dữ liệu đầu vào và liên kết câu trả lời với đúng ngữ cảnh học tập. Quy trình cần biến học liệu thành dữ liệu truy xuất được, chọn

đoạn liên quan, đưa ngữ cảnh vào prompt, yêu cầu nguồn dẫn và lưu lại nguồn đã sử dụng để kiểm tra sau này.

5.1.2 Giải pháp

Luồng vận hành của tính năng được chia thành hai giai đoạn: chuẩn bị tri thức khi giáo viên thêm hoặc cập nhật học liệu, và hỏi đáp khi học sinh tương tác tại màn hình học bài.

a, Chuẩn bị tri thức từ học liệu. Khi giáo viên thêm học liệu, phía máy chủ lưu bản ghi `lesson_materials` cùng trạng thái xử lý. Nội dung văn bản nhập trực tiếp có thể được đưa ngay vào bước đánh chỉ mục; với tệp PDF, DOCX hoặc PPTX, dịch vụ AI tải tệp về thư mục tạm để trích xuất văn bản.

Trong bước trích xuất, hệ thống ưu tiên MarkItDown để chuyển nhiều định dạng tài liệu về văn bản/Markdown thống nhất. Cách này giảm số bộ xử lý riêng lẻ và tạo đầu vào sạch hơn cho bước chia đoạn. Với các tệp MarkItDown không xử lý được, hệ thống dùng pypdf cho PDF, python-docx cho DOCX và python-pptx cho PPTX. Nội dung PDF được gắn số trang, còn PPTX được gắn số slide, nhờ đó kết quả truy xuất vẫn có thể lần ngược về nguồn ban đầu.

Sau khi trích xuất thành công, hệ thống cập nhật `extracted_text`, chuyển trạng thái học liệu sang `ready` và xóa lỗi xử lý cũ nếu có. Nếu không trích xuất được nội dung hoặc tệp không hợp lệ, trạng thái chuyển sang `failed` và hệ thống lưu thông báo lỗi rút gọn. Cách tổ chức này giúp giáo viên biết học liệu nào đã sẵn sàng cho AI Tutor, học liệu nào cần tải lại hoặc chỉnh sửa.

b, Chiến lược chunking. Văn bản học liệu thường dài và có cấu trúc không đều. Nếu chia theo số ký tự cố định, một khái niệm có thể bị cắt giữa câu. Nếu giữ nguyên toàn bộ tài liệu, truy xuất sẽ kém chính xác và prompt trở nên quá dài. Hệ thống sử dụng `RecursiveCharacterTextSplitter` để chia văn bản theo các mức ưu tiên: đoạn văn, dòng, câu, từ và cuối cùng là ký tự. Kích thước mặc định của mỗi chunk là 1000 ký tự, với phần chồng lấn 150 ký tự giữa hai chunk liên tiếp.

Phần chồng lấn có vai trò giữ lại ngữ cảnh ở ranh giới giữa hai đoạn. Ví dụ, nếu một định nghĩa nằm cuối chunk trước và ví dụ minh họa nằm đầu chunk sau, overlap giúp hai đoạn vẫn giữ được một phần liên hệ. Mỗi chunk được tính mã băm SHA-256 từ nội dung. Khi một học liệu được xử lý lại, hệ thống xóa các chunk cũ theo `material_id` rồi ghi lại các chunk mới, tránh tình trạng cơ sở tri thức còn chứa phiên bản cũ của tài liệu.

c, Tạo và lưu trữ vector. Mỗi đoạn văn bản được chuyển thành vector bằng

mô hình *embedding* tương thích OpenAI. Cấu hình hiện tại sử dụng `text-embedding-3-small` với 1536 chiều. Trước khi lưu, hệ thống đối chiếu số chiều vector với cấu hình; giá trị không khớp sẽ dừng quá trình xử lý và đánh dấu học liệu lỗi. Pgvector yêu cầu các vector có số chiều nhất quán, nếu không truy vấn có thể thất bại hoặc cho kết quả không đáng tin cậy.

Các đoạn được lưu vào bảng `knowledge_chunks` cùng khóa học, bài học, tài liệu gốc, loại nguồn, thứ tự, mô hình *embedding* và thời điểm đánh chỉ mục. Nhờ đó, mỗi kết quả truy xuất đều có thể truy ngược về bài học hoặc tài liệu cụ thể.

d, Hỏi đáp theo luồng RAG. Khi học sinh đặt câu hỏi, giao diện gửi nội dung kèm `course_id` và `lesson_id`. Dịch vụ AI tạo vector cho câu hỏi bằng chính mô hình đã dùng khi đánh chỉ mục. Truy vấn pgvector chỉ xét các đoạn thuộc đúng khóa học và bài học hiện tại. Độ tương đồng được tính từ khoảng cách vector; điểm càng cao thì đoạn văn càng gần câu hỏi về mặt ngữ nghĩa.

Ở mức triển khai, phía máy chủ nhận câu hỏi qua API, lưu tin nhắn vào bảng `tutor_chat_messages`, sau đó gọi dịch vụ AI nội bộ. Dữ liệu gửi sang dịch vụ AI có cấu trúc rút gọn như sau:

```
{
  "course_id": "...",
  "lesson_id": "...",
  "question": "...",
  "chat_history": [
    { "role": "user", "content": "..." },
    { "role": "assistant", "content": "..." }
  ]
}
```

Trong đó, `course_id` và `lesson_id` tạo thành phạm vi truy xuất của bài học hiện tại, `question` là câu hỏi mới của học sinh, còn `chat_history` chứa tối đa các lượt trao đổi gần nhất để giữ mạch hội thoại. Dịch vụ AI trả về câu trả lời cùng danh sách nguồn dẫn:

```
{
  "answer": "... [1] ",
  "citations": [
    {
      "chunk_id": "...",
      "rank": 1,
      "score": 0.82,
```

```

    "source_title": "...
  }
]
}

```

Cấu trúc này làm rõ ranh giới giữa máy chủ LMS và dịch vụ AI. Máy chủ chịu trách nhiệm xác thực học sinh, quản lý phiên trò chuyện và lưu nguồn dẫn; dịch vụ AI truy xuất các đoạn liên quan, tạo *prompt* và sinh câu trả lời.

Với câu hỏi q và bài học hiện tại l , tập ứng viên trước hết được giới hạn như sau:

$$\mathcal{C}_l = \{c \mid c.courseId = q.courseId \wedge c.lessonId = l\} \quad (5.1)$$

Với mỗi chunk c trong $\mathcal{C}_l$, hệ thống dùng độ tương đồng cosine $\text{sim}(c, q)$ làm điểm xếp hạng. Các chunk có điểm nhỏ hơn 0,45 bị loại; hệ thống lấy tối đa 6 đoạn còn lại. Cách lọc cứng theo `lesson_id` phù hợp với giao diện hỏi đáp gắn trong từng bài học và tránh việc một transcript ở bài khác được dùng để trả lời thay cho bài hiện tại.

e, Tạo *prompt* và sinh câu trả lời. Nếu tìm được ngữ cảnh phù hợp, *prompt* chứa tên khóa học, tên bài học, mô tả bài học và các đoạn đã truy xuất. Môi trường đánh giá sử dụng mô hình `gpt-4.1-mini` với tham số `temperature` bằng 0,2. Nội dung *prompt* yêu cầu mô hình chỉ dùng ngữ cảnh của bài học, không suy diễn từ đoạn văn chỉ gần chủ đề và phải ghi nguồn bằng ký hiệu [1], [2].

Nếu không tìm thấy đoạn tài liệu đủ liên quan, hệ thống không gửi một *prompt* mở để mô hình tự suy diễn. *Prompt* trong trường hợp này yêu cầu mô hình nói rõ rằng tài liệu hiện tại chưa cung cấp đủ thông tin và gợi ý học sinh hỏi lại cụ thể hơn hoặc kiểm tra tài liệu bài học. Nếu bài học hiện tại chưa có tài liệu, *prompt* cũng nhắc mô hình không được dùng tài liệu của bài khác để giả vờ trả lời cho bài hiện tại.

f, Lưu phiên chat và nguồn dẫn. Kết quả trả về gồm câu trả lời và danh sách citations. Mỗi citation lưu định danh chunk, định danh tài liệu, thứ hạng, điểm số, đoạn trích ngắn, tiêu đề nguồn, bài học liên quan và loại nguồn. Backend kiểm tra các chunk thuộc đúng khóa học trước khi lưu citation vào CSDL. Với chế độ streaming, hệ thống gửi sự kiện citations trước, sau đó gửi dần nội dung câu trả lời qua Server-Sent Events. Cách làm này giúp giao diện có thể hiển thị nguồn tham chiếu sớm, đồng thời người học không phải chờ toàn bộ câu trả lời được sinh xong mới thấy phản hồi.

5.1.3 Kết quả đạt được và đánh giá

Giải pháp đã hình thành một luồng hỏi đáp có kiểm soát nguồn dữ liệu. Hệ thống không chỉ trả lời bằng văn bản, mà còn lưu lại các nguồn đã được dùng để tạo câu trả lời. Với học sinh, citation giúp các em quay lại đúng tài liệu liên quan. Với giáo viên hoặc người quản trị, citation giúp kiểm tra xem câu trả lời có dựa trên học liệu phù hợp hay không.

Để đánh giá chất lượng truy xuất, đề án sử dụng hai chỉ số chính là *Recall@K* và *Mean Reciprocal Rank*. Với một tập câu hỏi kiểm thử, giả sử G là tập chunk đúng được gán nhãn thủ công và C_K là tập K chunk đầu tiên hệ thống truy xuất được, *Recall@K* được tính như sau:

$$\text{Recall@K} = \frac{|C_K \cap G|}{|G|} \quad (5.2)$$

Chỉ số này đo khả năng hệ thống lấy được các đoạn học liệu cần thiết. Bên cạnh đó, *Mean Reciprocal Rank* đánh giá vị trí xuất hiện của chunk đúng đầu tiên:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i} \quad (5.3)$$

Trong đó N là số câu hỏi kiểm thử và rank_i là vị trí của chunk đúng đầu tiên đối với câu hỏi thứ i . Nếu chunk đúng thường đứng ở vị trí đầu, MRR sẽ cao. Hai chỉ số này phù hợp với bài toán của đề án vì chất lượng câu trả lời phụ thuộc trực tiếp vào việc truy xuất đúng học liệu trước khi sinh câu trả lời.

Ngoài truy xuất, hệ thống còn đánh giá tỷ lệ câu trả lời có citation bằng cách tách câu và kiểm tra ký hiệu trích dẫn. Nếu câu trả lời có nhiều mệnh đề kiến thức nhưng không gắn nguồn, kết quả này cho thấy prompt chưa đủ chặt hoặc mô hình chưa tuân thủ tốt yêu cầu trích dẫn. Công cụ đánh giá RAG của dịch vụ AI đã hiện thực các phép đo này, gồm recall truy xuất, MRR, tỷ lệ câu có citation, độ trễ trung bình, số đoạn ngữ cảnh trung bình và số lần sinh câu trả lời thất bại. Nhờ đó, việc đánh giá AI Tutor không chỉ dựa trên cảm nhận khi đọc câu trả lời, mà có các chỉ số kiểm tra được.

Để các chỉ số trên có ý nghĩa, tập đánh giá phải phản ánh đúng cách hệ thống được sử dụng. Corpus thực nghiệm gồm 4 tài liệu thuộc 2 bài học của khóa HSG “HẠT”, trong đó có slide, tài liệu đọc và transcript bài giảng; sau bước xử lý, corpus có 85 chunk. Bộ đánh giá gồm 25 câu được xây dựng sau khi chốt corpus: 15 câu truy xuất trực tiếp từ một tài liệu, 5 câu tổng hợp nhiều tài liệu nhưng trong cùng

bài học và 5 câu không có đáp án. Với 20 câu có đáp án, các chunk và tài liệu liên quan được gán nhãn thủ công trước khi chạy đánh giá; lịch sử hội thoại được để trống để các mẫu không ảnh hưởng lẫn nhau.

Kết quả chi tiết được trình bày tại Mục 4.4.5. Trên bộ đánh giá nội bộ, hệ thống đạt Recall@6 bằng 0,771, MRR bằng 0,746 và Source Recall@6 bằng 0,875. Hệ thống trả lời 17/20 câu có đáp án và từ chối đúng 5/5 câu không có thông tin, với thời gian phản hồi trung bình khoảng 3,99 giây. Nhóm câu hỏi một tài liệu đạt Source Recall@6 bằng 0,933, trong khi nhóm cần nhiều tài liệu chỉ đạt 0,700. Chênh lệch này cho thấy giới hạn chính nằm ở khả năng lấy đủ nguồn trong top-K; ngoài ra, tỷ lệ câu có citation mới đạt 0,323. Do bộ đánh giá chỉ thuộc một khóa học và hai bài học, các số liệu này là kết quả thực nghiệm nội bộ, không được dùng để suy rộng thành chất lượng chung cho mọi môn học hoặc mọi loại tài liệu.

5.2 Kiểm soát quá trình làm bài kiểm tra

Phân hệ bài kiểm tra phục vụ ba bối cảnh: làm thử công khai, kiểm tra định kỳ trong khóa học và quiz tự luyện sau bài giảng. Thiết kế phải kiểm soát đúng vị trí sử dụng, số lượt làm và trạng thái bài nộp, đồng thời bảo toàn dữ liệu khi học sinh thoát trang, mất mạng hoặc quay lại làm tiếp.

5.2.1 Bài toán và vấn đề

Trong thực tế vận hành, bài kiểm tra xuất hiện ở ba vị trí: (i) khu vực luyện tập công khai cho học sinh làm thử; (ii) phạm vi khóa học cho bài thi định kỳ hoặc bài tập chung; (iii) bài học dạng quiz để tự luyện ngay sau nội dung vừa học. Cả ba trường hợp cùng sử dụng cấu trúc đề, câu hỏi, đáp án và bài làm, nhưng khác nhau về quyền truy cập, thời gian mở, số lượt làm và vị trí hiển thị.

Nếu tách ba trường hợp trên thành ba mô hình dữ liệu riêng, hệ thống sẽ bị lặp logic tạo đề, lưu đáp án, chấm điểm và thống kê kết quả. Ngược lại, nếu gom tất cả vào một loại bài kiểm tra duy nhất mà không biểu diễn rõ vị trí sử dụng, hệ thống khó kiểm soát quyền truy cập và khó mở rộng. Ngoài ra, bài làm của học sinh phải được bảo toàn trong các tình huống không lý tưởng. Khi đang làm bài, học sinh có thể mất kết nối mạng, đóng nhầm trình duyệt hoặc quay lại sau một khoảng thời gian. Hệ thống cần cho phép tiếp tục lượt làm đang dở nếu lượt làm đó vẫn hợp lệ, đồng thời không được tạo thêm lượt làm mới ngoài số lần cho phép.

5.2.2 Giải pháp

Giải pháp được thiết kế quanh ba khái niệm chính: `assessment`, `assessment_placement` và `submission`. `Assessment` lưu nội dung của một bài kiểm tra; `assessment_placement` lưu đúng một vị trí sử dụng và các quy tắc

mở bài; `submission` lưu một lượt làm cụ thể. Trong phạm vi sản phẩm, một `assessment` chỉ được dùng cho một `placement`. Khi cần dùng lại đề ở đợt hoặc bối cảnh khác, giáo viên clone `assessment` để dữ liệu vận hành và `submission` của từng đợt độc lập, đồng thời vẫn tái sử dụng được quy trình biên soạn.

a, Tách dữ liệu xem trước và dữ liệu phòng làm bài. API xem trước chỉ trả về metadata của bài kiểm tra như tiêu đề, thời gian làm bài, số lượt tối đa, trạng thái mở và thông tin vị trí sử dụng. Nội dung câu hỏi không được trả về ở bước này. Chỉ khi học sinh bắt đầu hoặc tiếp tục một lượt làm hợp lệ, hệ thống mới trả về `workspace` gồm `section`, `item`, đáp án đã lưu và thời gian còn lại. Cách tách này giúp giảm nguy cơ lộ đề trước khi học sinh thật sự vào bài.

b, Kiểm soát ba vị trí sử dụng. `Placement` công khai được dùng cho bài luyện tập mở, không gắn với khóa học hoặc bài học và cần có slug để học sinh truy cập từ khu vực công khai. `Placement` theo khóa học được dùng cho bài kiểm tra định kỳ hoặc bài tập giao thêm, bắt buộc gắn với một khóa học cụ thể và yêu cầu học sinh đã ghi danh. `Placement` theo bài học được dùng cho quiz tự luyện sau bài giảng, bắt buộc gắn với một bài học có kiểu `quiz`; khóa học được suy ra từ bài học để tránh gắn nhầm quiz sang khóa khác.

Khi giáo viên hoặc quản trị viên xuất bản bài kiểm tra, hệ thống kiểm tra tính hợp lệ của `placement`. Với `placement` theo khóa học và theo bài học, môn học và khối lớp của bài kiểm tra phải khớp với khóa học. Sau khi `assessment` đã có `submission`, các thay đổi có thể làm sai lệch lịch sử bài làm bị hạn chế. Nếu cần tổ chức một đợt mới, thao tác clone tạo bản sao câu hỏi và cấu hình cơ sở để người dùng chỉnh `placement` mà không trộn kết quả cũ.

Hệ thống cũng phân biệt quyền của quản trị viên và giáo viên. Quản trị viên có thể thao tác ở phạm vi toàn hệ thống, còn giáo viên chỉ được gắn bài kiểm tra vào các khóa học hoặc bài học do mình quản lý. Điều này giữ được sự đơn giản của mô hình vai trò hiện tại, nhưng vẫn tránh việc giáo viên can thiệp vào nội dung của người khác.

c, Tạo và tiếp tục lượt làm. Khi học sinh bắt đầu làm bài, hệ thống kiểm tra ba nhóm điều kiện: người dùng đã đăng nhập, `placement` đang trong thời gian mở và học sinh có quyền truy cập. Với `placement` theo khóa học hoặc bài học, quyền truy cập được xác định thông qua bản ghi ghi danh. Sau đó, hệ thống kiểm tra xem học sinh đã có `submission` ở trạng thái `doing` hay chưa. Nếu có, hệ thống trả lại `submission` đang làm thay vì tạo mới. Nếu chưa có, hệ thống đếm số lượt đã làm và chỉ tạo `submission` mới khi chưa vượt quá `max_attempts`.

Ở phía giao diện, khi học sinh mở lại một bài kiểm tra đang làm dở, hệ thống

nhận biết submission còn trạng thái *doing* và chuyển học sinh vào đúng phòng làm bài cũ. *Workspace* được mở bằng cả *placementId* và *submissionId*, nên học sinh không thể tùy ý mở bài làm của người khác. Các đáp án đã lưu trước đó được dựng lại từ dữ liệu *submission*, giúp học sinh có thể tiếp tục làm nếu đã kịp lưu bài trước khi thoát trang hoặc mất kết nối.

Thời gian còn lại được tính theo hai ràng buộc: thời lượng làm bài và thời điểm đóng *placement*. Nếu bài kiểm tra có cả hai ràng buộc, hệ thống lấy mốc sớm hơn làm hạn cuối:

$$\text{deadline} = \min(\text{startTime} + \text{timeLimit}, \text{closeTime}) \quad (5.4)$$

$$\text{remainingSeconds} = \max(0, \text{deadline} - \text{now}) \quad (5.5)$$

Cách tính này xử lý được tình huống học sinh bắt đầu làm bài gần thời điểm đóng. Ví dụ, bài kiểm tra cho phép làm 60 phút nhưng đóng sau 20 phút nữa, hệ thống chỉ cho phép làm trong 20 phút còn lại.

d, Lưu đáp án và xử lý rủi ro khi làm bài. Mỗi lần học sinh bấm lưu nháp hoặc nộp bài, hệ thống kiểm tra *submission* thuộc đúng học sinh, trạng thái còn là *doing*, *placement* vẫn hợp lệ và học sinh vẫn có quyền truy cập. Sau đó, hệ thống kiểm tra từng câu trả lời. Với câu trắc nghiệm, các option được chọn phải thuộc đúng câu hỏi và không được lặp. Nếu câu trắc nghiệm chỉ cho phép một đáp án, hệ thống không cho phép gửi nhiều option. Với câu đúng sai, mỗi mệnh đề chỉ được trả lời một lần. Với câu tự luận, nội dung được lưu để giáo viên chấm sau.

Với trường hợp mất mạng hoặc đóng nhầm trình duyệt, hệ thống xử lý theo nguyên tắc rõ ràng: dữ liệu đã lưu vào *submission* thì được khôi phục khi học sinh quay lại, còn dữ liệu vừa nhập nhưng chưa gửi lên server thì không được coi là đã lưu. Vì vậy, giao diện có cảnh báo khi học sinh rời trang trong lúc còn thay đổi chưa lưu. Cách làm này không che giấu rủi ro mạng, nhưng giúp trạng thái bài làm nhất quán và dễ giải thích: muốn bảo toàn đáp án, học sinh cần lưu nháp hoặc nộp bài thành công.

Việc kiểm tra ở phía máy chủ là bắt buộc vì giao diện trình duyệt không phải chốt chặn bảo mật cuối cùng. Người dùng có thể sửa yêu cầu HTTP bằng công cụ bên ngoài giao diện. Nếu máy chủ không đối chiếu câu hỏi và phương án, học sinh có thể gửi đáp án không thuộc bài kiểm tra hiện tại hoặc tạo dữ liệu bài làm không nhất quán.

Các API chính trong luồng làm bài được tách theo đúng trạng thái nghiệp vụ:

```
GET    /learning/assessment-placements/
       :placementId
POST   /learning/assessment-placements/
       :placementId/attempts
GET    /learning/assessment-placements/
       :placementId/workspace
       ?submissionId=...
PUT    /learning/assessment-submissions/
       :submissionId/answers
POST   /learning/assessment-submissions/
       :submissionId/submit
```

API xem trước chỉ trả về thông tin tổng quan; API tạo lượt làm trả về bài đang thực hiện hoặc bài mới; API phòng thi trả về dữ liệu câu hỏi; hai API còn lại phục vụ lưu nháp và nộp bài. Dữ liệu lưu đáp án được phân biệt theo từng loại câu hỏi:

```
{
  "answers": [
    {
      "itemId": "...",
      "type": "mcq",
      "selectedOptionIds": ["..."]
    },
    {
      "itemId": "...",
      "type": "true_false",
      "selections": [
        { "optionId": "...", "selectedValue": true }
      ]
    },
    {
      "itemId": "...",
      "type": "numeric",
      "answerValue": 12.5
    },
    {
      "itemId": "...",
```

```

    "type": "essay",
    "answer": "... "
  }
]
}

```

Trường `type` xác định quy tắc kiểm tra cho từng loại dữ liệu: câu trắc nghiệm dùng `selectedOptionIds`, câu đúng sai dùng `selections`, câu số học dùng `answerValue`, còn câu tự luận dùng `answer`. Cấu trúc này tránh việc gom mọi loại câu trả lời vào một trường văn bản chung rồi xử lý thủ công về sau.

e, Hết giờ, nộp bài và chấm điểm. Phòng thi luôn nhận thời gian còn lại từ máy chủ. Khi hết giờ, giao diện yêu cầu nộp bài để phản hồi ngay; đồng thời máy chủ quét các bài ở trạng thái `doing` đã quá hạn và tự động thu bài ngay cả khi trình duyệt đã đóng. Hạn được tính theo mốc sớm hơn giữa giới hạn cá nhân và giờ đóng bài. Hệ thống cũng ghi nhận vi phạm phòng thi; ở lần vi phạm thứ năm, bài làm được tự nộp. Do quy tắc được thực thi tại máy chủ, việc sửa đồng hồ hoặc tắt JavaScript không kéo dài được thời gian làm bài.

Khi học sinh nộp bài, hệ thống tự chấm các câu có đáp án xác định. Với trắc nghiệm, hệ thống so sánh tập option học sinh chọn với tập option đúng. Với câu đúng sai, điểm được tính theo tỷ lệ số mệnh đề trả lời đúng:

$$\text{point} = \text{maxScore} \times \frac{\text{correctCount}}{\text{totalStatements}} \quad (5.6)$$

Với câu số học, hệ thống so sánh giá trị của học sinh với đáp án bằng phép toán số thập phân, tránh sai số do biểu diễn số thực. Tổng điểm tự động được cộng từ các item có đáp án xác định. Nếu bài kiểm tra có câu tự luận, submission đã nộp hoặc tự nộp nhưng chưa có điểm cuối cùng được xếp vào hàng chờ chấm; hệ thống vẫn tạo câu trả lời rỗng cho câu bị bỏ qua để giáo viên nhìn thấy đầy đủ đề. Nếu không có câu tự luận, điểm cuối cùng được hoàn tất tự động. Sau khi chấm xong, học sinh nhận thông báo và mới có thể xem đáp án đúng cùng phần giải thích do giáo viên nhập.

5.2.3 Kết quả đạt được và đánh giá

Giải pháp giúp hệ thống bao phủ ba bối cảnh sử dụng bằng cùng một mô hình dữ liệu và luồng nghiệp vụ, nhưng mỗi assessment có placement và tập submission riêng. Quy tắc clone khi dùng lại đề đánh đổi một phần dung lượng lưu trữ để đổi lấy lịch sử kết quả rõ ràng, dễ thống kê theo `assessmentId` và tránh tác động chéo giữa các đợt kiểm tra.

Hệ thống cho phép học sinh quay lại lượt làm đang dở nếu submission vẫn còn hiệu lực. Các đáp án đã lưu được khôi phục từ máy chủ, còn thay đổi chưa lưu được cảnh báo trước khi rời trang. Cơ chế này bảo toàn bài làm khi mạng gián đoạn, trình duyệt bị đóng nhầm hoặc học sinh chuyển thiết bị.

Các nhóm test case đánh giá giải pháp gồm: học sinh chưa ghi danh không mở được bài kiểm tra trong khóa học; không thể làm quá số lượt tối đa; mở lại bài đang làm trả về submission cũ; bài đã nộp không được lưu tiếp; item và option phải thuộc đúng đề; hết giờ thì bài được tự động thu; bài có tự luận chuyển sang chờ chấm; bài không có tự luận được hoàn tất và tính điểm tự động. Các trường hợp này bao phủ cả luồng thành công và những rủi ro nghiệp vụ của quá trình làm bài.

5.3 Xử lý video bài giảng bằng HLS và lưu trữ đối tượng

Video bài giảng ảnh hưởng trực tiếp tới trải nghiệm học tập và khả năng vận hành của hệ thống. HLS chia nội dung thành playlist và các đoạn media nhỏ để trình phát tải theo nhu cầu [16]. Đề án kết hợp HLS với Cloudflare R2 nhằm tách tệp dung lượng lớn khỏi máy chủ ứng dụng, chuẩn hóa định dạng phát và hỗ trợ nhiều học sinh xem cùng lúc.

5.3.1 Bài toán và vấn đề

Trong mô hình dạy học livestream, giáo viên thường có nhiều video ghi lại bài giảng. Một video ôn thi có thể kéo dài hàng chục phút đến vài giờ, dung lượng lớn và được nhiều học sinh xem lại trong cùng một giai đoạn cao điểm trước kỳ thi. Nếu backend lưu trực tiếp các video này trên ổ đĩa máy chủ và truyền nguyên tệp video cho từng học sinh, hệ thống sẽ gặp áp lực lớn về dung lượng, băng thông và số kết nối kéo dài. Khi nhiều học sinh cùng xem, backend dễ bị nghẽn ở phần truyền media, trong khi các API khác như đăng nhập, ghi nhận tiến độ hoặc hỏi AI vẫn cần phản hồi nhanh.

Ngoài vấn đề tải đồng thời, phát video dài trực tiếp từ một tệp nguồn cũng làm trải nghiệm học tập kém ổn định. Học sinh thường tua đến đúng đoạn cần ôn lại, học trên mạng yếu hoặc chuyển giữa nhiều thiết bị. Nếu trình duyệt phải xử lý một tệp video lớn, thao tác tua và tải lại sau gián đoạn sẽ nặng hơn. Vì vậy, hệ thống cần chuyển video sang định dạng phù hợp với phát trực tuyến, chia nhỏ nội dung để trình duyệt tải từng phần, đồng thời đưa tệp media ra khỏi backend ứng dụng.

5.3.2 Giải pháp

Giải pháp được triển khai theo hướng pipeline xử lý bất đồng bộ. Backend không coi video là dữ liệu nghiệp vụ thông thường, mà quản lý video qua metadata, trạng thái xử lý và đường dẫn object. Nội dung video được lưu trong kho đối tượng, còn

backend tập trung vào xác thực, kiểm tra quyền học và cung cấp đường truy cập hợp lệ.

a, Tách metadata và object. Khi người quản lý nội dung upload video, hệ thống tạo bản ghi `media` để lưu loại media, trạng thái, object key, MIME type, dung lượng, thời lượng và người tải lên. Nội dung tệp được lưu trên Cloudflare R2, một dịch vụ lưu trữ đối tượng tương thích S3. Backend chỉ giữ object key và metadata cần thiết để quản lý media. Cách thiết kế này giúp backend không phụ thuộc vào ổ đĩa cục bộ để lưu video lâu dài.

b, Khóa media trước khi xử lý. Sau khi video được upload thành công, hệ thống bắt đầu quá trình chuyển đổi HLS. Trước khi chạy FFmpeg, dịch vụ transcode gọi thao tác khóa media. Nếu media đã bị xóa, không phải video hoặc đã có tiến trình xử lý khác lấy được khóa, dịch vụ dừng lại. Cơ chế này tránh tình huống cùng một video bị xử lý song song, gây ghi đè playlist hoặc tạo nhiều kết quả không nhất quán.

c, Chuyển đổi sang HLS. Dịch vụ xử lý tạo thư mục tạm riêng cho media, tải video nguồn từ R2 về máy chủ, sau đó dùng FFprobe để đọc thời lượng. Tiếp theo, FFmpeg được gọi để chuyển video sang HLS với codec video H.264, audio AAC, thời lượng segment 6 giây và playlist dạng VOD. Kết quả gồm một tệp `index.m3u8` và nhiều segment `segment_xxx.ts`. Playlist mô tả thứ tự các segment, còn trình duyệt chỉ tải những segment cần phát tại thời điểm hiện tại. Khi học sinh tua video, trình phát có thể chuyển đến nhóm segment tương ứng thay vì tải lại toàn bộ tệp nguồn.

d, Upload kết quả và cập nhật trạng thái. Sau khi FFmpeg xử lý xong, hệ thống upload toàn bộ playlist và segment lên R2 trong thư mục HLS riêng của media. Nếu thành công, bản ghi media được cập nhật sang trạng thái sẵn sàng, lưu object key của playlist và thời lượng video. Nếu bất kỳ bước nào lỗi, media được đánh dấu thất bại. Dù thành công hay thất bại, hệ thống đều xóa thư mục tạm để tránh làm đầy ổ đĩa máy chủ.

e, Phục vụ video qua route có kiểm soát. Khi học sinh học bài, frontend không truy cập tùy ý vào toàn bộ bucket. Backend cung cấp route lấy playlist và segment HLS. Route này chỉ chấp nhận tên tệp đúng mẫu playlist hoặc segment. Với playlist và segment, backend trả về đúng kiểu nội dung tương ứng để trình phát video xử lý. Việc giới hạn tên tệp giúp tránh truy cập ngoài phạm vi video bài học, đồng thời tạo điểm kiểm soát để kết hợp kiểm tra quyền học.

Về mặt tải hệ thống, HLS giúp mỗi lượt xem được chia thành nhiều request nhỏ theo từng segment. Thiết kế này phù hợp với cache và CDN hơn so với truyền một

tệp video lớn qua backend. Trong phạm vi đề án, Cloudflare R2 được dùng để lưu trữ object lâu dài; khi triển khai thực tế, các segment HLS có thể được phục vụ qua lớp cache ở gần người học hơn, giúp giảm độ trễ tải video và giảm áp lực băng thông cho máy chủ ứng dụng.

Ở mức API, route phát HLS có dạng:

```
GET /learning/lessons/:lessonId/hls/:fileName
```

Tham số `lessonId` xác định bài học đang xem, còn `fileName` chỉ được nhận hai dạng: `index.m3u8` hoặc `segment_xxx.ts`. Khi nhận request, service `getLearningLessonHlsFile` kiểm tra học sinh có quyền học bài đó, bài học có video hệ thống, media đã ở trạng thái `ready`, sau đó mới đọc object từ R2. Nếu `fileName` là playlist, backend sửa lại các đường dẫn segment trong playlist để các request tiếp theo vẫn đi qua route có kiểm soát. Nếu `fileName` là segment, backend stream nội dung segment về trình duyệt với `Content-Type` phù hợp.

5.3.3 Kết quả đạt được và đánh giá

Giải pháp HLS và lưu trữ đối tượng giúp hệ thống xử lý video theo hướng phù hợp hơn với vận hành thực tế. Backend không còn là nơi lưu giữ lâu dài các tệp video lớn, mà chỉ quản lý metadata, trạng thái và quyền truy cập. Video sau xử lý được chuẩn hóa thành playlist và segment, thuận lợi hơn cho phát trên trình duyệt, tua video và phục vụ nhiều lượt xem cùng lúc.

Quy trình này cũng có khả năng xử lý lỗi rõ ràng. Nếu nguồn video không tồn tại, FFmpeg lỗi hoặc không tạo được segment, media không bị đánh dấu sẵn sàng sai mà chuyển sang trạng thái thất bại. Người quản lý nội dung có thể nhận biết video chưa dùng được. Ngoài ra, cơ chế khóa media và cleanup thư mục tạm giúp tránh hai lỗi thường gặp trong xử lý media: chạy trùng job và để lại tệp tạm sau khi xử lý.

Các tiêu chí đánh giá giải pháp gồm: video upload chuyển sang trạng thái xử lý; sau khi transcode thành công có playlist và các segment trên R2; thời lượng video được cập nhật; trình phát tải được playlist và segment đúng kiểu nội dung; học sinh có thể tua đến các đoạn khác nhau của video; route học tập chỉ nhận playlist hoặc segment hợp lệ; media lỗi được đánh dấu `failed`; thư mục tạm được xóa sau xử lý. Các tiêu chí này phản ánh đúng rủi ro của luồng video bài giảng trong một LMS có doanh thu từ khóa học, thay vì chỉ kiểm tra việc upload một tệp thành công.

5.4 Tự động hóa ghi danh sau thanh toán qua webhook

Hệ thống cần giảm thao tác kiểm tra chuyển khoản và cấp quyền thủ công khi bán khóa học. SePay cung cấp *webhook* để chuyển thông tin giao dịch tới ứng dụng ngay khi phát sinh, đồng thời hỗ trợ HMAC-SHA256 và API Key [30]. Trên nền cơ chế này, đồ án tổ chức luồng đơn hàng, VietQR, chống yêu cầu trùng, đối soát giao dịch và tự động ghi danh.

5.4.1 Bài toán và vấn đề

Thanh toán chuyển khoản trong lớp học trực tuyến phát sinh nhiều sai lệch. Học sinh có thể bấm mua nhiều lần do mạng chậm, chuyển thiếu tiền, chuyển sau khi đơn hết hạn hoặc nhập sai mã hóa đơn. Nhà cung cấp cũng có thể gửi lại *webhook* khi lần gửi trước chưa nhận được phản hồi. Nếu hệ thống cấp quyền cho mọi sự kiện có trạng thái thành công, khóa học có thể bị cấp sai; nếu chỉ kiểm tra thủ công, mục tiêu tự động hóa không đạt được.

Bài toán cần giải quyết là tự động hóa nhưng vẫn kiểm soát được sai lệch. Hệ thống phải hạn chế yêu cầu trùng, chỉ ghi danh khi giao dịch khớp và giữ lại dữ liệu để quản trị viên xử lý trường hợp bất thường. Việc cập nhật thanh toán, hoàn tất đơn hàng và tạo bản ghi ghi danh phải nhất quán; không được xảy ra tình trạng đơn hàng đã hoàn thành nhưng chưa có quyền học hoặc ngược lại.

5.4.2 Giải pháp

Giải pháp được triển khai theo mô hình đơn hàng trước, thanh toán sau. Mọi giao dịch thanh toán đều được đối chiếu với một đơn hàng đã tồn tại trong hệ thống.

a, Chống yêu cầu trùng bằng khóa bất biến. Các thao tác tạo thanh toán và hủy đơn đều yêu cầu trường *Idempotency-Key*. Ở lần gọi đầu, hệ thống lưu khóa cùng giá trị băm của phương thức, đường dẫn và nội dung yêu cầu. Nếu cùng khóa và nội dung được gửi lại sau khi xử lý xong, kết quả đã lưu sẽ được trả về thay vì thực hiện nghiệp vụ lần nữa. Hệ thống từ chối trường hợp cùng khóa nhưng khác nội dung; yêu cầu đang xử lý cũng không được chạy đồng thời lần thứ hai.

Cơ chế này cần thiết trong luồng mua khóa học vì người dùng có thể bấm nút thanh toán nhiều lần khi mạng chậm. Nếu chỉ dựa vào ràng buộc duy nhất ở CSDL, các yêu cầu trùng vẫn đi qua nhiều bước nghiệp vụ trước khi bị chặn. Khóa bất biến giúp phát hiện sớm và trả về kết quả ổn định cho giao diện.

b, Tạo đơn hàng và mã hóa đơn. Khi học sinh mua khóa học, hệ thống loại bỏ các khóa học trùng trong request, kiểm tra khóa học đã xuất bản, kiểm tra học sinh chưa ghi danh các khóa học đó và tính tổng tiền. Trước khi tạo đơn mới, hệ thống làm hết hạn các đơn chờ quá thời gian và kiểm tra xem học sinh có đơn pending

còn hiệu lực hay không. Nếu có, hệ thống trả về đơn đang chờ thay vì tạo thêm đơn mới. Cách làm này tránh việc một học sinh tạo nhiều đơn chờ song song cho cùng một nhu cầu mua.

Mỗi đơn hàng có mã hóa đơn duy nhất `order_invoice_number`. Khi tạo thanh toán qua SePay, backend sinh mã VietQR gồm số tài khoản, ngân hàng, số tiền và nội dung chuyển khoản là mã hóa đơn. Mã hóa đơn là dữ liệu quan trọng để webhook ngân hàng có thể ghép giao dịch với đơn hàng.

c, Xác thực và chuẩn hóa *webhook*. Khi SePay gửi sự kiện, bộ chuyển đổi của nhà cung cấp kiểm tra API key hoặc chữ ký HMAC. Với HMAC, thời điểm gửi được đối chiếu để hạn chế phát lại sự kiện cũ, sau đó chữ ký được so sánh bằng `timingSafeEqual`. Dữ liệu hợp lệ được chuẩn hóa thành sự kiện thanh toán nội bộ gồm mã sự kiện, mã giao dịch, mã hóa đơn, số tiền, đơn vị tiền tệ, hướng giao dịch, trạng thái và thời điểm thanh toán.

Dữ liệu *webhook* từ SePay chứa các trường gắn với giao dịch ngân hàng như mã sự kiện, ngân hàng, thời gian, số tài khoản, mã đơn hàng, hướng chuyển tiền, số tiền và mã tham chiếu. Bộ chuyển đổi tách các dịch vụ nghiệp vụ khỏi cấu trúc dữ liệu gốc bằng cách chuẩn hóa về sự kiện nội bộ:

```
{
  "provider": "sepay",
  "eventId": "...",
  "transactionRef": "...",
  "orderInvoiceNumber": "...",
  "amount": 200000,
  "currency": "VND",
  "status": "success",
  "direction": "in",
  "paidAt": "..."
}
```

Trong đó, `orderInvoiceNumber` được lấy từ trường `code` của SePay và dùng để ghép giao dịch với đơn hàng. Nhờ bước chuẩn hóa, dịch vụ đối soát chỉ làm việc với một cấu trúc ổn định, ngay cả khi nhà cung cấp thay đổi tên trường hoặc hệ thống bổ sung kênh thanh toán khác.

d, Lưu sự kiện *webhook* và giao dịch thanh toán. Trước khi đối chiếu, hệ thống lưu sự kiện do nhà cung cấp gửi và tạo bản ghi giao dịch thanh toán. Dữ liệu sự kiện được dùng để chống xử lý lặp và theo dõi trạng thái tiếp nhận *webhook*; bản ghi giao dịch lưu thông tin dòng tiền để đối soát, tìm kiếm và xử lý thủ công.

Việc tách hai loại dữ liệu giúp hệ thống không mất dấu các giao dịch bất thường.

e, Đối soát giao dịch. Sau khi tạo bản ghi, hệ thống phân loại giao dịch theo điều kiện nghiệp vụ. Sự kiện không phải giao dịch tiền vào hoặc không có trạng thái thành công được đánh dấu `ignored`. Trường hợp thiếu mã hóa đơn hoặc không tìm thấy đơn hàng nhận trạng thái `unmatched`; sai lệch đơn vị tiền tệ được chuyển sang diện kiểm tra thủ công.

Với đơn hàng tìm thấy, hệ thống tiếp tục kiểm tra trạng thái thanh toán. Nếu khoản thanh toán không còn ở trạng thái chờ (`pending`), giao dịch được chuyển sang diện kiểm tra thủ công (`manual_review`) với mã lý do `payment_not_pending`. Trường hợp số tiền nhận được nhỏ hơn tổng tiền đơn hàng cũng được chuyển sang diện này với mã `under_paid`. Giao dịch đến sau khi đơn hàng hết hạn hoặc không còn chờ xử lý được ghi nhận bằng trạng thái `late_success`. Hệ thống chỉ xác nhận giao dịch khi khoản thanh toán đang chờ, số tiền đã đủ và đơn hàng còn hiệu lực.

Bảng 5.1: Các nhóm kết quả đối chiếu giao dịch thanh toán

Trạng thái	Ý nghĩa
<code>matched</code>	Giao dịch khớp với đơn hàng hợp lệ, hệ thống hoàn tất đơn hàng và cấp quyền học.
<code>unmatched</code>	Giao dịch thiếu mã hóa đơn hoặc không tìm thấy đơn hàng tương ứng.
<code>manual_review</code>	Giao dịch cần quản trị viên kiểm tra lại, ví dụ thiếu tiền, đơn hàng hết hạn hoặc payment không còn ở trạng thái chờ.
<code>ignored</code>	Giao dịch không phải tiền vào hoặc không phải trạng thái thành công nên không tham gia cấp quyền học.

f, Hoàn tất đơn hàng trong transaction. Khi giao dịch hợp lệ, hệ thống cập nhật payment sang `success`, cập nhật payment transaction sang `matched`, hoàn tất đơn hàng và tạo các bản ghi `enrollments` cho các khóa học trong đơn. Toàn bộ thao tác được thực hiện trong transaction CSDL. Nếu một bước lỗi, transaction rollback, tránh tình trạng dữ liệu thanh toán và quyền học lệch nhau.

Hệ thống xử lý *webhook* gửi lặp bằng ràng buộc duy nhất theo nhà cung cấp và mã sự kiện. Nếu một sự kiện đã xử lý được gửi lại, máy chủ trả về kết quả cũ và không tạo thêm bản ghi ghi danh.

5.4.3 Kết quả đạt được và đánh giá

Giải pháp giúp tự động hóa phần lớn luồng ghi danh sau thanh toán. Với giao dịch khớp chính xác, học sinh được cấp quyền học mà không cần giáo viên hoặc

trợ lý kiểm tra biên lai thủ công. Với giao dịch bất thường, hệ thống không tự động cấp quyền nhưng vẫn lưu đủ dữ liệu để quản trị viên rà soát.

Giải pháp cũng cải thiện tính nhất quán dữ liệu. Cấp quyền học không phải một thao tác rời rạc, mà gắn với trạng thái đơn hàng và payment trong cùng transaction. Điều này phù hợp với yêu cầu thực tế của hệ thống thương mại hóa khóa học: học sinh chỉ được ghi danh khi đơn hàng đã hoàn tất hợp lệ.

Khóa bất biến giữ thao tác mua khóa học nhất quán khi người dùng bấm lập hoặc trình duyệt gửi lại yêu cầu do mạng chậm. Dữ liệu sự kiện và bản ghi giao dịch cho phép quản trị viên phân biệt giao dịch khớp, thiếu mã hóa đơn, đến muộn hoặc thiếu tiền. Quyền học chỉ được cấp khi giao dịch thỏa mãn đầy đủ điều kiện nghiệp vụ.

Các trường hợp kiểm thử gồm: (i) gửi lập yêu cầu tạo đơn với cùng khóa trả lại cùng kết quả; (ii) dùng cùng khóa cho nội dung khác bị từ chối; (iii) không tạo đơn mới khi còn đơn chờ hiệu lực; (iv) *webhook* hợp lệ tạo bản ghi ghi danh; (v) sự kiện gửi lập không tạo dữ liệu trùng; và (vi) giao dịch thiếu mã, thiếu tiền, đến muộn hoặc chi tiền không tự động cấp quyền. Nhóm kiểm thử này bao phủ cả luồng thành công và các sai lệch thường gặp khi xác nhận chuyển khoản.

Chương này đã phân tích bốn bài toán kỹ thuật tiêu biểu trong quá trình hiện thực sản phẩm: hỏi đáp theo học liệu có nguồn dẫn, kiểm soát quá trình làm bài, xử lý video HLS kết hợp lưu trữ đối tượng và tự động hóa ghi danh qua *webhook*. Giá trị của các giải pháp không nằm ở từng công nghệ riêng lẻ, mà ở việc chúng cùng phục vụ một quy trình LMS thống nhất và xử lý các trạng thái lỗi thường gặp trong vận hành thực tế. Chương tiếp theo tổng kết phạm vi sản phẩm đã hoàn thành, các hạn chế còn tồn tại và hướng phát triển của hệ thống.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã xuất phát từ bài toán vận hành dạy học trực tuyến còn phân tán ở các nhóm giáo viên và trung tâm nhỏ: livestream ở một nền tảng, tài liệu gửi qua kênh khác, video cũ khó tìm lại, đăng ký khóa học cần nhiều thao tác thủ công và giáo viên khó theo dõi quá trình học tập của học sinh. Trên cơ sở đó, đề tài đã xây dựng một hệ thống quản lý học tập trực tuyến tích hợp AI nhằm tập trung các hoạt động chính của một khóa học vào cùng một nền tảng.

Về chức năng, hệ thống đã triển khai các nhóm nghiệp vụ gồm quản lý người dùng; khóa học theo cấu trúc khóa học-chương-bài; upload và xử lý media; học bài và xem tài liệu; theo dõi danh sách, tiến độ chi tiết của từng học sinh; tạo, tổ chức, làm và chấm bài kiểm tra; thông báo; blog; mua khóa học; xử lý thanh toán và cấp quyền học. Dịch vụ AI hỗ trợ hỏi đáp theo học liệu nội bộ bằng RAG, trong đó câu trả lời được tạo từ các đoạn tài liệu truy xuất và trả về nguồn tham chiếu.

Về mặt kỹ thuật, đồ án đã áp dụng kiến trúc phân lớp để tách giao diện, xử lý nghiệp vụ, truy cập dữ liệu và các tích hợp hạ tầng. Backend được tổ chức theo các module nghiệp vụ, frontend được chia theo các nhóm chức năng và dịch vụ AI được tách riêng để xử lý ingest tài liệu, embedding, truy xuất ngữ cảnh và sinh câu trả lời. Cơ sở dữ liệu PostgreSQL kết hợp pgvector được sử dụng để lưu dữ liệu nghiệp vụ và dữ liệu vector, trong khi Cloudflare R2 được dùng để lưu trữ media dung lượng lớn. Các thành phần đã được đóng gói thành Docker image, triển khai trên VPS qua HTTPS và kiểm tra migration, healthcheck, đăng nhập cùng sao lưu-phục hồi CSDL. Benchmark RAG trên 25 câu của khóa HSG “HẠT” đạt Recall@6 bằng 0,771, MRR bằng 0,746 và từ chối đúng 5/5 câu ngoài tài liệu.

So với các công cụ được khảo sát ở Chương 2, hệ thống không đặt mục tiêu thay thế những nền tảng lớn như Moodle, Google Classroom hoặc Udemy. Điểm khác biệt của đồ án nằm ở việc kết hợp các nhu cầu thường bị tách rời trong mô hình dạy học trực tuyến quy mô nhỏ: đóng gói học liệu thành khóa học có cấu trúc, tự động hóa mua khóa học và cấp quyền học, quản lý bài kiểm tra, theo dõi học tập và hỗ trợ hỏi đáp theo học liệu riêng. Cách tiếp cận này phù hợp với nhóm giáo viên tự do hoặc trung tâm nhỏ muốn tự chủ hơn về nội dung, dữ liệu học viên và quy trình vận hành.

Đồ án còn bốn hạn chế chính: (i) môi trường public mới được kiểm tra ở mức triển khai và luồng chức năng, chưa đo tải, kiểm thử bảo mật chuyên sâu hoặc đánh

giá với người dùng thật; (ii) benchmark RAG mới gồm 25 câu thuộc một khóa học và hai bài học, chưa có tập kiểm thử độc lập đủ đại diện cho nhiều môn học và dạng tài liệu; (iii) video HLS mới sử dụng một mức chất lượng và chưa có hàng đợi xử lý bền vững; (iv) mô hình phân quyền hiện phù hợp với một đơn vị vận hành, chưa hỗ trợ nhiều trung tâm và các vai trò chuyên biệt như trợ giảng hoặc kế toán.

Qua quá trình thực hiện đồ án, sinh viên đã rút ra một số bài học quan trọng. Khi xây dựng một hệ thống có nhiều nghiệp vụ liên quan, việc thiết kế ranh giới giữa service, repository, dữ liệu và tích hợp bên ngoài cần được thực hiện sớm để tránh mã nguồn bị trộn trách nhiệm. Với các chức năng có yếu tố AI, chất lượng dữ liệu đầu vào, cách truy xuất ngữ cảnh và cơ chế từ chối khi thiếu thông tin quan trọng không kém việc gọi mô hình ngôn ngữ. Với các chức năng vận hành như thanh toán và media, hệ thống cần chú ý nhiều đến trạng thái lỗi, xử lý lặp và khả năng khôi phục, thay vì chỉ xử lý luồng thành công.

6.2 Hướng phát triển

Ưu tiên trước mắt là hoàn thiện môi trường triển khai thực tế với HTTPS, reverse proxy, sao lưu CSDL và giám sát tài nguyên. Khi hạ tầng ổn định, hệ thống cần được đo thời gian tải trang, thời gian phản hồi API, thời gian xử lý video, độ ổn định của webhook và khả năng phục vụ nhiều học sinh đồng thời.

Trải nghiệm học tập có thể được mở rộng bằng lịch học, nhắc học theo lịch, phân tích kết quả theo chủ đề và gợi ý nội dung ôn tập dựa trên điểm yếu. Phân hệ bài kiểm tra cần bổ sung trộn đề, phân tích độ khó, độ phân biệt của câu hỏi và cơ chế giám sát phù hợp hơn thay vì chỉ dựa vào số lần rời màn hình.

Đối với AI Tutor, bước tiếp theo là mở rộng benchmark sang nhiều khóa học và môn học, đồng thời tách tập dữ liệu dùng điều chỉnh cấu hình khỏi tập kiểm thử cuối. Kết quả hiện tại cho thấy cần ưu tiên truy xuất kết hợp vector và từ khóa, đa dạng hóa tài liệu trong top-K, bổ sung reranking và tăng mức tuân thủ citation. Các khả năng như gợi ý bài ôn tập, tóm tắt bài giảng, giải thích lỗi sai hoặc tạo câu hỏi nháp chỉ nên được bổ sung khi có cơ chế kiểm duyệt chuyên môn.

Khi phục vụ nhiều trung tâm, kiến trúc phân quyền cần tách dữ liệu khóa học, học sinh, đơn hàng và báo cáo theo từng đơn vị. Mô hình này cũng cần các vai trò như trợ giảng, nhân viên vận hành, kế toán và quản lý trung tâm.

Phần media cần hỗ trợ nhiều mức chất lượng HLS, CDN, đường dẫn truy cập có thời hạn, hàng đợi xử lý video và cơ chế chạy lại công việc thất bại. Với thanh toán, các chức năng còn thiếu gồm đối soát định kỳ, hoàn tiền và xử lý giao dịch bất thường.

Đồ án đã hoàn thành các luồng cốt lõi của một LMS tích hợp bán khóa học, theo dõi học tập, đánh giá và hỏi đáp theo học liệu nội bộ. Kết quả được xác nhận ở mức mã nguồn, kiểm thử nội bộ và môi trường triển khai public; mức sẵn sàng sản phẩm ở quy mô lớn chỉ có thể được đánh giá sau khi đo tải, kiểm thử bảo mật và thử nghiệm với người dùng thật.

TÀI LIỆU THAM KHẢO

- [1] Google, *Get started with google classroom*, <https://support.google.com/edu/classroom/answer/6020279>, Truy cập ngày 29/06/2026, 2026.
- [2] Google, *Learn about gemini in classroom*, <https://support.google.com/edu/classroom/answer/15410566>, Truy cập ngày 29/06/2026, 2026.
- [3] Moodle Pty Ltd, *Ai tools - moodledocs*, https://docs.moodle.org/502/en/AI_subsystem, Truy cập ngày 29/06/2026, 2026.
- [4] Udemy, Inc., *Udemy ai assistant: Frequently asked questions*, <https://support.udemy.com/hc/en-us/articles/27554582323863-Udemy-AI-Assistant-Frequently-Asked-Questions>, Truy cập ngày 29/06/2026, 2026.
- [5] OpenJS Foundation, *Node.js documentation*, <https://nodejs.org/docs/latest/api/>, Truy cập ngày 06/06/2026, 2026.
- [6] OpenJS Foundation, *Express - node.js web application framework*, <https://expressjs.com/en/>, Truy cập ngày 06/06/2026, 2026.
- [7] Microsoft, *Typescript documentation*, <https://www.typescriptlang.org/docs/>, Truy cập ngày 06/06/2026, 2026.
- [8] PostgreSQL Global Development Group, *Postgresql documentation*, <https://www.postgresql.org/docs/>, Truy cập ngày 06/06/2026, 2026.
- [9] Prisma Data, Inc., *Prisma documentation*, <https://www.prisma.io/docs>, Truy cập ngày 06/06/2026, 2026.
- [10] Zod, *Zod documentation*, <https://zod.dev/>, Truy cập ngày 06/06/2026, 2026.
- [11] M. Jones, J. Bradley, and N. Sakimura, “Json web token (JWT),” Internet Engineering Task Force, Tech. Rep. RFC 7519, 2015, Truy cập ngày 06/06/2026. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [12] Cloudflare, *Cloudflare r2 s3 api compatibility*, <https://developers.cloudflare.com/r2/api/s3/api/>, Truy cập ngày 06/06/2026, 2026.
- [13] Amazon Web Services, *Amazon s3 documentation*, <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>, Truy cập ngày 06/06/2026, 2026.

- [14] FFmpeg, *Ffprobe documentation*, <https://ffmpeg.org/ffprobe.html>, Truy cập ngày 06/06/2026, 2026.
- [15] FFmpeg, *Ffmpeg documentation*, <https://ffmpeg.org/ffmpeg.html>, Truy cập ngày 06/06/2026, 2026.
- [16] R. Pantos and W. May, “Http live streaming,” Internet Engineering Task Force, Tech. Rep. RFC 8216, 2017, Truy cập ngày 06/06/2026. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8216>
- [17] Meta Platforms, Inc., *React documentation*, <https://react.dev/>, Truy cập ngày 06/06/2026, 2026.
- [18] Vercel, Inc., *Next.js documentation*, <https://nextjs.org/docs>, Truy cập ngày 06/06/2026, 2026.
- [19] TanStack, *Tanstack query documentation*, <https://tanstack.com/query/latest/docs/framework/react/overview>, Truy cập ngày 06/06/2026, 2026.
- [20] Axios, *Axios documentation*, <https://axios.rest/pages/getting-started/first-steps>, Truy cập ngày 06/06/2026, 2026.
- [21] pmndrs, *Zustand documentation*, <https://zustand.docs.pmnd.rs/>, Truy cập ngày 06/06/2026, 2026.
- [22] React Hook Form, *React hook form documentation*, <https://react-hook-form.com/docs>, Truy cập ngày 06/06/2026, 2026.
- [23] Tailwind Labs, *Tailwind css documentation*, <https://tailwindcss.com/docs>, Truy cập ngày 06/06/2026, 2026.
- [24] Video.js, *Video.js documentation*, <https://videojs.org/guides/react/>, Truy cập ngày 06/06/2026, 2026.
- [25] Tiptap, *Tiptap documentation*, <https://tiptap.dev/docs/editor/getting-started/install/react>, Truy cập ngày 06/06/2026, 2026.
- [26] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/7181-attention-is-all-you-need>
- [27] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>

- [28] M. Richards, *Software Architecture Patterns*. O'Reilly Media, 2015, Truy cập ngày 06/06/2026. [Online]. Available: <https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/>
- [29] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002, Truy cập ngày 06/06/2026. [Online]. Available: <https://martinfowler.com/books/ea.html>
- [30] SePay, *Hướng dẫn tích hợp webhook*, <https://docs.sepay.vn/tich-hop-webhooks.html>, Truy cập ngày 29/06/2026, 2026.